

Software Project Management

Software Project Management

Lec-1

What is a project?

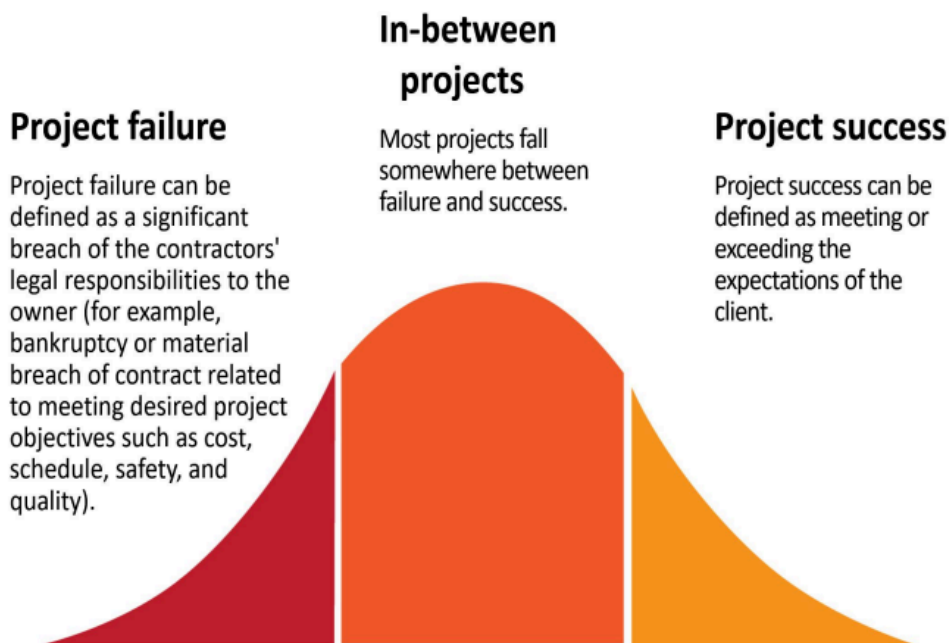
-A project is “a piece of planned work or activity that is completed over a period of time and intended to achieve a particular aim” Projects are by definition ephemeral; they come and go, depending on an organization’s needs, and eventually come to a close.

Project Performance

Success/failure $\neq$ just cost + schedule. On-time + budget project can still miss expectations.

Project Failure vs Success

- **Failure:** major breach of contractor's legal duty to owner (bankruptcy, contract breach re: cost/schedule/safety/quality).
- **In-between:** most projects land here.
- **Success:** meets/exceeds client expectations.



Project Life Cycle (3 stages, cyclic)

- **Making Stage:** cost, time, quality, safety.
- **Using Stage:** suitability of purpose.
- **Operating Stage:** effectiveness, cost.
- Then → **Retirement/Reuse Stage** → cycle repeats.

Project Outcome

Measurable output: scope, cost, schedule, quality, safety. Broader sense: impact vs larger goals.

Project Success (def)

Degree to which project is done well. Subjective — stakeholders judge diff, based on their own expectations/role.

Success vs Failure (quick)

- Success: on time, in budget, meets user needs + quality.
- Failure: missed deadlines, cost overrun, poor satisfaction.
- CHAOS Report: 50%+ projects face major challenges.

Sustainability & Living Order

Construction industry pushing sustainability. Software analog: write code that runs on old hardware → less e-waste.

Characteristics of Software Project

- Intangible output (software itself)
- Needs innovation + problem-solving
- Complex, dynamic, team-based
- Requirements + tech evolve mid-project

Software Project Environment

Technical + organizational + social context. Shaped by policy, culture, stakeholder expectations. Needs strong comms + leadership.

Project Management Process Groups (5)

Initiating → Planning → Executing → Monitoring & Controlling → Closing

Scheduling Concepts

- **Gantt Chart:** visual task timeline
- **Milestone:** major event marker
- **Dependency:** relation between tasks (order constraint)
- **Critical Path:** sequence of tasks that sets total project duration

Stakeholders

Project Manager, Dev Team, Clients/Users, Sponsors/Execs, Regulatory Bodies.
Alignment across all = key to success.

Project Planning Basics

- Set objectives + deliverables
- **WBS** (Work Breakdown Structure): split work into manageable chunks
- Estimate time, cost, resources
- **Risk Management:** find + mitigate uncertainty
- **Quality Management:** ensure output meets requirements
- Balance: speed vs cost vs performance

Key Tools

- Gantt chart, PERT chart
- **CPM** (Critical Path Method)
- PM software: MS Project, Jira, Trello
- Regular review + reporting

Managing & Developing Modern Software —

Lecture 2

Core Definitions

SE 1.0 (Codeware): Software built by humans manually writing rules/logic/algorithms in languages like C, C++, Java. Logic-based, deterministic, rule-driven. High manual effort, no learning capability.

SE 2.0 (Neuralware): Software trained (not programmed) — ML/DL models learn patterns from data. Learning-based, probabilistic, data-driven. Devs design architectures + supply datasets instead of writing direct logic.

SE 3.0 (Promptware): Software customized/fine-tuned from pre-trained Foundation Models (FMs, e.g. GPT) via prompt engineering instead of training from scratch. Bridges AI + SE via prompt-based design.

Software 4.0 (Agentware): Orchestrated layer of Agent Apps built atop FMware (Promptware/Neuralware/Codeware combined). Highest customization — agents coordinate across S1–S3 layers.

FATE: Fairness, Accountability, Transparency, Ethics — compliance framework for ML systems.

- **Fairness:** mitigate algorithmic bias, no discrimination (race/gender/age), equitable outcomes.
- **Accountability:** clear ownership/responsibility for AI actions, failures, harms.
- **Transparency:** make AI decision-making understandable — document data provenance + model architecture.
- **Ethics:** broader moral principles — user well-being, privacy, misuse prevention, align w/ org + society values.

Comparative Analysis Tables (likely exam angle)

SE Generations Compared

Aspect	SE 1.0 Codeware	SE 2.0 Neuralware	SE 3.0 Promptware	SW 4.0 Agentware
Built by	Manual coding	Training on data	Prompting/fine-tuning FM	Orchestrating agents over FMs
Nature	Deterministic	Probabilistic	Probabilistic + adaptable	Autonomous, multi-step
Dev input	Explicit logic	Architecture + datasets	Prompts + fine-tune	Agent design + orchestration
Examples	Websites, DBs, apps	Image/speech recog, recommenders	ChatGPT, Copilot, Bard, LLaMA	Agent apps (multi-agent orchestrated systems)
Effort type	High manual coding	Data + model tuning effort	Prompt design effort	Orchestration/composition effort

SE (Traditional) vs ML (SE2.0) – Fundamental Differences

Dimension	Traditional SE	ML (SE 2.0)
Primary asset	Code	Data (discovery, versioning, provenance)
Complexity driver	Code logic	Data quality/complexity > code complexity
Required skills	PM, Developer, Tester	Above + deep ML knowledge (build/evaluate/tune models)

Component structure	Modular, isolated components	Entangled components – can affect each other during training/tuning (→ non-monotonic error behavior)
Versioning focus	Code versioning	Data versioning + model versioning

Case-based cue: if a scenario describes "components affecting each other unexpectedly after retraining" → Component Entanglement issue (SE2.0-specific, doesn't happen in classical SE).

Case-based cue: if scenario is about reproducibility failing because dataset changed silently → Data Provenance issue.

The ML Workflow – 9 Stages (case-based Qs often ask "which stage does X belong to")

Preparation

1. Model Requirements – define feasible/useful features, pick model type
2. Data Collection – source/integrate/collect datasets
3. Data Cleaning – remove noisy/inaccurate records
4. Data Labeling – assign ground truth labels (manual/crowd-sourced)

Modeling

5. Feature Engineering – extract/select informative features
6. Model Training – train/tune on clean labeled data
7. Model Evaluation – test on held-out/safeguard data + human eval

Operations

8. Model Deployment — push inference code to target device(s)

9. Model Monitoring — continuous error monitoring in production

Note: loop-backs exist — Model Training $\leftarrow$ Feature Engineering (retrain w/ new features); Model Monitoring/Deployment issues can loop back to earlier stages.

Case-based cue: "model works in test but degrades after a few months in production" $\rightarrow$ Model Monitoring gap (stage 9), possibly data drift.
Case-based cue: "labels inconsistent across annotators" $\rightarrow$ Data Labeling issue (stage 4).

Essential Challenges & Best Practices (SE 2.0) — grouped, likely comparative/matching Qs

1. **Data Availability/Collection/Cleaning/Mgmt:** data quality & availability, provenance & evolution, aggregation, feature extraction/synthesis.
2. **End-to-End Pipeline Support:** iterative experimentation, integration & automation, UX & monitoring, dashboards for model value.
3. **Model Evolution/Evaluation/Deployment:** rigorous testing (systematic eval, scorecards, automated test sets), quality/reliability (human-in-the-loop, integration & versioning).
4. **Model Debugging & Interpretability:** understanding failures, layered/tiered modular architecture, visualization techniques.
5. **Compliance (FATE):** ethical AI development, align with team practices/behavior.

Software Project Management in Software 3.0 Era — Lecture 3 Exam Notes

Core Definitions

SE 3.0 (Promptware): Software customized/fine-tuned via Foundation Models (FMs, e.g. GPT) using prompts instead of training from scratch. Devs/users interact via prompt engineering + fine-tuning. Examples: ChatGPT, Copilot, Bard, LLaMA.

Software 4.0 (Agentware): Orchestrated Agent Apps sitting atop FMware (S1/S2/S3 combined) — max customization layer.

Developing AIware — 3 approaches:

- **Prompt Engineering:** crafting input (question + example + formatting) to LLM to steer output.
- **Fine-tuning:** further training a pre-trained FM on task-specific data.
- **RAG (Retrieval Augmented Generation):** augment prompt with retrieved external data/context before generation.

Prompt Engineering (mechanics): Input = Question + Example + Formatting → LLM → AI Response.

Prompt Design Strategies (definitions — likely matching/case Qs)

Type	Definition	Trigger phrase pattern
Instructional Prompt	Direct command telling model exactly what to do	"Generate test cases for..."

Contextual Prompt	Supplies background/data/examples for grounding	"Given the following bug report/diff..."
Role-based Prompt	Assigns a persona to shape response style/expertise	"You are a Senior DevOps Engineer..."
Chain-of-Thought (CoT) Prompt	Asks for stepwise reasoning before conclusion	"Explain step-by-step how you would find root cause..."
System Prompt	Configures persistent/standing behavior across a session/agent	"System: You are 'CodeQA', always return answers in 3 parts..."

Case-based cue (how to identify type in a scenario):

- Scenario gives model a job title/persona → Role-based
- Scenario pastes logs/bug reports/code diffs as grounding → Contextual
- Scenario demands step-by-step hypothesis testing / reasoning chain → Chain-of-Thought
- Scenario sets a fixed permanent response format/behavior for all future outputs → System Prompt
- Scenario is just a one-off direct task command, no persona/context → Instructional

Best Practices in Prompt Engineering

Be specific (define output format + success criteria) → Add context (scope/tools/dataset) → Use examples (tone/structure/style) → Avoid ambiguity (test iteratively) → Evaluate responses (refine like debugging code).

Prompt Engineering in SDLC

- Prompts treated as versioned software artifacts (managed via Git or prompt libraries e.g. PromptHub, LangSmith).
- Used across SDLC: Requirement elicitation ("describe system constraints"), Testing/QA ("find logical errors in this code"), Deployment (system prompt defines agent behavior).
- Prompts = reusable project assets, like source code or test cases.

Case-based cue: scenario says "team stores prompts in Git, versions them, reuses across sprints" → Prompt Engineering in SDLC / prompts as software artifacts.

Testing & Refining Prompts — Evaluation Criteria (comparative Qs)

Criterion	Question it answers
Accuracy	Does output meet task requirements?
Consistency	Are results reproducible across runs?
Relevance	Does output align with context/constraints?

Refinement cycle: Prompt → Response → Review → Adjust (loop).

Metric Calculation — BLEU & ROUGE (exam-relevant, actual metrics given)

BLEU (Bilingual Evaluation Understudy)

- Measures precision: how much of the *generated* text overlaps with the *reference* text.
- Used in: machine translation, code generation.
- Higher BLEU → generated output closer to reference.
- Formula concept: $BLEU \approx (\text{overlapping n-grams between candidate \& reference}) / (\text{total n-grams in candidate})$ – i.e., **precision-oriented**.

ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

- Measures recall: how much of the *reference* text is captured by the *generated* output.
- Used in: summarization, report generation, explanation tasks.
- Higher ROUGE → better content coverage of reference.
- Formula concept: $ROUGE \approx (\text{overlapping n-grams between candidate \& reference}) / (\text{total n-grams in reference})$ – i.e., **recall-oriented**.

Comparative cue (likely exam trap): BLEU = precision-style (did I say too much extra / is generated text accurate to reference?). ROUGE = recall-style (did I capture everything from the reference?). If scenario = translation/code-gen quality check → BLEU. If scenario = summarization completeness check → ROUGE.

Case-based cue: "Model generated a summary; teacher wants to check if summary captured all key points from original document" → ROUGE (recall of reference content). Case-based cue: "Model translated a sentence; check how closely it matches professional reference translation" → BLEU (precision vs reference).

Ethical & Responsible Prompting

- Risks: bias, hallucination, data leakage from prompts.
- Must validate outputs against ethical standards + data governance policy.

- Avoid prompts that: encourage harmful/biased output, reveal private/copyrighted data.
- Part of responsible AI project management (ties back to FATE from Lecture 2 – Fairness/Accountability/Transparency/Ethics).

Fine-Tuning and RAG

FT def: adapt pre-trained model to specific task/domain via further training on curated dataset.

Training from scratch vs FT:

- Scratch: massive data, extreme compute, high fail risk, impractical for 99% projects
- FT: leverage existing knowledge, small dataset (100s-1000s), low cost/time, faster path

When NOT to fine-tune:

- privacy/regulatory conflict w/ pretrain data
- task needs different architecture/inductive bias
- must guarantee zero weight change → use inference-time adapter instead
- compute/latency budget too tight → use smaller/distilled model

Types:

- Full FT: update ALL weights. Thorough but expensive, full-size model copy.
- PEFT: update small subset/add new params. Cheap, fast, tiny adapter files (~50MB vs 15GB).

PEFT methods:

- LoRA: freeze base weights, inject small trainable low-rank matrices into layers (usually attention). Only these trained. Most popular/effective.
- QLoRA: LoRA + 4-bit quantized base model → big memory savings, adapters trained in higher precision (NF4/FP16).

LoRA vs QLoRA table:

Aspect	LoRA	QLoRA
Base precision	FP16/BF16	4-bit quantized
Memory	high savings	very high (3-5x more)
Accuracy	near full	near LoRA
Speed	good	slightly slower (dequant cost)
Hardware	multi-GPU for 13B+	single 24GB GPU for 65B possible
Use case	mid-size models	very large LLMs

FT workflow (5 steps, case-derivation likely):

1. Task Definition – classify target Q/requirement
2. Data Prep – collect+label examples w/ correct label
3. Model Selection – choose base (BERT-base, Llama-3-8B)
4. Tuning – run training, feed labeled data via LoRA
5. Evaluation – test on unseen data, check accuracy

Example case: sentiment classify (3 class) → BERT-base → 10k labeled reviews 80/10/10 split → metric: accuracy + macro-F1.

Key libs: HF transformers (load model), HF peft (apply LoRA), HF datasets (load/process data).

Building Alware project w/ LoRA — 3 pillars:

- Data Pipeline: project = data, need robust collection/clean/version (garbage in garbage out)
- Experiment Loop: track base model, dataset version, hyperparams, metrics per run
- Model Deployment: output = artifact (e.g. 50MB adapter), needs own deploy/serving strategy

PM phases: discovery → data collection → modeling → integration → testing → deployment → monitoring → maintenance

Challenges (comparison-style Q likely):

- Data & cost: labeling is #1 bottleneck; GPU time = biggest cost
- Versioning nightmare: track base model version + dataset version + code (git commit) + model artifact — all 4 needed for reproducibility
- Model drift: real-world changes → perf. decays over time
- Catastrophic forgetting: over-specialize → forget general knowledge
- Evaluation hard: accuracy not enough, need bias/fairness/robustness checks, define metrics upfront

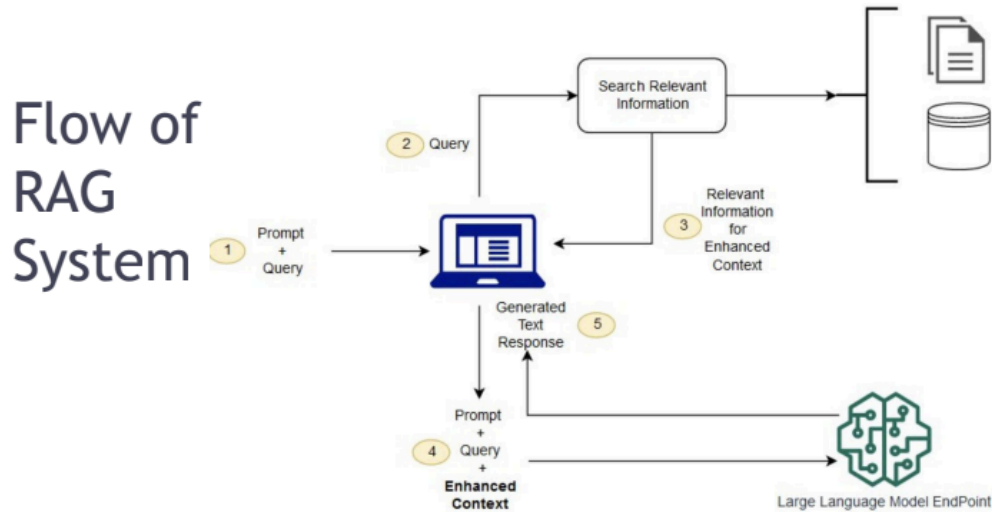
Risk categories: data risk (bad/insufficient labels), technical risk (scale/latency fail), ethical/legal (bias, privacy), operational (retrain cost, no monitoring). Mitigation: pilot, audit, staging, KPI/SLA.

RAG def: AI arch combine retrieval system + LLM. Gen response use model knowledge + externally retrieved up-to-date data. Splits knowledge mgmt (index+retrieve) from generation.

3 stages:

- **Indexing:** collect docs → chunk → embed → store in vector DB

- **Retrieval:** query → embedding → search index → rank/filter
- **Generation:** retrieved content injected into prompt → LLM generates grounded response → less hallucination



RAG vs Fine-Tuning (comparison table, memorize):

Aspect	RAG	FT
Idea	retrieve external knowledge at query time	embed knowledge into weights
Update	easy (change docs)	need retrain
Freshness	real-time capable	frozen at train time
Hallucination	lower	higher
Explainability	high (show sources)	low
Scalability	scales well	poor as data grows
Latency	higher (retrieval step)	lower
Best use	QA, enterprise search	style/domain lang adapt

Query translation (query reformulation before retrieval):

- Multi-query: rewrite orig query → multiple variants → retrieve all → ↑recall
- RAG-fusion: multi-query + merge/re-rank results via score aggregation, favor docs appear across queries
- Least-to-most: decompose complex Q → simple sub-Qs → retrieve stepwise, good for multi-hop

Query routing (decide which knowledge source):

- Logical: rule-based (keywords, metadata, role) → route to fixed pipeline
- Semantic: embed query, compare similarity → route to closest domain/tool

Types of RAG (know def each):

- Standard/Vector: embed docs as vectors, similarity search → gen
- Graph RAG: knowledge as entities+relations in graph, traverse for multi-hop reasoning
- Conversational RAG: maintains dialogue history/context across turns
- Multi-Source RAG: retrieve from docs+DB+APIs, aggregate+normalize before gen

Case-style Q likely: "system needs real-time policy + multi-hop reasoning across entities" → Graph RAG + routing. "chatbot remembering prior turns" → Conversational RAG.

BLEU Score

Definition: BLEU (Bilingual Evaluation Understudy) – metric measuring **precision**: how much AI-generated (candidate) text overlaps with human (reference) text. Common in machine translation, code generation. Higher BLEU → closer match to reference. Measures **syntactic similarity** (word/phrase overlap, not meaning).

Example Setup

- **Reference (R):** "The project manager approved the software release."
→ $r = 7$ words
- **Candidate (C):** "The manager approved the release of the software."
→ $c = 8$ words

Step 1: Compute n-gram overlaps

Unigram (1-word) precision:

- Ref unigrams: the, project, manager, approved, the, software, release
- Cand unigrams: the, manager, approved, the, release, of, the, software
- Matches: 6 out of 8 candidate words
- **$p1 = 6/8 = 0.75$**

Bigram (2-word) precision:

- Ref bigrams: the project, project manager, manager approved, approved the, the software, software release
- Cand bigrams: the manager, manager approved, approved the, the release, release of, of the, the software
- Matched bigrams (clipped count): "manager approved" (1), "approved the" (1), "the software" (1) → 3 matches out of 7 candidate bigrams
- **$p2 = 3/7 \approx 0.42$**

(Clipped count = don't count a match more times than it appears in the reference – prevents inflating precision by repeating a word/phrase.)

Step 2: Brevity Penalty (BP)

Purpose: penalizes candidates that are too *short* — a short output can fake high precision (few words, all matching) but is a poor translation overall. BP counteracts this.

Formula:

$$BP = \begin{cases} 1 & \text{if } c > r \\ e^{(1-r/c)} & \text{if } c \leq r \end{cases}$$

- Here $c = 8, r = 7 \rightarrow c > r \rightarrow \mathbf{BP} = 1$ (no penalty, candidate not shorter)

Step 3: Combine into BLEU-N score

Formula (BLEU-2 uses $n=1,2$ with uniform weights $w_1=w_2=0.5$):

$$BLEU-2 = BP \cdot \exp \left(\sum_{n=1}^2 w_n \ln(p_n) \right)$$

Plug in values:

$$BLEU-2 = 1 \cdot \exp(0.5 \ln(0.75) + 0.5 \ln(0.42)) = \exp(-0.57) \approx 0.56$$

Interpretation

BLEU ≈ 0.56 (56%) $\rightarrow$ AI sentence is moderately close to reference: good structural/word overlap, but not identical phrasing (word order and a couple word choices differ).

Quick recap of the 3-step method (useful for case questions)

1. **N-gram overlap** $\rightarrow$ compute precision p_n for each n (unigram p_1 , bigram p_2 , ... up to whatever BLEU-N is asked)

2. **Brevity Penalty** → compare candidate length c vs reference length r ;
 $BP=1$ if $c>r$, else exponential penalty
3. **Combine** → $BLEU-N = BP \times$ geometric mean of p_n 's (via exp of weighted sum of $\ln(p_n)$)

Class Task Solution: BLEU Score Calculation

Reference (R): "Sorts a list of numbers in ascending order." → tokens: Sorts, a, list, of, numbers, in, ascending, order → $r = 8$

Candidate (C): "Sorts numbers in increasing order." → tokens: Sorts, numbers, in, increasing, order → $c = 5$

Step 1: N-gram Overlaps

Unigram (1-word):

- Ref unigrams: Sorts, a, list, of, numbers, in, ascending, order
- Cand unigrams: Sorts, numbers, in, increasing, order
- Matches: Sorts ✓, numbers ✓, in ✓, increasing ✗ (ref has "ascending", not a match), order ✓
- Matches = 4 out of 5
- $p_1 = 4/5 = 0.8$

Bigram (2-word):

- Ref bigrams: Sorts a / a list / list of / of numbers / numbers in / in ascending / ascending order
- Cand bigrams: Sorts numbers / numbers in / in increasing / increasing order
- Matches: "Sorts numbers" ✗, "numbers in" ✓, "in increasing" ✗, "increasing order" ✗
- Matches = 1 out of 4
- $p_2 = 1/4 = 0.25$

Step 2: Brevity Penalty (BP)

- $c = 5, r = 8 \rightarrow c \leq r$, so use exponential formula:

$$BP = e^{(1-r/c)} = e^{(1-8/5)} = e^{-0.6} \approx 0.549$$

(Penalty applied since candidate is shorter than reference – it dropped words like "a list of".)

Step 3: BLEU-2 Score

$$BLEU-2 = BP \cdot \exp(0.5 \ln(p_1) + 0.5 \ln(p_2))$$

Plug in values:

- $\ln(0.8) = -0.223$
- $\ln(0.25) = -1.386$
- $0.5(-0.223) + 0.5(-1.386) = -0.805$
- $\exp(-0.805) \approx 0.447$

$$BLEU-2 = 0.549 \times 0.447 \approx 0.245$$

Result & Interpretation

$$BLEU-2 \approx 0.245 \text{ (24.5\%)}$$

Much lower than the earlier example (0.56). This makes sense: the candidate is shorter (dropped "a list of"), uses a different word ("increasing" vs "ascending"), so both the brevity penalty and n-gram precision drag the score down – indicating weaker overlap with the reference despite conveying similar meaning.

CodeBLEU Notes

Definition: CodeBLEU is a code-specific evaluation metric that extends BLEU. Instead of relying only on surface-level lexical overlap, it combines 4 components to better judge code similarity (since code has structure and semantics that plain text-BLEU can't capture).

Formula

$$\text{CodeBLEU} = w_{\text{NG}} \cdot \text{NG} + w_{\text{WNG}} \cdot \text{WNG} + w_{\text{AST}} \cdot \text{AST} + w_{\text{DF}} \cdot \text{DF}$$

Component	Meaning
NG	n-gram BLEU score – standard modified BLEU (lexical match)
WNG	weighted n-gram score – gives higher weight to important tokens like identifiers (variable/function names)
AST	syntax match – how similar the parsed Abstract Syntax Trees are (structure)
DF	data-flow match – how similar variable/data dependency flow is (semantics)

Worked Example

Reference (human): `def add(a, b):\n return a + b`

Candidate (AI): `def add(x, y):\n return x + y`

Step 1: Compute NG (standard BLEU part)

Tokenize both → 12 tokens each.

- Ref: `def, add, (, a, ,, b, ), :, return, a, +, b`
- Cand: `def, add, (, x, ,, y, ), :, return, x, +, y`

Unigram precision: matches (def, add, (, ,,), :, return, +) = 8 out of 12

→ $p_1 = 8/12 \approx 0.67$ (identifiers $a \neq x$, $b \neq y$ don't match)

Bigram precision: overlapping bigrams (def add, add (,) :, : return) = 4 out of 11

$$\rightarrow p_2 = 4/11 \approx 0.36$$

Geometric mean: $\sqrt{(0.67 \times 0.36)} \approx 0.49$

BP: candidate length = reference length $\rightarrow$ BP = 1

$$\rightarrow NG \approx 0.49$$

Step 2: Compute WNG (weighted n-gram)

Give identifiers extra weight (here weight = 2.0) since they matter more for code correctness; other tokens weight = 1.0.

- Matched weighted tokens (non-identifier matches): def, add, (, ,), :, return, + $\rightarrow 8 \text{ tokens} \times \text{weight } 1 = 8$
- Candidate total weighted tokens: 10 non-identifier tokens $\times 1 + 2$ identifiers $\times 2 = 10 + 4 = 14$

$$\text{WNG} = \text{matched_weight} / \text{candidate_total_weight} = 8/14 \approx 0.57$$

Step 3: Compute AST match score

Compare parsed tree shapes/node types (function def, params, return, binary op). Structure identical, only identifier names differ.

$$\rightarrow \text{AST} = 1.00 \text{ (perfect structural match)}$$

Step 4: Compute Data-Flow (DF) match score

Compare how values flow (param $\rightarrow$ used in return expression). Variable *names* differ but the *flow pattern* is identical.

$$\rightarrow \text{DF} = 0.95 \text{ (near-perfect equivalence)}$$

Step 5: Combine into CodeBLEU (equal weights, 0.25 each)

$$\text{CodeBLEU} = 0.25(0.49) + 0.25(0.57) + 0.25(1.0) + 0.25(0.95) \\ = 0.1225 + 0.1425 + 0.25 + 0.2375 \approx 0.75$$

CodeBLEU $\approx 0.75 \rightarrow$ judged a good result for code generation (structure/data-flow near-perfect even though variable names differ).

ROUGE Score — Notes

What is ROUGE

Recall-Oriented Understudy for Gisting Evaluation. Compares AI-generated (candidate) text against human-written reference text via n-gram overlap. Higher ROUGE = closer match to reference. Used for summarization, machine translation, code generation. Measures syntactic similarity.

BLEU vs ROUGE

Feature	BLEU	ROUGE
Main focus	Precision	Recall
Measures	How much of candidate is in reference	How much of reference is captured by candidate
Goal	Fluency/adequacy of generated text	Content coverage of reference
Denominator	n-gram count in candidate	n-gram count in reference

Steps to Compute ROUGE-N

1. Tokenize both reference (R) and candidate (C) sentences.
2. Extract n-grams from each (n=1 $\rightarrow$ unigram, n=2 $\rightarrow$ bigram, ...).

3. Count overlapping n-grams (clipped count – overlap capped at reference's count for that gram).
4. Precision = overlap / total n-grams in candidate
5. Recall = overlap / total n-grams in reference
6. $F1 = 2 \cdot P \cdot R / (P + R)$

Worked Example (from slides)

- Reference: "The project manager approved the software release." (r = 7 tokens)
- Candidate: "The manager approved the release of the software." (c = 8 tokens)

ROUGE-1 (unigram)

- Overlap tokens: the(2), manager, approved, release, software → total overlap = 6
- Precision = $6/8 = 0.75$
- Recall = $6/7 \approx 0.857$
- ROUGE-1 F1 ≈ 0.80

ROUGE-2 (bigram)

- Ref. bigrams: the project, project manager, manager approved, approved the, the software, software release (6 total)
- Cand. bigrams: the manager, manager approved, approved the, the release, release of, of the, the software (7 total)
- Overlaps: manager approved, approved the, the software → 3
- Precision = $3/7 \approx 0.4286$
- Recall = $3/6 = 0.5$
- ROUGE-2 F1 ≈ 0.462

Class Task – Solved

- Reference: "Sorts a list of numbers in ascending order." (8 tokens)

- Candidate: "Sorts numbers in increasing order." (5 tokens)

ROUGE-1

- Overlap: Sorts, numbers, in, order → 4
- Precision = $4/5 = 0.80$
- Recall = $4/8 = 0.50$
- ROUGE-1 F1 ≈ 0.615

ROUGE-2

- Ref bigrams: sorts a, a list, list of, of numbers, numbers in, in ascending, ascending order (7)
- Cand bigrams: sorts numbers, numbers in, in increasing, increasing order (4)
- Overlap: numbers in → 1
- Precision = $1/4 = 0.25$
- Recall = $1/7 \approx 0.143$
- ROUGE-2 F1 ≈ 0.182

Note: "ascending" $\neq$ "increasing" → no match, drags bigram score down hard despite same meaning. Shows ROUGE weakness — pure lexical overlap, no semantic understanding.

Semantic Similarity Score

Core Idea

ROUGE/BLEU = lexical (word overlap). Semantic similarity = meaning overlap. Text → embeddings (vectors) → compare meaning, not just words. Needed for chatbots, summarizers, recommenders — outputs must preserve meaning even if wording differs.

BERT

- **Bidirectional Encoder Representations from Transformers.** Google AI.
- Bidirectional: reads left→right AND right→left at same time → full context.
- Attention: model looks at all words at once, weighs importance, captures long-range dependency.
- Input: [CLS] token1 token2 ... [SEP] → each token → vector (embedding).
- [CLS] vector after processing = whole-sentence meaning representation. Used for tasks like sentiment classification.

Comparison Table (likely comparative-analysis Q)

Model/Approach	Direction	Strength	Weakness
BERT	Bidirectional	Full context understanding	Heavier than unidirectional
GPT	Unidirectional (L→R)	Good for generation	Misses full-sentence context for understanding tasks
RNN/LSTM (no attention)	Sequential	Handles sequences	Treats words equally / forgets long-range info
CNN (text)	Local window	Detects local patterns	Poor long-range dependency capture

BERT Variants (comparative)

- RoBERTa, T5 → higher accuracy, heavier compute cost.
- DistilBERT, ALBERT → faster/lighter, may lose nuance.
- ELECTRA → efficient training, more complex setup.

- GPT models → best for generation, expensive to fine-tune. (Pattern: accuracy vs speed/cost tradeoff — expect a "which model for X constraint" case question.)

Code Clone Types (semantic vs syntactic) — case-based likely

- Type-1: exact clone (only whitespace/comments differ).
- Type-2: renamed identifiers, otherwise structurally same.
- Type-3: near-miss, small edits/added-removed statements, still textually similar.
- Type-4: semantic/functional clone — different implementation/structure/variable names but same behavior. Textual similarity NOT required.
- **Type-1/2/3 = syntactic clones. Type-4 = semantic clone.**
- Use case: comparing two AI-generated code outputs that look different but do the same job → check Type-4 clone equivalence, not text diff.

LLM-as-a-Judge

- Use an LLM to evaluate another AI's output against custom criteria (quality, correctness, meaning match).
- Prompt pattern: "Does candidate match reference in meaning/functionality? Rate/explain."
- Jury approach: multiple LLMs judge → majority vote. Tie → human-in-the-loop breaks it.

Types of judging (case questions may describe one, ask to identify)

1. Pairwise comparison — given 2 responses, pick a better one (model/prompt/config comparison).
2. Criteria-based(reference-free) — judge tone/clarity/correctness with no ground truth given.

3. Criteria-based (reference-based) – judge given a reference/source doc, scores response against it.

Biases (LLM judge) – likely matching/identify-the-bias question

- Position bias – favors response based on order shown, not quality.
- Verbosity bias – longer = "better" regardless of content.
- Authority bias – swayed by citing authoritative-sounding sources.
- Diversity bias – skewed by identity markers (gender/ethnicity) in text.
- Self-enhancement bias – favors outputs from the same model/family.

Other limitations

- Struggles with complex reasoning/creative/expert tasks.
- Lacks proprietary/domain-specific context.
- Misleading confidence – convincing-sounding but wrong answers fool it.
- Still needs human oversight for critical/complex judgments.

Likely Question Angles

- Compare BERT vs GPT (bidirectional vs unidirectional) – why BERT better for understanding tasks.

- **BERT** reads both directions at once → full context before/after each word.
GPT reads left→right only → predicts next word using only past context, blind to what comes after.

Example: "The bank raised interest rates" vs "The bank of the river". Word "bank" – GPT only sees left context when encoding it. BERT sees both "raised interest rates" (right side) AND left side simultaneously → disambiguates meaning correctly.

Understanding tasks (classification, sentiment, NER, similarity) need full-sentence context to interpret each word correctly → bidirectional wins.

Generation tasks (writing next word) naturally sequential left→right → unidirectional (GPT) fits fine there, doesn't need future words since it's generating them.

- Why use semantic similarity over ROUGE/BLEU → meaning vs lexical overlap.

-**ROUGE/BLEU** count exact word/n-gram overlap → miss paraphrases. Example from your ROUGE note: "ascending order" vs "increasing order" = same meaning, zero n-gram match → ROUGE-2 tanked to 0.182 despite correct answer.

Semantic similarity uses embeddings (vectors) → compares meaning, not surface words. "Ascending" and "increasing" land close in vector space → high similarity score even with different words.

Solve

1(a) How LLM-as-a-Judge Works & Its Limitations (5 marks)

How it works: LLM-as-a-Judge uses a large language model as an automated evaluator instead of a human reviewer. A prompt is written that defines the evaluation criteria (e.g., "Does this candidate output match the reference in meaning/functionality? Rate 1–5 and explain."). The LLM then scores, ranks, or selects between outputs based on this instruction. Three common modes are used:

- **Pairwise comparison** — given two responses, the LLM picks the better one (useful for comparing models, prompts, or configurations).
- **Reference-free evaluation** — the LLM judges tone, clarity, or correctness with no ground truth provided.
- **Reference-based evaluation** — the LLM is given a source document/reference and scores the candidate against it.

To improve reliability, teams often use a **jury of LLMs** (e.g., GPT, Gemini, Claude) that each score independently; results are merged, and if there's a tie or disagreement, a human-in-the-loop makes the final call.

Limitations and their impact on reliability, fairness, and decision-making:

- **Position bias** — the LLM may favor a response simply due to its order in the prompt, reducing reliability of rankings.
- **Verbosity bias** — longer responses get rated as "better" even when content quality is equal, skewing decisions toward wordier (not necessarily correct) outputs.
- **Authority bias** — citing authoritative-sounding sources sways the judge regardless of actual correctness, harming fairness.
- **Diversity bias** — identity-related markers (gender, ethnicity) in text can skew judgment, introducing fairness/ethical risk.
- **Self-enhancement bias** — a judge model may favor outputs from its own model family, undermining impartiality when comparing across vendors.
- **Difficulty with complex reasoning and lack of domain/proprietary context** — the judge may misjudge specialized or business-critical outputs.
- **Misleading confidence** — an LLM judge can be fooled by convincing-but-wrong answers, giving false assurance to project managers.

Impact on project management: because these biases can silently distort quality scores, LLM-as-a-Judge should never be the sole gatekeeper for release decisions — it needs calibration against human-labeled ground truth and human oversight for high-stakes or ambiguous cases, otherwise reliability (inconsistent scores), fairness (biased outcomes), and decision quality (false confidence) in the software delivery pipeline are all put at risk.

1(b) CodeBLEU Calculation (5 marks)

Reference code:

```
def inc(a):  
  
    return a + 1
```

Candidate code:

```
def inc(x):  
    return x + 1
```

CodeBLEU is a composite metric designed for code (unlike plain BLEU, which only checks textual overlap). It combines four weighted components:

$\text{CodeBLEU} = \alpha \cdot \text{BLEU} + \beta \cdot \text{BLEU_weighted} + \gamma \cdot \text{Match_ast} + \delta \cdot \text{Match_dataflow}$ (commonly $\alpha=\beta=\gamma=\delta=0.25$)

Step 1 – Standard n-gram BLEU (textual match): Tokenize both:

- Reference: `def, inc, (, a, ), :, return, a, +, 1` → 10 tokens
- Candidate: `def, inc, (, x, ), :, return, x, +, 1` → 10 tokens

Matching tokens: `def, inc, (, ), :, return, +, 1` → 8 matches (`a` ↔ `x` do not match, they are different identifiers). Precision = Recall = $8/10 = \mathbf{0.80}$ (equal length, so BLEU ≈ 0.80)

Step 2 – Weighted n-gram match (keyword-weighted): CodeBLEU gives higher weight to language keywords (`def, return`) and operators, which all match exactly. Only the identifier token (`a` vs `x`) mismatches, and identifiers are weighted lower than keywords. This raises the score above plain BLEU: Weighted n-gram match $\approx \mathbf{0.85}$

Step 3 – Syntax match (AST-based): Parse both into Abstract Syntax Trees. Both produce identical tree shapes: `FunctionDef` → `Parameter` → `Return(BinOp(Name, Add, Constant))`. Only the parameter's *name* differs, not the tree structure. Since AST comparison is structural (not name-based), the trees match almost perfectly: Syntax (AST) match $\approx \mathbf{1.00}$

Step 4 – Dataflow match (semantic): The dataflow graph tracks how variables are defined and used. In both versions, the single parameter is defined once and used once, in the same operation (`param + 1`, then returned). The data-dependency pattern is identical regardless of the identifier name: Dataflow match $\approx \mathbf{1.00}$

Step 5 – Compute CodeBLEU: $\text{CodeBLEU} = 0.25(0.80) + 0.25(0.85) + 0.25(1.00) + 0.25(1.00) = 0.20 + 0.2125 + 0.25 + 0.25 = \approx \mathbf{0.91}$

Interpretation for the project manager: Although the raw text differs (parameter renamed from `a` to `x`), the syntax and dataflow components confirm the candidate is functionally and structurally equivalent to the reference. A high CodeBLEU (~0.91) indicates the submitted code preserves correctness and structure despite minor identifier changes — a case where plain BLEU/ROUGE alone would under-credit the candidate, showing why CodeBLEU is preferred for code quality/maintainability review over purely textual metrics.

2(a) Best Practices in Prompt Engineering (5 marks)

Key best practices used in modern AI-enabled software projects:

1. **Clarity** — writing unambiguous, specific instructions rather than vague requests, so the model has only one reasonable interpretation.
2. **Context setting** — providing relevant background (domain, prior conversation, data schema, business rules) so the model reasons within the correct frame instead of guessing.
3. **Role specification** — assigning the model a persona/role (e.g., "act as a senior QA engineer") to steer tone, depth, and perspective of the output.
4. **Constraint definition** — explicitly stating boundaries: output format, length limits, forbidden actions, style rules — so responses stay controlled and predictable.
5. **Iterative refinement** — treating prompt design as a cycle: test output, identify failure patterns, adjust wording/constraints, retest, rather than expecting a perfect prompt on the first try.

How these improve project outcomes:

- **Reliability** improves because clear, constrained prompts reduce the variance between repeated runs — the model behaves predictably across similar inputs.
- **Ambiguity is reduced** because context and constraints remove guesswork, so the model doesn't have to infer missing intent, which is a common source of incorrect outputs.
- **Reproducibility** is supported because a well-specified prompt (role + context + constraints) produces consistent behavior across team members, environments, and future prompt reuse — critical when multiple developers or CI pipelines depend on the same AI component behaving the same way each time.

Overall, treating prompt design with the same rigor as software requirements engineering (clear, testable, versioned) is what separates a fragile AI feature from a production-reliable one.

2(b) Why Prompts Should Be Version-Controlled + Strategy (5 marks)

Scenario recap: Different team members independently modified system prompts (tone guidelines, safety constraints, response templates) over several weeks. This caused inconsistent chatbot outputs, unexpected production behavior, and the team lost the ability to trace which prompt change caused which regression.

Why prompts must be version-controlled: Prompts function as executable specifications of model behavior — a small wording change can significantly shift output quality, tone, or safety compliance, just like a code change can introduce a bug. Without version control:

- Changes are untracked, so **no one can identify which edit caused a regression**.
- Multiple people editing the same prompt file leads to **overwritten or conflicting changes**.
- There is **no rollback path** if a new prompt performs worse in production.
- Debugging performance degradation becomes guesswork instead of a traceable diff.

Proposed structured version-control strategy:

1. **Store prompts as versioned artifacts in the code repository** (e.g., `prompts/support_bot/v1.2.0/system_prompt.txt`), not as ad-hoc edits in a dashboard.
2. **Naming convention:** semantic versioning per prompt component, e.g., `<bot-name>_<component>_v<major.minor.patch>` — major = behavior change, minor = added constraint, patch = wording tweak.
3. **Workflow:** every prompt change goes through a pull request with a description of intent (what behavior is being changed and why), peer review, and a required test run against a fixed evaluation set (sample conversations) before merge.
4. **Rollback procedure:** tag each deployed prompt version; if production metrics (accuracy, user complaints, safety flags) degrade after a release, immediately

redeploy the last known-good tagged version while the new one is investigated offline.

5. **Change log:** maintain a changelog mapping each version to the observed evaluation results, so future changes can be compared against historical performance.

This turns prompt engineering into a disciplined, auditable process — the same rigor applied to source code — ensuring reliable, traceable, and recoverable chatbot behavior in production.

3(a) How Fine-Tuning Specializes a General-Purpose Model (5 marks)

Fine-tuning takes a pre-trained, general-purpose model (trained on broad, diverse data) and continues training it on a smaller, task/domain-specific dataset. This adjusts the model's internal weights so its behavior shifts from generic competence toward specialized accuracy in the target domain.

How this improves domain-specific performance: The base model already has general language/coding understanding; fine-tuning teaches it the specific vocabulary, patterns, edge cases, and conventions of the target domain (e.g., legal contracts, medical notes, a company's internal codebase style), which a general model would not reliably capture from generic pretraining alone.

How this aligns the model with project requirements: Fine-tuning lets a team encode project-specific rules, formats, and expected behaviors directly into the model's weights (e.g., always output JSON in a particular schema, follow a company's coding style, respect specific compliance constraints) — rather than relying entirely on prompt instructions at inference time, which can be inconsistently followed.

How this enhances overall software functionality:

- Reduces the need for long, fragile prompts, since domain knowledge is baked into the model itself.
- Improves consistency and accuracy on repetitive, domain-specific tasks (e.g., ticket classification, code review comments in a specific stack).

- Makes the AI component a more reliable, specialized building block within the larger software system, rather than a generic tool bolted on with prompt engineering alone.

In short: fine-tuning converts a broad general-purpose model into a specialized component that better fits the exact requirements, vocabulary, and constraints of the software project it's embedded in.

3(b) LoRA and QLoRA (5 marks)

LoRA (Low-Rank Adaptation): Instead of updating all of a model's (often billions of) parameters during fine-tuning, LoRA freezes the original pre-trained weights and injects small, trainable low-rank matrices into specific layers (typically attention layers). Only these small matrices are trained, drastically reducing the number of trainable parameters while still adapting the model's behavior to the new task/domain.

QLoRA (Quantized LoRA): QLoRA extends LoRA by first **quantizing** the frozen base model's weights to a lower precision (commonly 4-bit) before applying the same low-rank adapter training on top. This shrinks the memory footprint of the base model itself, not just the trainable parameters.

Key differences:

Aspect	LoRA	QLoRA
Base model precision	Full/half precision (e.g., FP16/FP32)	Quantized (e.g., 4-bit)
GPU memory required	Moderate	Much lower

Training speed	Fast (small param count)	Slightly slower per step (dequantization overhead) but enables larger models on same hardware
Hardware needed	Mid-range GPUs sufficient for smaller models	Enables fine-tuning very large models on consumer-grade/limited GPUs
Accuracy trade-off	Minimal loss vs full fine-tuning	Slightly more precision loss due to quantization, but generally close to LoRA's accuracy

How these differences influence project decisions:

- **Model customization:** QLoRA allows teams to fine-tune much larger models than their hardware would normally support, expanding which base models are even feasible to customize in-house.
- **Training efficiency:** LoRA is preferable when compute is already sufficient and maximum accuracy retention is prioritized; QLoRA is preferable when the priority is fitting a large model's fine-tuning job into constrained infrastructure.
- **Resource allocation and project planning:** QLoRA lowers the GPU/memory budget needed, which can shift a project from requiring expensive multi-GPU clusters to running on a single consumer or lower-tier cloud GPU — directly affecting cost estimates, timeline (less time waiting for scarce high-end GPU availability), and team decisions about which model size is realistically achievable within budget.

Time & Scheduling

1. Core Concepts

Time & Scheduling: critical for meeting deadlines + optimizing resource use.

- Static scheduling: planned once at start, fixed.
- Dynamic scheduling: adapted as project changes.

Software Project Scheduling Problem (SPSP)

- Definition: assigning tasks → resources → timeline, under constraints (skills, precedence, availability).
- Classified as NP-hard optimization problem.
- SBSE (Search-Based Software Engineering) view: solved using evolutionary algorithms / metaheuristics (GA, PSO, ACO).
- Objectives: minimize duration, balance workload, avoid resource conflicts/overload, meet deadlines.

Likely conceptual Q: "Why is SPSP considered NP-hard?" → Because task-resource-timeline assignment space grows combinatorially with constraints (skills, precedence, availability); no known polynomial algorithm guarantees optimal solution, so metaheuristics are used instead of exact methods.

2. Scheduling Techniques (know all 4 + purpose)

Technique	Purpose
WBS (Work Breakdown Structure)	Task breakdown, leveling, resource allocation
CPM (Critical Path Method)	Dependency-focused, finds critical path, deterministic durations
PERT (Program Evaluation & Review Technique)	Probabilistic time estimates, handles uncertainty

Gantt Charts & Milestones	Visualizing timeline
---------------------------	----------------------

Conceptual Q: "**CPM vs PERT – key difference?**" → CPM uses single deterministic duration per task; PERT uses three estimates (O, M, P) to compute expected duration, handling uncertainty. CPM = certainty-based, PERT = probability-based. Both then use forward/backward pass to find critical path.

3. CPM – Forward & Backward Pass (MUST MASTER – high scenario-Q chance)

Definitions:

- ES = Earliest Start
- EF = Earliest Finish = ES + Duration
- LS = Latest Start = LF - Duration
- LF = Latest Finish
- Slack = LS - ES (also = LF - EF)

Forward Pass (left→right): compute ES, EF

- ES of a task = max(EF of all predecessors)
- EF = ES + Duration
- Project duration = max EF of final task(s)

Backward Pass (right→left): compute LS, LF

- Start from last task: LF = project duration
- LF of a task = min(LS of all successors)
- LS = LF - Duration

Slack interpretation:

- Slack = 0 → task is critical, any delay = project delay
- Slack > 0 → task has flexibility, can delay up to slack days without affecting completion

Critical Path = path through all tasks with slack = 0 (longest path through network).

Worked example (from slides) – practice this exact type

Tasks: T1(4)→T2(6)→T4(7)→T7(4)→T8(6), and T1→T3(5)→T6(8)→T8

Task	Dur	Depends	ES	EF	LS	LF	Slack
T1	4	—	0	4	0	4	0 (critical)
T2	6	T1	4	10	4	10	0 (critical)
T3	5	T1	4	9	8	13	4
T4	7	T2	10	17	10	17	0 (critical)
T5	3	T2	10	13	14	17	4
T6	8	T3	9	17	13	21	4
T7	4	T4,T5	17	21	17	21	0 (critical)
T8	6	T6,T7	21	27	21	27	0 (critical)

- Total project duration = 27 days
- Critical path = T1 → T2 → T4 → T7 → T8
- Non-critical (slack>0): T3, T5, T6 – each can slip without delaying project.

Key rule to remember for scenario Qs: when a task has MULTIPLE predecessors, take the MAX EF (forward). When a task has MULTIPLE successors, take the MIN LS (backward).

4. PERT – Expected Duration Formula

$$TE = \frac{O + 4M + P}{6}$$

Where O = Optimistic, M = Most Likely, P = Pessimistic.

Worked example

Task	O	M	P	TE
T1	2	3	5	$(2+12+5)/6 = 3.2$
T2	4	6	10	$(4+24+10)/6 = 6.3$
T3	3	4	7	$(3+16+7)/6 = 4.0$

Then reapply CPM logic with TE values:

- T1 EF = 3.2
- T2 EF = 3.2 + 6.3 = 9.5
- T3 EF = 9.5 + 4.0 = 13.5 days (PERT-based project duration)

Scenario Q pattern: give O/M/P for 3–4 tasks with dependencies
→ compute TE for each → chain them via CPM-style forward pass → find project duration. Practice this – very likely to appear as numeric problem.

Conceptual Q: "Why does PERT weight Most Likely (M) by 4?" → Approximates a beta distribution; M is weighted more heavily since it's the most probable outcome, O and P represent tail extremes.

5. Diagram Types

- Gantt Chart: horizontal bar chart, task vs time, shows overlap/duration visually.
- Network Diagram: nodes = activities, arrows = dependency type:
- FS (Finish-to-Start) – most common, B starts after A finishes
- SS (Start-to-Start) – B starts when A starts
- FF (Finish-to-Finish) – B finishes when A finishes
- SF (Start-to-Finish) – rare, B finishes when A starts

Conceptual Q: "Differentiate FS, SS, FF, SF with example." — Be ready to draw/explain each with a 2-task example.

6. Automated/Heuristic Scheduling (SBSE)

Metaheuristics used when problem is NP-hard / too complex for manual/exact methods:

- Genetic Algorithms (GA)
- Particle Swarm Optimization (PSO)
- Ant Colony Optimization (ACO)

7. Factors Affecting Scheduling

Team skills & availability · Task complexity · Requirements clarity · Technology maturity · Inter-task dependencies · External constraints (vendors, clients)

8. Challenges in Time Scheduling

- Unstable parameters, environmental changes, competing goals → manual scheduling inefficient
- Limited research on dynamic models / large-scale uncertainty; gaps in interactive/adaptive approaches
- GA-type methods don't guarantee global optimum; computational complexity can hinder real use

Conceptual Q: "Why can't we just always use GA for scheduling?"
→ No guarantee of global optimum, high computational cost, may not scale/adapt well to dynamic real-time changes.

9. Modern Tools

Jira · Microsoft Project · Trello · Asana · OpenProject · GanttProject

Likely Exam Question Bank (self-test)

Conceptual:

1. **Q1: Define SPSP. Why is it NP-hard?**

A: SPSP = assign tasks→resources→timeline under constraints (skill, precedence, availability). NP-hard bc solution space grows combinatorial w/ tasks×resources×constraints, no poly-time exact algo exists, so search/metaheuristics used instead.

2. **Q2: Differentiate static vs dynamic scheduling.**

A: Static: plan fixed once at start. Dynamic: adapts as project changes (scope shift, resource loss, delay).

3. **Q3: Compare WBS, CPM, PERT – when would you use each?**

A: WBS: task breakdown + resource allocation (structuring phase). CPM: deterministic durations, finds critical path, dependency-focused. PERT: durations uncertain, uses O/M/P estimates, probabilistic. Use WBS first to break work down → CPM if durations known/fixed → PERT if durations uncertain/risky.

4. **Q4: What does zero slack mean? What does positive slack allow?**

A: Slack = LS – ES. Zero slack → task critical, any delay pushes whole project. Positive slack → task has buffer, can delay up to slack days w/o affecting project completion.

5. **Q5: Explain PERT's three-point estimate and justify the weighting formula.**

A: Three-point: O (best case), M (most likely), P (worst case). Formula $TE = (O + 4M + P) / 6$ – approximates beta distribution, weights M ×4 since it's most probable outcome, O/P just mark tail extremes. Gives realistic expected duration vs single-point guess.

6. **Q6: Name 3 metaheuristics used in SBSE scheduling and why they're preferred over exact methods for SPSP.**

A: GA (Genetic Algorithm), PSO (Particle Swarm Optimization), ACO (Ant Colony Optimization). Preferred bc SPSP NP-hard – exact

methods (brute force/ILP) don't scale for large task sets;
metaheuristics find good-enough solution in reasonable time, handle multiple objectives (duration, workload, conflicts) at once.

7. **Q7: List challenges in real-world dynamic scheduling.**

A: Unstable parameters + environment changes + competing goals
→ manual scheduling inefficient. Limited research on dynamic models / large-scale uncertainty – gap in interactive/adaptive approaches. Metaheuristics (GA etc.) don't guarantee global optimum + costly compute, hard to scale.

Cost & Risk Evaluation

PART A: COST

1. Why Cost Estimation Matters

- Drives budgeting, staffing, planning, decision-making.
- **Underestimation** → funding shortfall, rushed work, quality drop, scope cuts.
- **Overestimation** → lost bids/proposals, inefficient resource allocation, wasted budget.

2. Software Cost Components

Type	Examples
Direct	labor, tools, materials
Indirect	training, overheads

Hidden	rework, delays, integration issues
--------	------------------------------------

Conceptual Q: "Why are hidden costs dangerous in estimation?"
→ Not visible upfront, arise from rework/integration/delay, so estimates that ignore them run over-budget even if direct+indirect costs were accurate.

3. Cost Estimation Techniques (know all 6)

- Expert judgment
- Algorithmic models (e.g., **COCOMO**)
- Analogy-based estimation
- Parametric models
- Use-case-based estimation
- Machine learning methods (recent research)

4. Worked Example — Payroll Software (memorize pattern, numbers will change in exam)

Modules: M1 (5,000 LOC, Medium), M2 (8,000 LOC, High), M3 (3,000 LOC, Low) Cost/developer-day = BDT 400. Total LOC = 16,000.

(a) Expert Judgment

Manager estimates developer-days directly per module → × rate/day.

- M1 = 120 days × 400 = 48,000
- M2 = 200 days × 400 = 80,000
- M3 = 60 days × 400 = 24,000
- **Total = BDT 152,000**
- Interpretation: subjective, relies on experience, simple but not data-driven.

(b) Analogy-Based Estimation

Use a **similar past project's** cost/LOC ratio, scale to current LOC.

- Past: 12,000 LOC cost BDT 180,000 → rate = 15/LOC
- Current total LOC = 16,000 → Cost = 16,000 × 15 = **BDT 240,000**
- Interpretation: quick, but accuracy depends on how similar the past project really is.

(c) Bottom-Up Estimation

Per-module: Developer-days = LOC ÷ productivity(LOC/day, varies by complexity) → × rate. Productivity: Low=50, Medium=40, High=30 LOC/day.

Module	LOC	Complexity	Prod.	Dev-days	Cost
M1	5,000	Medium	40	125	50,000
M2	8,000	High	30	≈267	≈106,800
M3	3,000	Low	50	60	24,000

- **Total ≈ \$180,800**
- Interpretation: most detailed/accurate technique, IF productivity data reliable.

(d) COCOMO (Constructive Cost Model)

Formula: **Effort (person-months) = a × (KLOC)^b**

Modes (memorize table — very testable):

Mode	Use case	a	b
Organic	Small, simple, familiar, clear requirements	2.4	1.05
Semi-Detached	Medium size, mixed complexity	3.0	1.12
Embedded	Large, complex, strict constraints	3.6	1.20

a,b are **empirically fitted constants** from Boehm's historical project data — not arbitrary.

Worked calc (Organic mode):

- $KLOC = 16,000/1000 = 16$
- $Effort = 2.4 \times (16)^{1.05} \approx 2.4 \times 17.4 \approx \mathbf{41.8 \text{ person-months}}$
- Assume 20 working days/month $\rightarrow$ Developer-days = $41.8 \times 20 \approx 836$
- $Cost = 836 \times \$400 \approx \mathbf{\$334,400}$
- Interpretation: systematic, accounts for **non-linear** size scaling (larger projects cost disproportionately more due to coordination overhead — that's why $b > 1$).

Scenario Q pattern: give LOC + mode (organic/semi-detached/embedded) + rate/day $\rightarrow$ compute effort $\rightarrow$ person-months $\rightarrow$ developer-days $\rightarrow$ total cost. Also compare answer against another technique (expert/analogy/bottom-up) done in same problem, and explain WHY estimates differ.

Conceptual Q: "Why do the 4 estimation techniques give 4 different totals for the same project?" (152k vs 240k vs 180.8k vs

334.4k) → Each uses a different basis: expert judgment = subjective opinion; analogy = historical ratio (may not truly match); bottom-up = actual per-module productivity data (most grounded); COCOMO = empirical non-linear model accounting for complexity growth. No single "correct" number — estimation accuracy depends on data quality and method fit.

5. Sources of Estimation Error

Requirements volatility · Lack of historical data · Human bias · Complexity underestimation · Environmental factors

6. Factors Influencing Cost & Schedule Estimation

Team experience · Technology maturity · Project size & complexity · Development model (Agile vs Waterfall) · Organizational culture

PART B: RISK

7. Risk — Definition & Types

Risk = uncertain event/condition that, if it occurs, can positively or negatively affect project objectives (time, cost, etc.) Types: **technical, managerial, financial, operational**

8. Risk Identification Frameworks

- **Boehm's Top 10 Risk Categories** (memorize — high MCQ/short-Q chance):
 1. Personnel shortfalls
 2. Unrealistic schedules & budgets

3. Developing wrong functions/properties (inexperience with application)
 4. Requirement changes/instability
 5. Shortfalls in externally furnished components / early planning
 6. Customer conflicts / lack of user involvement
 7. Software complexity (too complex to test)
 8. Unrealistic performance expectations
 9. Inadequate tool support/technology
 10. Organizational/management issues
- **Risk Breakdown Structure (RBS):** hierarchical categorization –
 - Technical (inadequate testing, tech failures, performance bottlenecks)
 - Project Management (schedule slippage, budget overruns, poor planning)
 - Organizational/External (team turnover, low stakeholder engagement, resource unavailability)
 - External/Environmental (regulatory changes, market changes, natural disasters)
 - **Risk in Global Software Development (GSD):** timezone differences, culture gaps, communication gaps.

Example RBS (payroll project) – know how to build one for scenario Qs

L1	L2	L3 (specific)

Technical	Complexity	Payroll calc logic errors
Technical	Integration	Reporting module fails to integrate w/ HR DB
Project Mgmt	Scheduling	M1 delayed – developer unavailable
Project Mgmt	Budget	Extra testing exceeds budget
Organizational	Personnel	Developer leaves project
External	Regulatory	Change in tax law/labor regulation

Scenario Q pattern: given a project description, build an RBS table (L1→L2→L3) for it, or classify given risks into Boehm's categories / RBS branches.

9. Risk vs Uncertainty (classic exam distinction)

	Probability & impact known?
Risk	Yes – can estimate both (e.g., 30% chance library integration fails, costs \$5,000)

Uncertainty	No — probability or impact unknown/unpredictable (e.g., sudden regulation change)
--------------------	---

Conceptual Q: "Give an example each of risk and uncertainty in a software project, and explain the difference."

10. Risk Response Strategies (memorize table)

Strategy	Description	Example
Avoid	Change plan to eliminate risk	Switch to proven technology
Mitigate	Reduce likelihood or impact	Add extra testing/training
Transfer	Shift risk to third party	Buy insurance, outsource
Accept	Acknowledge, do nothing	Tolerate minor delays

Scenario Q pattern: given a risk description, name the best response strategy + justify. E.g. "A rare library incompatibility risk with low impact" → Accept. "Critical vendor risk" → Transfer (insurance/contract).

11. Risk Analysis & Assessment

- Probability vs Impact
- Qualitative: risk matrix (probability × impact grid)

- Quantitative: **Expected Monetary Value (EMV)**

12. Modeling Risk & Uncertainty – 4 approaches (very likely numeric scenario Q)

(a) Probability-Based Models (EMV)

EMV = Probability × Impact Example: 20% chance server outage, \$10,000 impact → $EMV = 0.2 \times 10,000 = \$2,000$

Scenario Q pattern: given several risks with probability & \$ impact, compute EMV for each, rank/prioritize, sometimes sum for total risk exposure.

(b) Scenario-Based Analysis

Consider best-case/worst-case "what-if" scenarios (e.g., project finishes in 6 months best, 12 months worst).

(c) Monte Carlo Simulation

Runs many simulated iterations using probability distributions (usually **triangular distribution**: min, most likely, max) to predict project duration/cost distribution.

(d) Decision Trees

Visual: decisions → risks → probabilities → outcomes; used to compare alternative strategies under uncertainty.

13. Monte Carlo Simulation – Worked Example (MASTER THIS, high scenario-Q chance)

Tasks with (Min, Most Likely, Max) days – **triangular distribution** used because we only know 3 points, not full distribution.

Task	Min	Most Likely	Max
Design	4	6	10
Coding	8	12	16
Testing	3	5	9

Steps:

1. Define triangular distribution per task (min/likely/max).
2. Random-sample each task per iteration (small demo used 5 iterations; real practice = 10,000+).
3. Sum D+C+T per iteration = total duration.

Sample iterations:

Iter	D	C	T	Total
1	5	11	4	20
2	7	12	6	25
3	4	10	5	19

4	6	14	7	27
5	6	12	5	23

Analysis:

- Average duration = $(20+25+19+27+23)/5 = 114/5 = \mathbf{22.8 \text{ days}}$
- $P(\text{finish} \leq 22 \text{ days}) = \text{iterations} \leq 22 \rightarrow \{1(20), 3(19)\} = 2/5 = \mathbf{40\%}$
- $P(\text{finish} \leq 25 \text{ days}) = \{1, 2, 3, 5\} = 4/5 = \mathbf{80\%}$

Decision-making:

- Deadline = 22 days $\rightarrow$ only 40% success chance $\rightarrow$ **high risk of delay**
- Deadline = 25 days $\rightarrow$ 80% success chance $\rightarrow$ **acceptable risk**

Full-scale version (1000+ iterations) $\rightarrow$ produces smooth histogram/bell-shaped distribution:

- Most likely duration $\approx$ 23 days (peak)
- 22-day deadline $\rightarrow$ $\sim 17.3\%$ probability of on-time finish
- 25-day deadline $\rightarrow$ $\sim 63.1\%$ probability of on-time finish
- (Small-sample 5-iteration demo overestimates probabilities vs large-scale 1000-iteration truth — larger n = more reliable %.)

Key exam point: with only 5 iterations, results are noisy/unreliable — that's WHY real Monte Carlo uses thousands of iterations for a smooth, trustworthy probability distribution.

Scenario Q pattern: given task min/likely/max + a small set of sample iterations $\rightarrow$ compute average duration, compute $P(\text{finish within } X \text{ days})$ by counting iterations, then interpret decision (accept/reject deadline based on risk tolerance).

Likely Exam Question Bank (self-test)

Conceptual:

1. Why does cost estimation matter? Consequences of under/overestimation.
2. Differentiate direct, indirect, hidden costs.
3. Compare expert judgment, analogy-based, bottom-up, and COCOMO estimation – strengths/weaknesses of each.
4. Explain COCOMO's three modes and why a,b differ across them.
5. Why is COCOMO's effort formula non-linear ($KLOC^b$, $b > 1$)?
6. Define risk. How does it differ from uncertainty? Give examples.
7. List Boehm's top 10 risk categories (or explain a few).
8. Explain the 4 risk response strategies with examples.
9. What is EMV? How is it calculated?
10. Why use triangular distribution in Monte Carlo simulation instead of normal distribution?
11. Why does Monte Carlo need thousands of iterations, not just 5?

Scenario/Numeric (practice with fresh numbers):

1. Given LOC + complexity per module + rate/day → compute cost via expert judgment / analogy / bottom-up / COCOMO, and explain why results differ.
2. Given a project description → build an RBS (L1/L2/L3) and/or classify risks into Boehm's categories.
3. Given a risk (probability + impact) → compute EMV; given multiple risks → rank by EMV.
4. Given a risk scenario → recommend Avoid/Mitigate/Transfer/Accept + justify.
5. Given task (min/likely/max) + a handful of sampled iterations → compute average duration and $P(\text{finish within deadline})$, then make an accept/reject-deadline recommendation.

6. "Which project mode (organic/semi-detached/embedded) fits this project description?" → apply COCOMO table based on described size/complexity/team familiarity.

Cyber/Software Audit — Exam Notes

Question-setter lens: definitions asked conceptually, laws/sections asked as MCQ or match, cases (NID, citizen portal) asked as scenario problems needing "identify the audit failure + fix."

1. Software Audit — Policy Perspective

Core idea: systematic review of digital systems, data, controls. Not tech/coding skill — governance, accountability, risk, compliance focus.

Key question it answers: "Are our digital systems safe, controlled, accountable?"

Likely Q (conceptual): Define IT/software audit from a policy (non-technical) perspective. Answer skeleton: systematic review → governance/accountability/risk/compliance → no coding needed → answers safety/control/accountability question.

Likely Q (scenario): "A minister claims IT audit isn't needed since he has no IT background." Counter this. Answer skeleton: audit is governance issue not technical issue — leadership accountable for cyber risk regardless of tech skill (see topic 3).

2. Traditional Audit vs Cyber/Digital Audit

Traditional	Cyber/Digital
Paper files, physical records	Online systems, databases
Manual approvals/signatures	Automated decisions/transactions
Visible fraud/errors	Hidden risks (data misuse, access abuse)

Likely Q (conceptual): Compare traditional vs digital audit – 3 differences.

Likely Q (scenario): Auditor finds no paper trail for an approval – system auto-approved it. Which audit type applies, what risk category? Answer: digital/cyber audit – automated decision risk, hidden misuse risk (not "visible fraud").

3. IT Auditing = Policy Issue, Not IT Issue

Reasoning chain: digital systems affect citizen services, public funds, national reputation → cyber risk = governance + leadership responsibility.

Consequences of cyber failure: data leaks, service disruption, loss of public trust.

Likely Q (scenario): A ministry data leak occurs; officials blame "IT department." Evaluate. Answer: wrong – cyber risk is governance/leadership responsibility, not solely IT dept's; accountability sits with leadership.

4. What IT Auditing Looks At (High Level)

Reviews: who can access systems/data · how citizen data protected · vendor control · incident handling · policy-vs-practice gap.

No technical testing required – memorize this line, common true/false trap.

Likely Q (scenario): Auditor wants to check policy compliance without doing penetration testing. Is this valid IT audit scope? Answer: yes – high-level IT audit doesn't require technical testing; reviews access, data protection, vendors, incidents, policy implementation.

5. Government Digital Assets (categories – memorize for scenario ID)

1. Citizen Data – NID, birth registration, social safety net data
2. Government Systems – ministry portals, service platforms, databases
3. Financial Systems – e-GP, digital payments, allowance distribution
4. Infrastructure – data centers, gov cloud, networks
5. Third-Party Assets – vendor-managed/outsourced platforms

Likely Q (scenario): "A payment disbursement app for social allowance goes offline." Classify asset type + audit angle. Answer: Financial System asset → audit angle: availability control, incident handling.

6. Government Threat Landscape (Policy View)

- Unauthorized access (staff/vendor)
- Data leakage from poor access control
- Weak passwords / shared accounts
- System downtime/disruption
- Vendor negligence / lack of accountability

Key line (exam favorite): "Threats are not only hackers – they often arise from weak management and controls."

Likely Q (conceptual): Are cyber threats to government systems mostly external hackers? Justify. Answer: No – list above shows internal/management-origin threats dominate (access control, passwords, vendor negligence) – reinforces governance framing.

7. Cyber Laws – Purpose

- Protect citizen data/digital services
 - Define responsibilities (govt + vendors)
 - Ensure accountability for incidents
 - Support legal action vs misuse/negligence
-

8. Key Cyber Laws & Policies in Bangladesh (memorize table – high MCQ probability)

Law	Role
Cyber Security Ordinance, 2025	current law on cyber offences
Personal Data Protection Act, 2026	first standalone data protection law
National Data Governance Ordinance, 2025	rules for managing govt data
ICT Act, 2006	original foundation law, still partly in force

9. Evolution of Bangladesh Cyber & Data Laws (sequence questions likely)

Cyber-offence law chain: Digital Security Act, 2018 → Cyber Security Act, 2023 → repealed 21 May 2025 → Cyber Security Ordinance, 2025 (current).

Data protection law chain: Personal Data Protection Ordinance, 2025 (6 Nov 2025) → amended Feb 2026 → made permanent as Personal Data Protection Act, 2026 (passed by Parliament, April 2026).

Exam trap line: "These laws change quickly – always verify current version before citing in audit."

Likely Q (scenario): An audit report cites "Digital Security Act 2018" as the current cyber-offence law in 2026. Evaluate. Answer: outdated/incorrect – DSA 2018 repealed via chain above; current law = Cyber Security Ordinance, 2025. Shows why auditors must verify current law version.

10. ICT Act, 2006 – Then and Now

- Original law: digital signatures, e-commerce recognition, first cyber-offence rules.
- Most controversial clause: Section 57 (offensive online content) – repealed, absorbed into DSA 2018.
- Offence provisions moved: DSA 2018 → Cyber Security Act 2023 → Cyber Security Ordinance 2025 (current).
- ICT Act 2006 still active for: digital signature + e-commerce provisions only.

Audit takeaway (verbatim-worthy): always check which law currently governs a given offence – often not ICT Act anymore.

Likely Q (scenario): A vendor contract references ICT Act 2006 Section 57 for an online-content offence clause. Is this valid today? Answer: invalid –

Section 57 repealed/absorbed into DSA 2018, then offence provisions moved on to Cyber Security Ordinance 2025. Contract should cite current ordinance.

11. Cyber Security Ordinance, 2025 – Penalties Table (memorize ranges)

Category	Sanction
Online Gambling	up to 2 yrs jail or Tk 1 crore fine or both
Cyber Fraud / Financial Deception	2–5 yrs jail + significant fines
Cyber Deception / Forgery	up to 5 yrs jail + fines
Sexual Harassment & Exploitation Content	increased penalties, special protection for children/women
Repealed Offences	cases under old sections void and dismissed

12. Sections 17–23: Core Cyber Offences

- Sect. 17 – Unauthorised Access & Hacking: illegal entry/interference with protected systems & critical infrastructure. Punishment: 5–7 yrs + fines; may be non-bailable.
- Sect. 20 – Online Gambling: creating/using gambling platforms punishable. Up to 2 yrs + fines.

13. Sections 21 & 22 – Fraud & Forgery

Cyber-based fraud, identity misrepresentation, falsification of digital documents. Penalty: imprisonment + significant fines.

14. Sections 25–26: Content-Related Offences

- Sect. 25 – Online Harm & Sexual Exploitation: posting/sharing intimate images without consent (revenge porn, blackmail). Penalties up to several years + fines; stricter for women/children victims.
- Sect. 26 – Religious/Communal Hate Content: content promoting group hatred punishable. Aim: public order – but risk of vague definition/misuse (mention as critique point).

Likely Q (scenario, common exam style): A government employee accesses a citizen database without authorization and leaks data. Which section applies + penalty range? Answer: Section 17 (Unauthorised Access & Hacking) – 5–7 yrs imprisonment + fines, possibly non-bailable.

Likely Q (scenario): Someone shares an ex-partner's private photo online without consent. Which section? Answer: Section 25 – Online Harm & Sexual Exploitation Content (revenge porn) – increased penalty if victim is woman/child.

15. Access Control & Identity Audit

Access control: defines who uses which system/data + what they can do → protects sensitive info, citizen data, financial systems, infrastructure.

Identity audit: systematic review of accounts/roles/permissions. Checks:

- old accounts disabled after staff leave/transfer
- users have appropriate access level for role
- vendor/third-party access controlled & monitored

Likely Q (scenario, very common): An auditor finds a former employee's account still active 6 months after resignation, with admin rights. Identify audit failure + fix. Answer: Identity audit failure — offboarding control gap (old account not disabled). Fix: enforce account deactivation on staff exit, periodic access review.

16. Data Audit

Definition: review of data collection, storage, usage, protection in govt systems → ensures accuracy, security, compliance.

Key Audit Questions (memorize as checklist):

1. Is data classified correctly (sensitive vs routine)?
2. Who owns/is responsible for the data?
3. Controls for integrity, confidentiality, availability (CIA triad)?
4. How is data shared with vendors/other depts?
5. Retention & disposal policy exists?

Likely Q (scenario): A department shares citizen NID data with a vendor with no data-sharing agreement or retention policy. What audit failures? Answer: fails Q4 (uncontrolled sharing) + Q5 (no retention/disposal policy) — data audit gaps.

17. Vendor & Outsourced IT Audit

Definition: review of systems/processes/services managed by external providers.

Key point (accountability trap): outsourced systems still hold govt/citizen data; senior officials remain accountable for governance even if IT is outsourced.

Focus areas:

- Contractual Compliance — security/data responsibilities defined? SLAs + penalties included?
- Access Management — vendor access limited/monitored? reviewed & revoked when unneeded?
- Data Handling & Security — sensitive data encrypted/backed up/classified? retention/disposal followed?
- Incident Reporting — vendors report breaches immediately? response tested/documented?

Practical questions: who owns vs who manages data? legal/policy compliance? regular vendor audits/reports? accountability for service failures?

Likely Q (scenario, high probability): A vendor-managed citizen portal suffers a breach; vendor doesn't report it for 2 weeks. Minister claims department not liable since "IT was outsourced." Evaluate + list audit gaps. Answer: Minister wrong — accountability remains with government even when outsourced. Gaps: Incident Reporting failure (no immediate breach report, no tested response procedure); possibly Contractual Compliance gap (no SLA/penalty clause enforcing timely reporting).

18. IT Audit Report Template (structure — sequence/label questions likely)

1. Executive Summary — audit title, dept/system, audit period, auditors, overall assessment (Compliant/Needs Improvement/High Risk), 3–5 key highlights
2. Objectives — purpose of audit
3. Scope — systems/apps/processes reviewed, in-scope vs out-of-scope

4. Methodology — policy/contract/log/access record review, interviews, incident report/audit trail review
5. Key Findings
6. Recommendations — table: Finding No. | Recommendation | Responsible Party | Priority (Immediate/Short-Term/Long-Term) | Target Completion

Likely Q (conceptual): List the sections of a standard IT audit report in order. Answer: Executive Summary → Objectives → Scope → Methodology → Key Findings → Recommendations.

19. OCAG — Bangladesh's Supreme Audit Institution

- OCAG = Office of the Comptroller and Auditor General — constitutionally mandated SAI of Bangladesh.
- Runs 17 audit directorates, incl. dedicated Directorate of IT & Public Services Audit.
- Audits ministry info systems + citizen-facing digital platforms.
- Methodology: OCAG Government Auditing Standards (2021) — risk assessment → evidence gathering → findings → follow-up.
- Reports go to: President → laid before Parliament → reviewed by Public Accounts Committee.

Likely Q (conceptual): Trace the flow of an OCAG IT audit report from completion to parliamentary review. Answer: President receives report → laid before Parliament → reviewed by Public Accounts Committee.

20. Real Case: NID System Audit (2026) — scenario goldmine

- 2026: Election Commission formally asked OCAG for IT audit of NID system (earlier internal audit committee had stalled).

- EC separately asked Bangladesh Computer Council (BCC) to run VAPT (Vulnerability Assessment and Penetration Testing).
- Why it mattered – breach history:
 - 2023: breach exposed ~50 million citizens' NID data via a partner organisation's website.
 - Later: NID + phone data leaked again via a Telegram bot.
 - 2026: technical fault on EC website exposed data of ~14,000 journalists.
- Exam takeaway: access control, vendor oversight, incident response are live/real audit findings, not just theory.

Likely Q (scenario): Given the NID breach history, which audit domains failed repeatedly (name at least 3) and what OCAG/BCC roles address them? Answer: Vendor/Third-Party oversight (partner site breach), Access Control (repeated leaks via unofficial channel/Telegram bot), Incident Response (technical fault exposure) → OCAG does governance/compliance IT audit; BCC does technical VAPT.

21. Mini Case Study – Citizen Service Portal (Digital/Smart Bangladesh)

Setup: govt dept launched online portal (certificates + benefit applications) under Digital Bangladesh/Smart Bangladesh initiative. Goal: cut physical visits, improve transparency, speed delivery.

System profile:

- Built by local private IT vendor
- Hosted on govt-approved data center
- Used nationwide by district/field offices
- Accessible to govt officials and outsourced support staff

Mirrors real incidents:

- 2023: ~50M citizens' NID data (names, phones, emails, NID numbers) exposed via partner site
- late 2023: NID/personal data leaked again via Telegram bot
- 2026: EC website fault exposed ~14,000 journalists' data
- 2026: EC → OCAG (IT audit) + BCC (pen testing)

Likely Q (full scenario, exam-style): "As lead auditor for this citizen portal, list 4 audit checks you'd run and which asset/threat category each maps to." Model answer approach:

1. Identity audit on outsourced support staff accounts → Access Control & Identity Audit / vendor threat
2. Data classification & sharing check for citizen data → Data Audit / Citizen Data asset
3. Vendor contract review — SLA, breach reporting clause → Vendor & Outsourced IT Audit
4. Incident response test (simulate breach reporting timeline) → Vendor Incident Reporting + Threat Landscape (vendor negligence)

Likely Q: "Who remains accountable if the outsourced support staff cause a data leak?" Answer: Senior government officials/department — accountability for governance stays with government even when IT is outsourced.

Quick-Fire Definitions (for MCQ/short-answer cramming)

- VAPT = Vulnerability Assessment and Penetration Testing
- OCAG = Office of the Comptroller and Auditor General (Bangladesh SAI)
- SLA = Service Level Agreement
- CIA (data audit context) = Confidentiality, Integrity, Availability
- Current cyber-offence law (as of context): Cyber Security Ordinance, 2025

- Current data protection law: Personal Data Protection Act, 2026
- Govt data governance rules: National Data Governance Ordinance, 2025
- Legacy law still partly active: ICT Act, 2006 (digital signature/e-commerce only)

Exam Strategy Notes

- Law-sequence questions (DSA2018→CSA2023→CSO2025; PDPO2025→PDPA2026) – expect fill-in-blank or ordering.
- Scenario Qs almost always map to one of: Access Control/Identity, Data Audit, Vendor Audit, Incident Response – identify which domain failed first, then cite the matching checklist item.
- "Senior officials remain accountable even if outsourced" – recurring answer for any vendor-blame scenario.
- NID case = go-to real example; can be cited as evidence for almost any government audit failure question.

Software Maintenance

Software Maintenance

Chapter 1: Basic Concepts and Preliminaries

1. Definitions

Software Maintenance: preventing software from failing to deliver intended functionality, via bug fixing. Event-driven, planned, no major architecture change. "Keeping installed system running, no change to design."

Software Evolution: continual change from lesser/simpler/worse state to higher/better state. Creating new but related designs from existing ones — new functionality, better performance, new platforms.

Key distinction (Bennett & Xu):

- Maintenance = all support activities after delivery
- Evolution = activities that change requirements

Legacy System: old program still used because it meets user needs, despite outdated tech. Resists modification. Falls in "Phaseout" stage.

Reengineering: examination + alteration of existing system to reconstitute it in new form + implementation of new form. Formula: **Reengineering = Reverse Engineering + Δ (alteration) + Forward Engineering**

Reverse Engineering: process of defining abstract, easier-to-understand representation of system (e.g., code $\rightarrow$ architecture). Does NOT change the system.

Forward Engineering: moving from high-level abstract design to physical implementation (traditional dev direction).

Impact Analysis: identifying portions of software potentially affected by a proposed change. Used for planning, making, tracking changes.

Refactoring: changing internal structure of software to make it easier to understand/cheaper to change, WITHOUT changing external/observable behavior. Verified via regression testing.

Program Comprehension: understanding existing system for planning/design/coding/testing changes. ~50% of maintenance effort (Corbi, 1989).

Software Reuse: using existing software knowledge/artifacts in new system development. 4 types (Capers Jones): Data reuse, Architectural reuse, Design reuse, Program reuse.

2. Evolution vs Maintenance

Def

	Maintenance	Evolution
Trigger	Bug/defect (event-driven)	Changing requirements/knowledge(req driven)
Scope	Fix, preserve functionality	New/better designs, requirements
Architecture	Unchanged	Can change significantly
Nature	Reactive	Growth/improvement-oriented

Timeframe

Theoretical basis- lehman's laws - ieee/iso standards

3. Software vs Hardware Maintenance

- Maintenance of physical systems often requires replacing broken and worn-out parts. For example, owners replace the worn-out tires and broken lamps of their cars. Similarly, a malfunctioning memory card is replaced with a good one.
- On the other hand, software maintenance is different than hardware maintenance.
- In hardware maintenance, a system or a component is returned to its original good state.
- On the other hand, in software maintenance, a software system is moved from its original erroneous state to an expected good state.

4. Software Maintenance Categories (memorize — very testable)

Mainly 2 types - **Intention based** and **Activity based**

Origin: Swanson (1976) defined 3 categories — corrective, adaptive, perfective. Later, ISO/IEC 14764 standard added a 4th — preventive. Some researchers view preventive as a subset of perfective.

1. Corrective Maintenance

Purpose: fix failures — processing failures (wrong output) and performance failures (can't meet real-time requirements). Isolate + correct defective elements. **Reactive** — done after a defect is detected.

Example: program aborts or produces incorrect results → fix it.

2. Adaptive Maintenance

Purpose: adapt system to changes in its data/processing environment. System changes/additions/deletions/enhancements to meet evolving environment needs.

Examples: support new hardware config; convert batch → online; make compatible with other apps; add WiFi support to a smartphone app that only had 3G.

3. Perfective Maintenance

Purpose: improvements — user experience, processing efficiency, maintainability.

Examples: make output more readable; make program faster; restructure code for readability/maintainability.

Also called "**maintenance for the sake of maintenance**" or "**reengineering**."

4. Preventive Maintenance

Purpose: prevent future problems before they occur — look ahead, identify risks, act **proactively**.

Example: good programming style reduces impact of future changes → fewer failures.

Restructuring code for comprehension.

"Software rejuvenation": proactively clean internal state (terminate, clean, restart) to prevent severe crashes — increases downtime slightly but avoids costlier failures. Common in safety-critical/high-availability systems.

Classic exam example: Patriot missile software — warning about "very long running time" affecting targeting accuracy issued, but restart-every-8-hours guidance not properly communicated → accident. Illustrates why preventive maintenance matters in safety-critical systems.

Quick memory table

Type	Trigger	Goal	Nature
Corrective	Defect/failure	Fix wrong behavior	Reactive
Adaptive	Environment change	Stay compatible	Reactive to external change
Perfective	Desire for improvement	Better UX/speed/maintainability	Proactive
Preventive	Anticipated future risk	Avoid future failures	Proactive

Activity-Based Classification (Kitchenham) — alternate view

Instead of *intention*, classify by *activity performed*:

- **Corrections** — fix **discrepancy** between expected and actual behavior
- **Enhancements** — change system behavior/implementation, split into 3:
 - modify existing requirements
 - create new requirements
 - modify implementation without changing requirements

EXERCISE SOLUTIONS (Chapter 1, End of Chapter)

1. Differences between software evolution and software maintenance.

Maintenance = keeping system running / fixing bugs without changing design; planned, event-driven, preserves functionality. Evolution = continual change from simpler/worse to better state; creates new but related designs, changes requirements, drives major architectural/functional growth. Maintenance is corrective/preserving; evolution is generative/improving.

2. Why must a software system in real-world use be changed to not become progressively less useful?

Per Lehman's 1st Law (Continuing Change): the operational environment, user needs, and assumptions embedded in the system constantly shift. If the system isn't adapted, its assumptions become invalid, mismatches with the real world grow, and it fails to satisfy evolving user needs — hence it becomes progressively less useful/satisfactory unless continually adapted.

3. Characteristics of maintaining software vs. new software systems.

- Maintenance is event-driven (triggered by CRs/bug reports); development is requirements-driven.
- Maintenance works on existing artifacts/constraints (existing architecture, code, docs); development starts fresh.
- Maintenance activities: investigation, modification, management, QA — often smaller-scope, localized changes.
- Maintenance requires understanding someone else's code (program comprehension ~50% effort) vs. building own mental model from scratch in development.
- Maintenance preserves overall system behavior/design; development defines it.

4. Which maintenance type is the exception, when a change leaves functional spec unchanged but affects design & source code? Options: (a) Corrective (b) Adaptive (c) Perfective (d) Preventive

Answer:(a) Corrective maintenance

Why the other three fit "unchanged functional spec, but design/code changes":

- **Adaptive** — you change the code to work with a new environment (new OS, new hardware, new database). The feature set (what the software does for the user) stays the same — you're just making it work somewhere new.
- **Perfective** — you improve the code's structure, speed, or readability. Again, the software still does exactly what it did before — you're just making it better internally.
- **Preventive** — you restructure code now to avoid future problems. Still no change to what the software does — just protecting it for later.

All three of these are planned, deliberate choices made *while* keeping functionality exactly the same. That's their whole point.

Why Corrective is different:

Corrective maintenance means fixing a bug — something in the software is broken or producing wrong output. Yes, technically the functional spec (what it's *supposed* to do) doesn't change. But

corrective maintenance isn't really about "choosing to preserve functionality while improving the system" — it's a reactive fix forced by a failure. The system was already failing to deliver its spec correctly; you're not preserving working behavior, you're repairing broken behavior.

So the other three are proactive, intentional changes with unchanged functionality as a design goal. Corrective is reactive — triggered by something going wrong, not a planned quality improvement. That reactive nature is what makes it the odd one out.

8. Is "a complex change likely to introduce new defects even when fixing others" a cost, benefit, or risk of a change request? Answer: Risk. Reasoning: it's an uncertain, potential negative outcome (possibility of introducing new defects) rather than a guaranteed cost (which would be a known resource expenditure) or a benefit. It should be weighed in impact/risk analysis before approving the change request — tied to concepts of ripple effect and change propagation.

9. System A (legacy) → migrate data weekly to System B for weekly sales-pattern analysis. Extraction/transform takes ~3 hrs for 1 week of data. Best migration frequency?
Options: (a) Migrate on update (b) Daily at 2am (c) Weekly, Sunday 2pm (d) Monthly, last Sunday 2am

Answer: (c) Migrate weekly, every Sunday at 2:00 p.m. Reasoning: System B's purpose is to analyze week-to-week sales pattern changes — so weekly granularity matches the actual use case exactly. Migrate-on-update or daily would waste resources (extraction takes 3 hrs, no need for finer granularity than the analysis requires) and add unnecessary load on the mission-critical System A. Monthly would be too infrequent — managers need weekly comparisons, and monthly migration would delay/aggregate data beyond what's needed, undermining the "week to week" analysis goal. Sunday evening (low-usage time) minimizes disruption to System A's mission-critical operations.

14. Benefits of reusing software.

- Better quality: previously tested modules have fewer faults → lower overall failure rate of new software
- Increased productivity: saves time/resources vs. building from scratch (though gains depend on size/complexity of reused vs. new components)
- Reduced development cost (dev cost is only ~40% of total lifecycle cost — Gartner)
- Significant maintenance benefit: cost savings during maintenance from reuse ≈ 2x savings during development (Schach's empirical study)

Chapter 2: Taxonomy of Software Maintenance and Evolution

1. Three Classification Schemes of Maintenance Activities (CORE TOPIC)

A. Intention-Based Classification (Swanson / ISO 14764)

Classify by **why** the change was made — the intent behind it. 4 categories: **Corrective, Adaptive, Perfective, Preventive** (see Ch.1 notes for full detail).

B. Activity-Based Classification (Kitchenham et al.)

Classify by **what activity was performed**, not the underlying intent.

- **Corrections** — fix **discrepancy** between expected and actual behavior
- **Enhancements** — effect changes to system (may not change behavior), 3 sub-types:
 - **modify existing requirements' implementation**
 - **add new requirements**
 - **modify implementation without changing requirements**

C. Evidence-Based Classification (Chapin et al., 2001) — MOST TESTABLE, most detailed

Classify based on **objective, observable evidence** — compare software before/after the activity. Independent of who did it, why, or what platform/language/methodology was used.

3 Objectives of this classification:

1. Base classification on objective evidence (observation/comparison before & after) — no need to know the original developer's intent.
2. Granularity should reflect the actual mix of activities seen in practice.
3. Classification independent of hardware, OS, design methodology, language, org practices, personnel.

4 entities examined for evidence of change:

- The whole software
- The external (noncode) documentation
- The properties of the program code
- The system functionality experienced by the customer

12 Mutually Exclusive Types (grouped into 4 clusters, low→high impact):

Cluster	Types (low→high impact)
Support Interface (software NOT changed)	Training → Consultive → Evaluative (default)
Documentation (code NOT changed)	Reformative → Updative (default)
Software Properties (function NOT changed, but code changed)	Groomative → Preventive → Performance → Adaptive (default)
Business Rules (function changed)	Reductive → Corrective → Enhancive (default)

Decision Tree logic (3 criteria questions, ask in order):

- **A: Was software changed?** No → Support Interface cluster
- **B: Was source code changed?** No → Documentation cluster
- **C: Was function (customer-experienced) changed?** No → Software Properties cluster; Yes → Business Rules cluster

Type definitions (know these cold):

- **Training** — use software as subject to train stakeholders
- **Consultive** — software used as basis for consultation (cost estimates, help desk, expert knowledge sharing)
- **Evaluative** — evaluate software (review code/docs, ripple effect examination, testing, debugging) — default/overall default type when ambiguous
- **Reformative** — modify noncode docs to meet stakeholder needs (readability, style)
- **Updative** — modify noncode docs to conform to implementation (replace outdated docs, add test plans) — default of Documentation cluster
- **Groomative** — anti-regression activities: substitute algorithms/modules, rename data, change access auth, backups, recompiling
- **Preventive** — constrain scope of future maintenance (improve maintainability, no visible customer impact)
- **Performance** — modify performance properties/characteristics (uptime, efficiency, storage, reliability) — visible to customer
- **Adaptive** — different technology/resources used (port to new platform, more COTS use, new protocols) — default of Software Properties cluster
- **Reductive** — constrain/reduce/remove customer-experienced functionality
- **Corrective** — fix bugs in customer-experienced functionality
- **Enhancive** — substitute/add/expand customer-experienced functionality — default of Business Rules cluster; MOST FREQUENT type in practice

Overall default type if ambiguous evidence: Evaluative.

Worked example (book's own — very likely to reappear as exam question): Engineer modified code for one component only (no other docs changed), rebuilt, ran regression suite, checked into version control, deployed. Customer only observed improved latency. → A=Yes (software changed), B=Yes (code changed), C=No (function unchanged, only performance/latency changed) → Falls into Software Properties cluster → **Performance type** is dominant (higher-impact than Evaluative, which also applies due to regression testing/evaluation activity). **Answer: Performance is the dominant maintenance type.**

2. Categories of Maintenance Concepts (4 dimensions)

Overview categories affecting maintenance: **Maintained Product, Maintenance Types, Maintenance Organization Processes, Peopleware.**

2.1 Maintained Product

- **Product** — coherent collection of artifacts (source code = key artifact; also manuals, help)
- **Product upgrade** — change/upgrade tied to a specific baseline (new version, patch, or restriction notice)
- **Artifact** — textual/graphical docs, COTS products, object code components

Key elements affecting maintenance performance:

- **Size** — small = 1 person; medium = team; large = multiple teams; larger = less efficient (harder root-cause analysis, ripple effect ID)
- **Age** ("software geriatrics") — older = harder to maintain (hard to find personnel/tools, docs may be missing)
- **Application domain knowledge** — domain expertise boosts maintainer productivity (e.g., computer engineer > aerospace engineer for IP networking maintenance); safety-critical domains demand preserving/increasing reliability
- **Level of abstraction** — in-house design needs CASE tool access; COTS-composed systems need integration/wrapper/glue-code skills
- **Initial product quality** — good quality = easier maintenance; poor/no docs → need reverse-engineering tools

Staged Model Maintenance Tasks by life cycle stage (Table 2.4):

Stage	Maintenance Task	User Population
-------	------------------	-----------------

Initial development	–	–
Evolution	Corrections, enhancements	Small
Servicing	Corrections	Growing
Phaseout	Corrections	Maximum
Closedown	Corrections	Declining

2.2 Maintenance Types (Activities)

4 broad classes of activities:

- **Investigation activity** — evaluate impact of a proposed change
- **Modification activity** — change system's implementation
- **Management activity** — configuration control / maintenance process management
- **Quality assurance activity** — ensure modifications don't damage system integrity (e.g., regression testing)

Corrective maintenance needs precise fault-location skills; enhancement needs broad system comprehension + dev environment (version control).

2.3 Maintenance Organization Processes

Two levels: **individual-level** (implementing one CR) vs **organization-level** (managing the whole CR stream).

Key concepts:

- **Development technology / Paradigm** — constrains maintenance procedures (e.g., SOA-developed system needs SOA-literate maintainers)
- **Procedure** types: **Method** (systematic, clear steps), **Technique** (less rigorous), **Script** (guideline for a document type)
- **Service-Level Agreement (SLA)** — contract specifying performance targets between customer & maintenance provider
- **Maintenance management** — manages the whole service (not just individual CRs); handles event mgmt, change control, config control
- **Event management** — handling stream of incoming CRs/problem reports
- **Change control** — evaluating investigation results, deciding whether to approve a change (via Change Control Board)
- **Configuration management** — maintaining system integrity (version/mod status tracking)

3 Support Levels (staffing hierarchy) — testable:

- **Level 1** — files problem reports, routes to right support person
- **Level 2** — experts who analyze user problems, recommend quick fixes/workarounds
- **Level 3** — programmers who perform actual code changes

2.4 Peopleware

- **Maintenance organization** — org that maintains the product
- **Client organization** — org that uses the system; relationship defined by SLA
- **Human resource** — maintenance personnel (managers, engineers) + client personnel (customers, users)

Key factors:

- Maintenance often seen as less prestigious than development → often assigned to novices → risk of introducing new defects (poor system understanding)
- Separation of dev vs. maintenance staff affects process: no separation → continuous evolution, same tools; full separation → maintainers may need specialized tools, especially if docs are poor
- User/customer factors: **size** (customer base size affects effort), **variability** (diverse customer needs widen maintenance scope), **common goals** (misalignment between customer and end-user needs → poor SLAs)

3. COTS-Based Systems Maintenance (from Ch.1, tested again here in context)

Motivations same as custom-built, but activities differ due to:

- **Skills** — CBS needs component monitoring/integration skills (black-box view), not just code analysis
- **Infrastructure/organization** — support group needed, cost may be shared across projects
- **COTS maintenance cost** — includes purchase/license/upgrade/training; shifts cost from in-house dev to fees
- **Larger user community** — resource (shared discovery) but less control
- **Modernization** — vendor keeps pace with tech (avoids obsolescence) but increases forced-upgrade costs/risk
- **Split maintenance function** — vendor maintains component; host org maintains overall system → coordination overhead
- **More complex planning** — multiple vendors/technologies = higher unpredictability

4. Evolution of Software Systems

4.1 SPE Taxonomy (Lehman, ~1980) — see also earlier answer in this chat

- **S-type**: fully/formally specified; correctness = matches formal spec; real-world change = entirely new problem (rare in practice)
- **P-type**: problem abstracted for practicality (exact solution exists but impractical); acceptability = solution "makes sense"; more dynamic, incremental changes
- **E-type**: embedded in real world, changes as world changes; acceptability = stakeholder satisfaction; unbounded domain; feedback-driven evolution — this is where Lehman's 8 Laws apply

4.2 Lehman's 8 Laws of Software Evolution (full detail — HIGH YIELD)

Apply only to **E-type** systems (CSS — closed source software, studied empirically).

#	Law (Year formulated)	Statement
I	Continuing Change (1974)	E-type programs must be continually adapted else become progressively less satisfactory
II	Increasing Complexity (1974)	Complexity increases unless work is done to maintain/reduce it
III	Self-Regulation (1974)	Evolution process is self-regulating; process/product measures ~normally distributed
IV	Conservation of Organizational Stability (1978)	Average effective global activity rate is invariant over product's lifetime
V	Conservation of Familiarity (1978)	Average content (growth) of successive releases is constant during system's life
VI	Continuing Growth (1991)	Functional content must continually increase to maintain user satisfaction
VII	Declining Quality (1996)	Perceived declining quality unless rigorously maintained/adapted to environment
VIII	Feedback System (1971–1996)	Evolution processes = multi-agent, multi-level, multi-loop feedback systems

Explain each briefly (conceptual Qs):

- **Law I:** assumptions valid at release become invalid as users/environment change → CRs emerge → vicious cycle of adapt-satisfy-adapt.
- **Law II:** adding changes → more modules/subsystems → more interfaces, errors, inconsistency risk → quality/evolution rate decline unless preventive maintenance invests in reducing complexity.
- **Law III:** large systems have their own "dynamics" — size, release intervals, fault counts stay roughly invariant release-to-release due to structural/organizational constraints (checks & balances across finance/business/HR/etc.).
- **Law IV:** changing resources/staffing has little long-term effect (cf. Brooks's Law — more staff can even reduce productivity due to communication overhead).
- **Law V:** too much change per release → developers/users lose mastery → temporary slowdown needed to re-familiarize; adding many new features → more new faults from unfamiliarity.
- **Law VI:** programs have "finite behavior" — limited relative to full domain potential; excluded properties become sources of errors/complaints → must keep growing functionality.
- **Law VII:** quality (from stakeholder perspective) declines due to growing entropy, faults, and domain mismatch unless continually adapted; entropy grows ~1–3/year; ties to "software aging"/"code decay."
- **Law VIII:** multi-loop (iterative), multi-level (affects multiple software/doc aspects), multi-agent (agents cooperate/compete) feedback drives evolution.

Important distinctions to not confuse:

- Law I (Continuing Change) = domain/operational changes happening TO the system
- Law VI (Continuing Growth) = need to GROW functionality to satisfy users
- These reflect different mechanisms — often hard to tell which underlies an observation (exam trap!)

Two types of "aging" (Visaggio) - conceptual:

- **Process execution aging** - performance degradation/transient failures while running continuously
- **Product aging** - quality degradation of code/docs due to frequent changes Symptoms of product aging: **Pollution** (unused modules/components), **Embedded knowledge** (domain knowledge scattered undocumented in code), **Poor lexicon** (meaningless identifiers), **Coupling** (elaborate control/data flow linking).

Code decay: code becomes very difficult to change — measured via (i) cost of change, (ii) calendar time to change, (iii) quality of changed software. Antithesis of evolution (evolution improves; decay degrades).

4.3 Empirical Studies of Lehman's Laws

- Belady & Lehman (1976): studied 20 releases of OS/360 → proposed first 5 laws.

- Yuen: re-examined data → Laws I & II supported; III–V not well supported across other systems; suggested III–V based more on human org behavior than software properties itself.
- **FEAST project** (Feedback, Evolution, And Software Technology): studied OS/360, VME OS, Logica FW banking system, Lucent telecom system. Growth curves ~linear or inverse-square. Increased confidence in Laws I, II, III, V, VI, VIII. Law VII supported by theoretical analysis. Law IV — neither supported nor falsified.
- Lawrence (1982): independent stats study — supported Laws I, II; not III–V.
- Pirzada (1988): Unix vs Lehman/Belady systems — academic vs industrial dev differ in evolution pattern.
- Godfrey & Tu (2000): Linux FOSS growth 1994–99 — **super-linear** (quadratic) growth.
- Schach et al. (2002): studied 365 Linux versions — module coupling grew **exponentially** → consistent with Laws VI & VII but conflicts with Laws III & V.
- Robles et al.: replicated Godfrey & Tu — concluded Laws 3, 4, 5 do NOT hold for large FOSS systems like Linux.

Inverse Square Law (conceptual, may appear as short numeric/analytical Q):

- $\Delta_i = E / s_i^2$ — incremental growth Δ_i inversely related to square of system "mass" s_i (larger mass = more resistance to change)
- Derived from assuming constant effort E per release (Law IV) and complexity $\propto (\text{modules})^2$ (Law II)
- Useful to **predict size of future releases** once a few releases' data is known.

4.4 Practical Implications of the Laws (Lehman's 50+ rules, 3 categories)

1. **Assumptions management** — identify/capture assumptions; periodic review; revalidate on any spec/design/impl/domain change; rewrite components in fast-changing environments; use tracking tools.
2. **Evolution management** — pursue antiregressive work (complexity control, restructuring, documentation) consistently; document assumptions; assess requirement trends in advance; involve domain specialists; use historical metrics/tools for planning.
3. **Release management** — classify release as **safe** (growth $\leq$ mean m), **risky** ($m < \text{growth} < m+2s$), or **unsafe** (growth $\geq \sim m+2s$); distribute growth across releases if unsafe; follow information hiding; alternate enhancement releases with clean-up/restructuring releases.

4.5 Evolution of FOSS Systems

- FOSS = Free/Libre Open Source Software; movement started by Richard Stallman (GNU project, 1984) + Free Software Foundation.
- "Free" = freedom to modify/redistribute; "Open" = source code accessibility (different semantics, easy exam trap).
- Eric Raymond: "The Cathedral and the Bazaar" — described FOSS approach.

FOSS vs CSS — 4 differences:

1. **Team structure** — CSS has dedicated specialist maintenance groups; FOSS mostly voluntary. **Onion model**: core members (small) → contributing developers → active users (testers/bug reporters) → passive users (outer layer, uninvolved).
2. **Process** — CSS = heavy docs/specs/design docs, strict style rules; FOSS = lighter process, source code is main knowledge-transfer artifact, less rigid docs; relies on release notes/bug DBs/mailling lists. **Linus's Law**: "given enough eyeballs, all bugs are shallow" (more public code exposure → faster bug discovery).
3. **Releases** — CSS has scheduled releases; FOSS often has "release early, release often" — dual streams (stable + development), development stream freezes before milestone release.
4. **Global factors** — FOSS devs geographically distributed worldwide via WWW; more coordination challenge than typical CSS teams (even if CSS teams also global, usually more localized).

Chapter: 3 Evolution and Maintenance Models

Lecture 3: Evolution and Maintenance Models

SDLC cost split: Dev = 1/4–1/3 total cost. Maintenance (ops) = rest, ~67%. Breakdown: Req 3%, Prelim design 3%, Detail design 5%, Impl 7%, Test 15%, Ops+Maint 67%.

Maintenance characteristics (defs, key for compare-to-dev Qs):

- Constraint of existing system: mods must fit existing arch/design/code. Dev has no such constraint.
- Shorter timeframe: maint = hrs–months. Dev = 1+ yrs.
- Available test data: maint reuse old test cases as regression + add new ones. Dev builds test cases from scratch.
- Exec vs source code split: released product = executable. Source can change w/o affecting exec in use → need strict SCM/VCS control, else source/exec mismatch. Docs must stay compatible too (tight doc control).

Models (each = definition + how it works)

Traditional SDLC Model

About one-fourth to one-third of all software life cycle costs are attributed to software development, and the remaining cost is due to operations and maintenance.

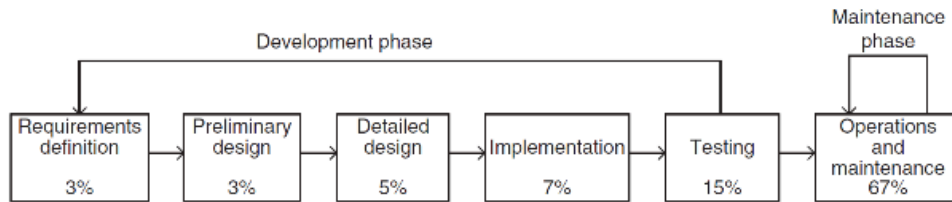


FIGURE 3.1 Traditional SDLC model. From Reference 1. © 1988 John Wiley & Sons

Quick Fix Model: Change made directly to code first, then docs updated after. No prior impact analysis/regression testing often. Resource constraints → changes sometimes undocumented. Old system code → directly patched → new system code (no requirements/design revisit).

Quick Fix Model

Necessary changes are quickly made to the code and then to the accompanying documentation

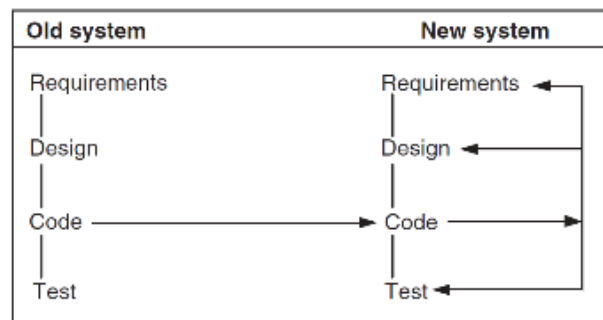


FIGURE 3.2 The quick fix model. From Reference 2. © 1990 IEEE

Iterative Enhancement Model: Changes start at highest-level docs (requirements), propagate down through design→code→test→analysis. Rationale: hard to grasp big req set at once → build system in progressively larger increments.

Iterative Enhancement Model

First changes are made to the highest level documents. Eventually, changes are propagated down to the code level.

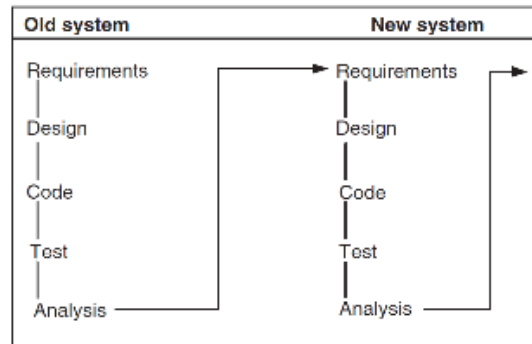


FIGURE 3.3 The iterative enhancement model. From Reference 2, © 1990 IEEE

Reuse-oriented Model: New version built by modifying components of old system + adding new ones. New system reuses many old components.

Full Reuse Model (special case of reuse-based dev): New system built from old system components + repository of artifacts (reqs/design/code/test from earlier/similar systems). Steps: (1) analyze+design new sys, (2) pull matching artifacts from repo. Activities: identify reusable components → understand them → modify for new reqs → integrate.

Full Reuse Model

A new system is built from components of the old system and others available in the repository.

- special case of reuse-based software development.
- The main assumption in this model is the availability of a repository of artifacts describing the earlier versions of the present and similar systems.

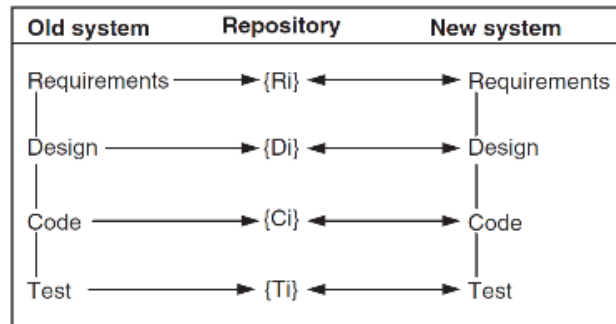


FIGURE 3.4 The full reuse model. From Reference 2. © 1990 IEEE

Key comparative fact (common exam Q): Quick Fix degrades maintainability FASTER than Iterative Enhancement. But counterintuitively, Iterative Enhancement lets orgs do maintenance FASTER than Quick Fix — despite Quick Fix usually chosen when short on time.

	Quick Fix	Iterative Enhancement
Where change starts	Code	Highest-level docs
Analysis before change	Often skipped	Done, propagated down
Doc consistency	Often broken	Maintained

Long-term maintainability	Degrades fast	Degrades slower
Speed of maint over time	Slower (counterintuitive)	Faster

Staged Models for Closed Source Software (CSS):

- **Simple Staged Model stages:** Initial development (build first working version) → Evolution (add features/capability per customer needs) → Servicing (only minor/emergency defect fixes, no new features) → Phaseout (no more servicing, vendor milks revenue) → Closedown (withdrawn, customers migrate away).

The Simple Staged Model for Closed Source Software

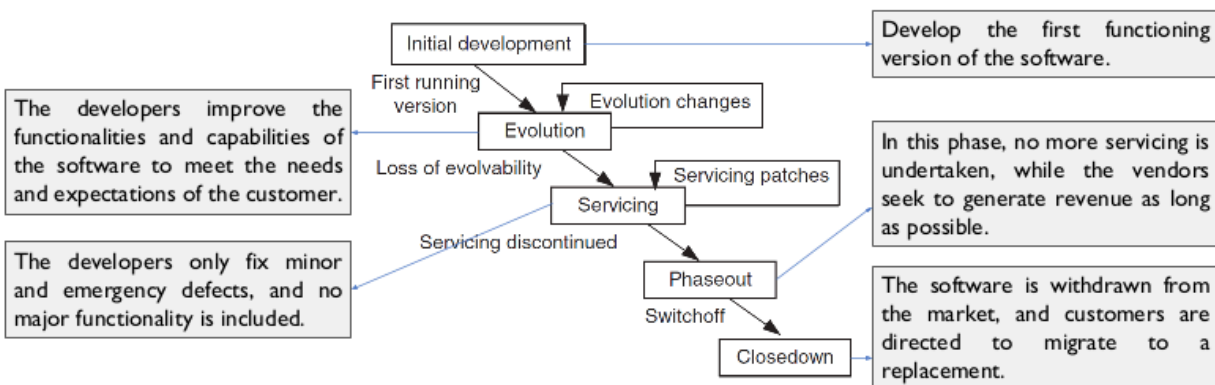


FIGURE 3.5 The simple staged model for the CSS life cycle. From Reference 6. © 2000 IEEE

- **Versioned Staged Model:** same 5 stages but repeats per version (Evolution v1→Servicing v1→Phaseout v1→Closedown v1, then Evolution v2...). Versioning scheme: `<product><version><release><build>` – version = strategic evolution changes, release = servicing patches, build = e.g. daily internal build.

The Versioned Staged Model for Closed Source Software

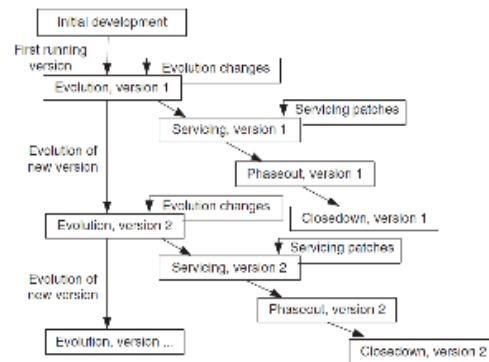


FIGURE 3.6 The versioned staged model for the CSS life cycle. From Reference 6. © 2000 IEEE

Many organizations use a scheme such as <product><version><release><build>, where **version** reflects the strategic changes made to the system during evolution, **release** reflects the servicing patches, and **build** reflects the, say, daily internal build of the software.

- FLOSS Staged Model:** same stages (Initial dev→Evolution→Servicing→Phaseout→Closedown) BUT dashed lines show system can loop back from Servicing/Phaseout to Evolution again (open source = can revive/re-evolve, unlike CSS linear decline).

The Staged Model for Free, Libre, Open Source Software

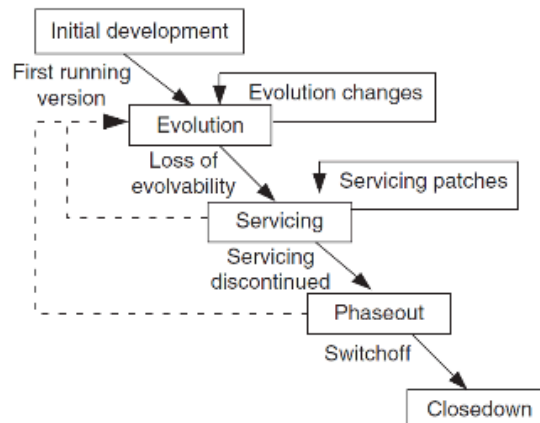


FIGURE 3.7 The staged model for the FLOSS system. From Reference 9. © 2007 ACM

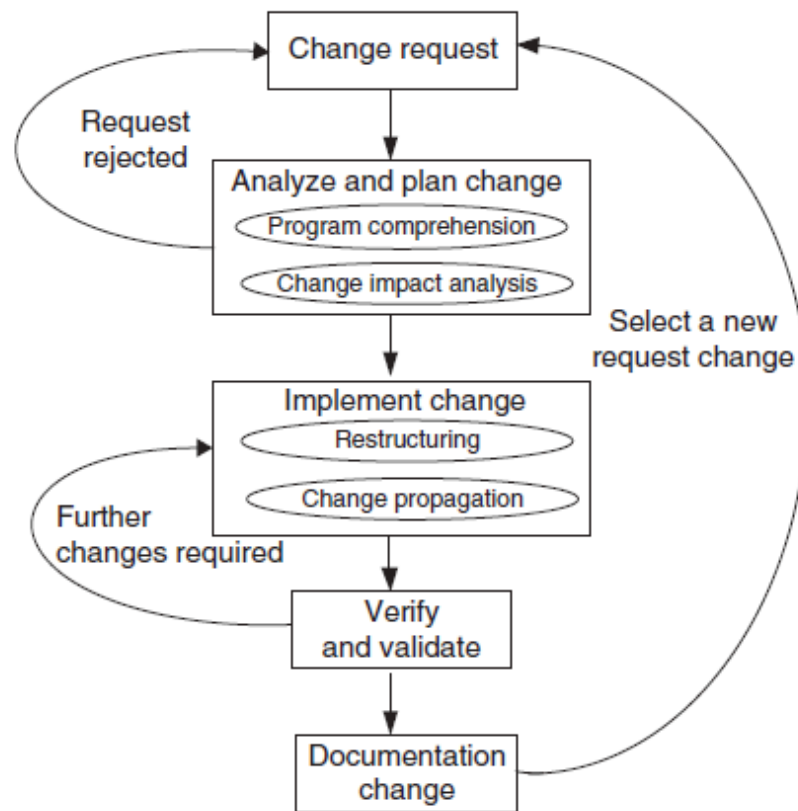


FIGURE 3.8 The change min-cycle. From Reference 12. © 2008 Springer

Lecture 4: Change Request (CR) Workflow

Change Request (CR) / Modification Request (MR): vehicle for recording info about a system defect, requested enhancement, or quality improvement. Defect reports + enhancement requests both documented as CRs. Placed under control of a change management system.

Change management system — purpose (3 pts):

1. Let users understand impact of modifications.
2. Let users identify which products a mod applies to.
3. Give maintenance staff tools to trace all activities req→code. (History: paper form → now repo-stored, linked to actual mods, automated.)

Objectives of change management:

- Common communication method among stakeholders.
- Uniquely identify/describe/track status of each change (simplifies progress reporting, better control).
- Maintain DB of all changes → used for monitoring/metrics.

Describing a CR – 2 key factors:

1. **Correctness:** CR must be unambiguous, easy to review; form documents essential info on SW/HW/doc changes.
2. **Clear communication to stakeholders:** avoid misinterpretation – e.g. marketing collects CRs but may not talk directly to eng team doing the work.

CR Life Cycle (state transition diagram) – memorize the diagram

Life Cycle of a CR

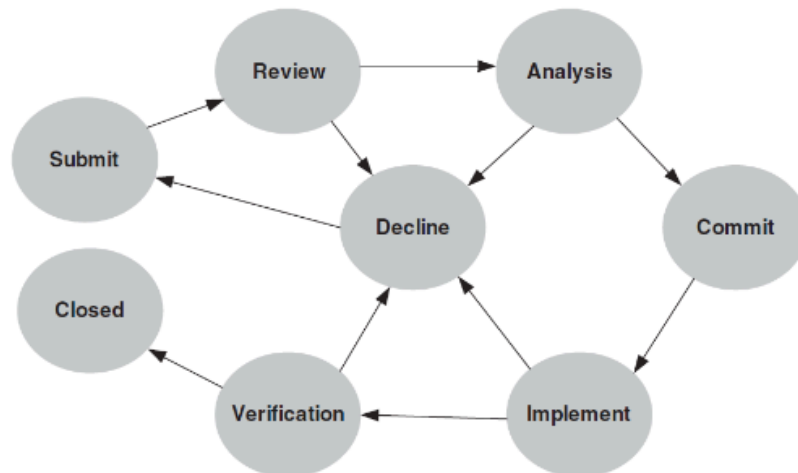


FIGURE 3.27 State transition diagram of a CR

Submit state: initial state of new CR. Sources = end users, customers, marketing managers. Fields: change_request_id, priority, description, maintenance_type, component, note, product, customer. → moves to Review based on priority.

Review state actions:

- Duplicate check (decline dupes, link to original)
- Clarification (ask submitter if ambiguous)
- Attribute review (accept/modify priority + maintenance_type)

- Severity assignment (normal/critical – independent of priority)
- Release planning (target software_release)
- Justification (marketing_justification) Fields: priority, severity, maintenance_type, decline_note, software_release, marketing_justification, description, note. → moves to Analysis.

Analysis state: impact analysis done (understand CR, estimate time_to_implement), high-level functional_spec prepared. Fields: component, problem_origin, impacts, time_to_implement, attachment, functional_spec_title/name/version, eng_assigned. → Decline if infeasible/undesirable, else → Commit.

Commit state: CR waits here before committed to specific release; CRs desired for a release get reviewed; some re-assigned to later release (after consult w/ customer, marketing, director of SE). Activities in this state:

- (i) eng team updates component docs (data/control flow diagrams, schematics),
- (ii) test personnel review CR+spec for testability,
- (iii) write new test cases,
- (iv) select regression tests. Only field updated: committed_release. → Implement.

Implement state: scenarios:

- (a) declined if infeasible;
- (b) if infeasible in current form, director of SE assigns EC (Engineering Change) number + explanation, EC doc linked to CR;
- (c) if doable, eng group writes code + runs unit tests. Fields: decline_note, ec_number, attachment, resolution. → Verification once ready for system-level testing.

Verification state: check compliance via one+ methods: Testing (run test suite), Inspection (code review), Analysis (statistical/math tools), Demonstration (live run). compliance graded: full / partial / noncompliance. If method ≠ Testing, must document rationale in note. Fields: verification_method, verification_status, compliance, tc_id, tc_results, defect_id, testing_note. → Decline if mods rejected; → Closed if tests pass.

Closed state: CR verified as incorporated into software → moved here from Verification.

Decline state - causes:

- Marketing rejects (insufficient business impact)
- Technically infeasible
- Sustaining test manager: changes couldn't be satisfactorily verified → Can be moved back to Submit by marketing after negotiating w/ customer.

Corrective Change & Automated Program Repair

1. Corrective Change

Definition: Software modification triggered by a **defect** — the code isn't doing what it should.

1.1 Three sources of defects

Type	What's wrong	Symptom
Design error	Design doesn't match requirement spec	Feature incomplete or built wrong from the start
Logic error	Flaw in how functionality is implemented	Crashes, unexpected termination, wrong output

Coding error	Code "works" but is inefficient/unchecked	High runtime/memory overhead, sluggish response
---------------------	---	---

Exam angle: Know the distinction — design error = wrong *what*, logic error = wrong *behavior*, coding error = wrong *efficiency* (functionally correct, but slow/bloated). A question might give you a scenario and ask you to classify it.

1.2 Two approaches to applying corrective changes

1. **Rule-Based Bug Detection and Fixing**
 2. **Automatic (Automated) approaches** — i.e., Automated Program Repair (APR)
-

2. Categories of Bugs (Li et al., 2006 taxonomy)

Bugs are classified along **three independent dimensions**: Root Cause, Impact, Software Component.

2.1 Root Cause

- **Memory bugs** — improper handling of memory objects
- **Concurrency bugs** — only manifest in multi-threaded/multi-process contexts: data races, deadlocks, synchronization issues
- **Semantic bugs** — inconsistent with design/programmer intent; this is the **catch-all default** — anything not classified as Memory or Concurrency is Semantic

2.2 Impact

- **Hang** — keeps running, doesn't respond
- **Crash** — halts abnormally
- **Data Corruption** — mistakenly changes user data
- **Performance Degradation** — correct but slow
- **Incorrect Functionality** — doesn't behave as expected
- **Unknown** — impact can't be determined from the bug report

2.3 Software Component

- **Core** — core functionality
- **GUI** — graphical interface
- **Network** — network environment/communication
- **I/O** — input/output handling

2.4 Sub-categories worth memorizing

Memory Bug subtypes:

- Memory Leak — fail to release unused memory
- Uninitialized Memory Read — read before init
- Dangling Pointer — pointer still references freed memory
- NULL Pointer Dereference
- Overflow — illegal access beyond buffer boundary
- Double Free — same memory freed twice

Semantic Bug subtypes:

- Missing Features — feature supposed to exist, doesn't
- Missing Cases — a case in a functionality not implemented
- Corner Cases — boundary cases handled wrong/ignored
- Wrong Control Flow
- Exception Handling — missing/improper
- Processing — wrong expression/equation evaluation
- Typo
- Other Wrong Functionality Implementation

Exam angle: A common trap question gives a bug description and asks you to tag it on all three dimensions at once (e.g., "a buffer overflow in the file-upload module that crashes the server" → Root Cause: Memory/Overflow, Impact: Crash, Component: I/O).

3. Rule-Based Bug Detection and Fixing

Core assumption: Bugs = deviations from **implicit programming rules**.

Where do rules come from?

- Written by developers of bug-finding tools, OR
- Refined from empirical observation "in the wild"

Mechanism: If a code snippet violates a specified rule → flagged as a bug.

Example rule: "release locks after acquiring them"

Rule templates (memorize the three patterns)

Template	Example
"Never/always do X"	"Do not use floating point in the kernel"
"Do X rather than Y"	"Use memory mapped I/O rather than copying"
"Always do X before/after Y"	"Check user pointers before using them in the kernel"

Limitation (implied): Only catches violations of *known, pre-specified* rules — can't catch novel bug patterns nobody wrote a rule for.

4. The Challenges of Program Repair (manual, human context)

1. **Root Cause Localization** — understanding the problem, navigating complex code to find where it actually fails
2. **Strategic Speculation** — forming a fix strategy, anticipating what might work
3. **Iterative Patch Evaluation** — apply patch → test → if fail, use failure to refine → repeat until a final patch works
4. **Compound Friction** — everything above gets *harder* with legacy systems, other people's code, and third-party dependencies

Exam angle: This is the human-repair-process baseline; APR exists specifically to automate/reduce these four frictions.

5. Automated Program Repair (APR)

5.1 Definition (know this cold — likely to be asked verbatim/paraphrased)

Given a buggy program **P** and a test suite **T** such that **P** fails one or more tests in **T**, find a **small modification** of **P** such that the modified program **P'** passes **T**.

- Test suite = correctness criterion (defines "fixed")
- Inputs: buggy program + test suite/spec of correctness
- Goal: patched program passes **all** tests without introducing regressions

5.2 Patch terminology (frequently tested distinction)

- **Plausible patch** — passes the *entire provided test suite* (but may still be wrong)
- **Correct patch** — a plausible patch that **actually** fixes the underlying bug (generalizes beyond the given tests)

→ **All correct patches are plausible, but not all plausible patches are correct.** This gap is the root of the "overfitting" problem (see 5.4).

5.3 APR pipeline (4 stages — diagram-based, likely to appear as a flow/ordering question)

1. **Fault Localization** — buggy program + test suite → identify suspicious elements/code locations
2. **Patch Generation** — produce candidate patch(es) for suspicious elements
3. **Patch Validation** — run candidate against test suite
4. **Correct Program** — output, if validation passes

5.4 Overfitted Repair (important gotcha concept)

A patch can pass all given tests by **hardcoding to the test cases themselves** rather than fixing the actual logic.

Example (from triangle-classifier bug): instead of fixing the faulty condition, patch inserts hardcoded `if (a==2 && b==2 && c==3) return ISOSCELES;` for every failing test input.

Why it's bad: passes the provided suite (= "plausible") but fails on any input outside it — not a real/general fix. This is the **overfitting problem** in APR.

5.5 Goals of APR

- **Reduce developer burden** — auto-suggest likely-correct patches
- **Required inputs** — buggy program + correctness spec
- **Test-driven mechanism** — failing tests localize bug; passing tests define behavior to preserve
- **Ultimate goal** — all tests pass, bug fixed, **no regressions introduced**

5.6 Challenges of APR (list — likely short-answer/MCQ fodder)

- Weak test specifications (tests may not fully capture intended behavior)
 - Patch overfitting
 - Large search space
 - Scalability
 - Patch correctness (plausible $\neq$ correct)
 - Maintainability (is the patch readable/sustainable?)
 - Real-time repair (speed constraints)
-

6. Overview of Repair Techniques (3 families — this is the backbone of the whole topic)

Diagram flow: Buggy Program $\rightarrow$ Fault Localization $\rightarrow$ branches into 3 technique families $\rightarrow$ Patch. A separate branch: Code Corpus $\rightarrow$ Learning/Inference $\rightarrow$ Model of Patches $\rightarrow$ Predict Patch (feeds Learning-Based Repair).

Family	Core idea
Heuristic Repair	Search syntactic variants via mutation + generate/validate loop
Constraint-Based Repair	Extract constraints from failing/passing tests $\rightarrow$ synthesize code via constraint solving

Learning-Based Repair	Learn from a code corpus → build a model → predict the patch
------------------------------	--

6.1 Heuristic Repair (deep dive)

- **Generate-and-Test loop:** search a space of syntactic program variants
- **AST Mutations:** modify program via mutation operators — delete, insert, replace statements/AST nodes
- **Search Strategies:** genetic programming, random search, heuristic search guided by fault localization
- **Validation:** iterate against test suite until a passing patch found

Mutation Selection trade-off (classic exam diagram — the circles/P/X figure):

- Search space **too small** → correct repair might not even be reachable
- Search space **too large** → search becomes intractable / dominated by plausible-but-incorrect patches
- Patch quality spectrum: **Incorrect patch (Px)** → **Plausible but incorrect (P)** → **Correct patch (P✓)**

Two guiding hypotheses for *where* to pull fix material from:

- **Plastic Surgery Hypothesis** — correct code can be "transplanted" from elsewhere in the *same program* to fix the buggy spot
- **Competent Programmer Hypothesis** — programmers are mostly competent; a mistake in one place likely has correct analogous logic written correctly *elsewhere* in the module

Why these work — Code Reusability & Repetitiveness of Software:

- Real code is repetitive; techniques favor **moving/reusing existing blocks** (e.g., an entire `if` condition) over synthesizing brand-new logic
- Example from slides: buggy `if (Character.isDigit(next))` fixed by borrowing the pattern `if (Character.isDigit(next) || next == '.')` found elsewhere in the same file (the "fixing ingredient")

6.2 Constraint-Based Repair (deep dive)

- **Semantic Analysis:** builds a mathematical spec the patch must satisfy

- **Symbolic Execution:** finds an expression that, substituted at the buggy location, makes failing tests pass
- **Repair Constraints:** treats the execution path as a formula → repair = constraint satisfaction problem
- **Program Synthesis:** uses an SMT solver / synthesis engine to generate code satisfying the constraints

Contrast with Heuristic: Heuristic *searches/guesses* variants and tests them; Constraint-based *derives* the fix mathematically from formal constraints – more precise, but needs formal/symbolic machinery.

6.3 Learning-Based Repair (deep dive)

- **Pattern Learning:** learns from large code repositories + historical human bug fixes
 - **Probabilistic Models:** ML/deep neural nets learn patterns of correct code or common fix templates
 - **Patch Generation & Ranking:** predicts/ranks likely-correct patches given bug context
 - **Human-Like Code:** aims for patches that read like human-written code (readability/quality focus)
-

7. LLMs for Automated Program Repair (recent trend – likely a "current state of the field" question)

- Booming research trend: **189 papers (2020–2025)**
- **24 programming languages** covered; **Java, Python, C, C++** most frequent targets
- **78 different LLMs** used; most popular: **ChatGPT, GPT-4, CodeT5, CodeLLaMA**
- **4 ways to leverage LLM knowledge for APR:**
 1. Fine-tuning
 2. Few-shot
 3. Zero-shot
 4. Agents
- Applied across **20 bug types**
- Strong in traditional APR territory (e.g., **semantic bugs**)
- Extended to previously unexplored/rare scenarios: **hardware bugs, Web UI bugs**

8. Quick-Fire Concept Map (for rapid review)

Corrective Change

└─ caused by: Design / Logic / Coding errors

└─ applied via:

└─ Rule-Based Detection (implicit rules → violation = bug)

└─ Automated Program Repair (APR)

└─ Definition: $P + T \rightarrow P'$ passes T (small mod)

└─ Plausible vs Correct patch

└─ Pipeline: Fault Localization → Patch Gen → Patch Validation → Fixed Program

└─ Risk: Overfitting (hardcoded-to-tests patches)

└─ Techniques:

| └─ Heuristic (mutate + search + test; Plastic Surgery / Competent Programmer hypotheses)

| └─ Constraint-Based (symbolic execution + SMT solving)

| └─ Learning-Based (corpus → model → predict patch)

└─ LLM-based APR (fine-tune / few-shot / zero-shot / agents)

9. Likely Exam Question Types

- Classify a described bug by Root Cause / Impact / Component (Section 2)
- Distinguish design vs logic vs coding error given a scenario
- Define APR formally (P, T, P') – Section 5.1

- Explain plausible vs correct patch, and why overfitting happens — 5.2, 5.4
 - Walk through the APR pipeline in order — 5.3
 - Compare Heuristic vs Constraint-Based vs Learning-Based repair — Section 6
 - Explain Plastic Surgery Hypothesis / Competent Programmer Hypothesis and why code reuse works — 6.1
 - Explain the mutation search-space trade-off (too small vs too large) — 6.1
 - State the rule template patterns with an example — Section 3
 - Recall LLM-APR stats/trend (2020–2025 growth, 4 usage modes) — Section 7
-

Source: SE 811 Software Maintenance, Lecture 5 — Toukir Ahammed

Adaptive Change

1. Adaptive Change — Definition

Definition: Change introduced to accommodate modifications in the software's **environment** (not in the software's own logic/requirements — that's the key distinguisher from corrective change).

"Environment" = totality of conditions influencing the product, including:

- Business rules
- Government policies
- Software and hardware operating systems

Modern environment additions (expanded scope vs. classical definition):

- Cloud infrastructure
- Container platforms (Docker, Kubernetes)
- Third-party web services
- Security and compliance requirements

Exam angle: Adaptive change $\neq$ fixing a bug (corrective) and $\neq$ adding a feature (perfective, covered elsewhere). It's about **keeping the software working as the world around it shifts**.

1.1 Illustrative examples

- A library/platform evolves its API → client applications must adapt
 - Porting a mobile app Android → iOS: Java code must become Swift, so it stays compilable/executable on the new platform
-

2. Key Characteristic: Cause is External

- Adaptive maintenance's cause is **usually outside the control** of the organization/developer.
- Hardware changes, new compiler versions, API changes = enhancements imposed by **third parties**.
- Adaptive-maintenance tasks are typically **enterprise/systemwide** – they hit large bodies of source code at once, not a single localized defect.

Real-world scale example: Migrating from .NET Framework → modern .NET (.NET 6, .NET 8) – affected thousands of organizations and millions of lines of code.

Exam angle: If asked "what makes adaptive change distinct from other maintenance types?" – answer: (1) externally triggered, (2) broad/systemwide impact, not localized.

3. Modern Drivers of Adaptive Maintenance (list – likely MCQ)

- Cloud migration
 - Container adoption
 - Operating system updates
 - Programming language evolution
 - Framework upgrades
 - Security requirements
 - Regulatory changes
 - Hardware architecture changes (e.g., ARM64)
-

4. Three Scenarios of Adaptive Change (the core structure of this lecture)

1. **Cross-system software porting**
2. **Cross-language software migration**
3. **Software library upgrade** (i.e., **API evolution**)

Everything below expands on these three.

5. Scenario 1: Cross-System Porting

Definition/trigger: Installation depends on hardware config, software config, device drivers. To run on a *different processor or OS* and stay compatible with different drivers → need adaptive changes.

5.1 Software Forking (important concept)

- **Software forking** = creating a variant product by copying and modifying an existing product.
- Considered an **ad hoc, low-cost alternative** to principled product-line development.
- Maintaining forks requires **porting** features/bug-fixes from one variant to another.

5.2 Classic example: BSD family

- **BSD (Berkeley Software Distribution)** — Unix-based OS, common ancestor.
- **OpenBSD, NetBSD, FreeBSD** — evolved from same origin, but maintained **independently**.
- Modern parallel: porting between **x86** ↔ **ARM64** architectures, or **Linux** ↔ **Windows containers**.

5.3 Empirical findings on cross-system porting (likely short-answer list)

- Maintaining forked projects requires significant effort porting patches across projects.
- Porting is **periodic**, and its rate does **not necessarily decrease over time** (doesn't taper off/disappear).

- **Ported changes are less defect-prone** than non-porting changes.
- Most ported changes propagate to peer projects **relatively quickly**, but some take a very long time.
- Porting activity is **concentrated in specific subsystems/components** – where forked projects share similar functionality/architecture.

Exam angle: These 5 empirical bullet points are prime true/false or fill-in-the-blank material – memorize the direction of each finding (less defect-prone, doesn't decrease, concentrated in shared subsystems).

6. Scenario 2: Cross-Language Migration

Definition/trigger: Software written in one language must be translated to another. Developers rewrite code **and** update language-specific libraries.

Classic motivating case: Legacy system in an old language (e.g., **Fortran**) migrated to a mainstream language (e.g., **Java**) – to ease maintenance and leverage modern language features.

6.1 Cross-Language Program Translation – approaches

- **Hard-coded translation rules:** researchers build tools that manually encode translation rules + implement missing functionality between languages.
 - Example: **COBOL** → **Java** – rules defined to generate Java classes, methods, packages from COBOL programs.
- **Statistical Machine Translation (phrase-based) approach:**
 - Example: **Java** → **C#** migration
 - Method: encode both Java and C# source files as sequences of **syntactic symbols** → align code → train **sequence-to-sequence translation**.

Exam angle: Contrast the two techniques – rule-based (manual, hard-coded) vs. statistical/MT-based (learned alignment, sequence-to-sequence). This mirrors the rule-based vs. learning-based contrast from Lecture 5's APR content – good cross-lecture link if asked to compare.

6.2 Mining Cross-Language API Rules

Problem: API conversion is hard — diverse API usage means an endless task of manually specifying translation rules / identifying mappings.

Automated mining approaches:

- **Java ↔ C# API mapping:** align code based on similar names → construct **API transformation graphs** for each pair of aligned statements.
- **API usage sequence mapping:** via **program dependency analysis**, representing API usage as a **graph-based model**.

Exam angle: Both approaches lean on **graph representations** — one on naming similarity + transformation graphs, the other on dependency analysis + usage graphs. Know this is about *automating* what would otherwise be manual rule-writing.

7. Scenario 3: Library Upgrade & API Evolution

Why it matters: Developers reuse existing frameworks/libraries instead of building from scratch. Ideally, APIs stay stable so upgrades don't ripple into client code. **In reality:**

- APIs change input/output signatures
- APIs change semantics
- APIs get deprecated

→ forces client developers to make adaptive changes.

Extreme case example: Porting a Java desktop app to iOS — language (→ Swift) **and** libraries differ → rewrite from scratch.

7.1 Risks of API Evolution

- API changes can cause **collateral evolution** — changes ripple into dependent code, potentially introducing bugs into previously mature/stable code.
- **Deprecated methods** may affect multiple client applications → forces updates to maintain compatibility.
- **Inadequate adaptation** to API changes → software defects or **build failures**.

Exam angle: "Collateral evolution" is a named concept — know it means a *chain reaction of forced changes triggered by one upstream API change*.

8. Tool Support for API Evolution & Client Adaptation

8.1 CatchUp (record and replay)

- **Assumption:** client-code adaptation changes are exact or similar to the changes made on the library side.
- **Mechanism:** records/stores refactorings in an **XML file** → replayed later to update client code.
- **Limitation:** restricted to only **3 refactoring types**:
 1. **Renaming** operations (types, methods, fields)
 2. **Moving** operations (classes to different packages, static members)
 3. **Change** operations (types, signatures)

8.2 Diff-CatchUp

- Automatically **recognizes API changes** in the reused framework.
- **Suggests plausible replacements** for obsolete APIs, based on **working examples** from the framework codebase.

8.3 SemDiff

- **Mines API usage changes** from other client applications or the library itself.
- Defines an **adaptation pattern** = a *frequent replacement* of a method invocation.
- Logic: if calls to **A.m** are repeatedly changed to **B.n** across several adaptations → **B.n** is likely a **correct replacement** for **A.m**.

Exam angle – compare the three tools (classic comparison table question):

Tool	Core mechanism	Key idea/limitation
CatchUp	Record & replay	Assumes client changes mirror library changes exactly/similarly; limited to rename/move/change refactorings

Diff-CatchUp	Recognize + suggest	Suggests replacements for obsolete APIs using working examples
SemDiff	Mining + frequency analysis	Frequent replacement pattern (A.m → B.n) across many adaptations = likely correct mapping

9. Modern Tool Support (contemporary landscape – likely a "name the tools" question)

IDEs:

- IntelliJ IDEA migration assistants
- Eclipse refactoring
- Visual Studio Upgrade Assistant

Automated refactoring:

- OpenRewrite

Dependency management:

- Maven, Gradle (Java/JVM ecosystem)
 - npm (JavaScript)
 - pip (Python)
 - NuGet (.NET)
-

10. Quick-Fire Concept Map

Adaptive Change

└─ Definition: change driven by environment shift (not internal defect/requirement)

- └ Cause: usually EXTERNAL (3rd party) + enterprise/systemwide scope
- └ Modern drivers: cloud, containers, OS updates, lang evolution, frameworks,
 - | security, regulation, hardware (ARM64)
- └ 3 Scenarios:
 - └ 1. Cross-System Porting
 - | └ Software forking (ad hoc alt. to product-line dev)
 - | └ Example: BSD → OpenBSD/NetBSD/FreeBSD; x86↔ARM64
 - | └ Empirical findings: periodic, less defect-prone, concentrated in shared subsystems
 - └ 2. Cross-Language Migration
 - | └ Example: Fortran → Java
 - | └ Translation approaches: hard-coded rules (COBOL→Java) vs.
 - | statistical MT / seq2seq (Java→C#)
 - | └ API rule mining: name-similarity + transformation graphs;
 - | dependency analysis + usage graphs
 - └ 3. Library Upgrade / API Evolution
 - └ Risks: collateral evolution, deprecated methods, build failures
 - └ Tools: CatchUp (record/replay, 3 refactor types) →
 - Diff-CatchUp (suggest replacements) →
 - SemDiff (mine frequent A.m→B.n replacement patterns)

11. Likely Exam Question Types

- Define adaptive change and explain what "environment" includes
 - Explain why adaptive change is distinct: external cause + systemwide impact (contrast with corrective change from Lecture 5)
 - Name and briefly describe the 3 adaptive-change scenarios
 - Explain software forking and why it's an "ad hoc" alternative to product-line development
 - Recall the 5 empirical findings on cross-system porting
 - Compare hard-coded rule-based translation vs. statistical/sequence-to-sequence translation for cross-language migration
 - Explain "collateral evolution" and how it can introduce bugs into stable code
 - Compare CatchUp vs. Diff-CatchUp vs. SemDiff (mechanism + limitation of each)
 - List modern tool categories: IDEs, automated refactoring, dependency managers – and name examples in each
-

Source: SE 811 Software Maintenance, Lecture 6 – Toukir Ahammed

Perfective Change & Preventive Change

PART A – Perfective Change

1. Definition

Perfective change = change undertaken to **expand existing requirements** of a system.

- Driven by users wanting to use a successful product in **new scenarios beyond its original scope**.
- Causes: enhance existing functionality **or** add new features.
- **Example:** image processing system originally for JPEG → perfective changes add PNG, SVG support.

Exam angle: Perfective = requirement *growth* (contrast: corrective = fix defect, adaptive = react to external environment change). If a question describes "adding a feature nobody asked for at launch, but users now want" → perfective.

2. Tyranny of Dominant Decomposition

Definition: A limitation where choosing **one primary way to structure a system** (e.g., decomposing by classes in OOP) makes it hard/impossible to cleanly separate **other overlapping concerns**.

- A system can be modularized in **only one way at a time**.
- Concerns added later (that don't fit the chosen decomposition) get **scattered across many modules** and **tangled** with one another.

Exam angle: Know the core insight — decomposition is *singular*, but real-world concerns are *plural/overlapping*. Whatever concern didn't get to be "the" decomposition axis suffers.

3. Crosscutting Concerns

Definition: Secondary design decisions that don't fit neatly into a single hierarchy — e.g., **logging, performance, error handling, synchronization**.

- These are **system-wide** requirements.
- Because of the tyranny of dominant decomposition, adding them requires **nonlocalized changes** throughout the system.
- Result: code for these concerns gets **duplicated and spread** across multiple modules → harder to maintain, understand, evolve.

Exam angle: Crosscutting concern = *direct consequence* of tyranny of dominant decomposition. Chain: dominant decomposition → some concerns can't be localized → those become crosscutting → code duplication/scattering.

4. Aspect-Oriented Programming (AOP)

Definition: A programming paradigm aiming to increase modularity by enabling **Separation of Concerns (SOC)** — specifically to solve the crosscutting-concerns problem.

Motivating example: Adding logging to every function.

- Logic is simple (print function name at entry) – but manually inserting into every function body is **tedious and error-prone**.

4.1 Core AOP Concepts (definitely memorize – classic define-the-term question)

Term	Meaning
Aspect	Modular unit (often a special class) encapsulating a single crosscutting concern
Join point	A specific point during execution where an aspect can be plugged in (e.g., method execution, exception handling)
Advice	The actual action/code an aspect executes at a join point
Pointcut	Expression/predicate matching specific join points – defines <i>where/when</i> an advice applies
Weaving	Process of linking aspects with target application objects – at compile time, load time, or runtime

Workflow: Developer defines logging logic as **advice** → specifies where to insert it via **pointcut** (e.g., function entry points) → **aspect weaver** reads the aspect-oriented code and generates equivalent object-oriented code with aspects integrated.

Exam angle: A question might give you a scenario ("logging every method call") and ask you to map it to advice/pointcut/join point/aspect – practice this mapping.

4.2 Why Use AOP? (3 benefits – likely short list question)

- **Eliminates Code Duplication** – no more scattering the same utility code across classes.

- **Cleaner Codebase** — core business methods stay focused on business logic, not infrastructure.
 - **Easier Maintenance** — change utility behavior (e.g., how errors are logged) in **one centralized aspect class**, instead of hunting across the whole codebase.
-

PART B — Preventive Change

5. Definition

Preventive change = change applied to **prevent malfunctions** or to **improve maintainability** of software (not fixing an existing bug, not responding to environment, not adding a feature — it's about internal software *health*).

Why needed: As software is corrected, adapted, and perfected over time, code becomes more complex and **drifts from its original design** → quality drops.

Ties to Lehman's Laws of Software Evolution:

The long-term effect of corrective, adaptive, and perfective changes is **deteriorating software structure** while **increasing entropy**.

→ Preventive change exists specifically to **counteract this entropy increase**.

Exam angle: This is the key linking concept across the whole course — corrective/adaptive/perfective changes *degrade* structure over time (per Lehman), and preventive change is the *remedy*. Expect a question connecting all four maintenance types.

6. Refactoring

Definition: Transforms a program from one representation to another **while preserving external behavior** (functionality and semantics) — the primary technique for preventive change.

Trigger example: After fixing bugs / adding features, code complexity can grow unmanageable → refactoring reduces complexity, increases **readability, reusability, maintainability**.

6.1 Refactoring Process – 5 Activities (memorize the order)

1. **Identify** where to apply what refactoring(s)
2. **Check** that the refactoring preserves program behavior
3. **Refactor** the code
4. **Assess** effect on software quality (complexity, readability, etc.)
5. **Maintain consistency** between refactored code and related artifacts (docs, tests, issue-tracking records)

Exam angle: "Refactoring is not easy" – because it's a 5-step process, not just "rewrite the code." Step 5 (artifact consistency) is often the one forgotten in short-answer responses – call it out explicitly.

6.2 High-Level vs. Low-Level Refactorings

Type	Description	Examples
High-level (composite)	Complex, behavior-preserving transformations	Creating an abstract superclass; subclassing + simplifying conditionals; capturing aggregations/components
Low-level (primitive)	Basic operations	Creating, deleting, or changing a program entity

7. Does Refactoring Always Preserve Behavior? (Nuanced/contradictory – high-value exam topic)

- **Academic definition:** refactoring = behavior-preserving transformation.
- **Practice ≠ theory:** Martin Fowler catalogs **72 types** of structural changes in OO programs, but these **do not necessarily guarantee** behavior preservation.
 - Fowler himself recommends **writing tests first**, because refactorings may change behavior.
- Developers often **interleave** refactoring with other behavior-modifying changes → "pure" refactoring revisions are **rare** in practice.

- **Johnson's definition** (aligned with these findings): *"refactoring improves behavior in some aspects but does not necessarily preserve behavior in all aspects."*

Exam angle: A classic "trick" question: "Does refactoring always preserve behavior?" → Correct nuanced answer: **theoretically yes, empirically/in practice often no** — cite Fowler's 72 catalog + test-first recommendation + Johnson's definition.

8. Contradicting Beliefs on Refactoring Benefits

Two opposing camps:

- **Pro-refactoring:** improves quality/maintainability; skipping it accrues **technical debt**, repaid later via higher maintenance cost.
- **Skeptical view:** refactoring doesn't give **immediate** benefits the way bug fixes/new features do.

8.1 Evidence Supporting Benefits

- Bug-fix time **decreases** after refactoring; **defect density decreases** after refactoring.
- **Productivity** (avg. ticket-resolution time) decreases (improves) after re-architecting.
- In defect-prediction models: refactoring-related features and defects show an **inverse correlation** — more refactoring in prior period → fewer subsequent defects.

8.2 Evidence Supporting the Opposite View

- Refactoring may **introduce bugs**: code churn correlates with defect density, and refactorings correlate with bugs.
- **Single-line changes** = lower error chance; **large commits** (which tend to include refactorings) = **higher** chance of inducing bugs.
- Refactorings often occur **together with other change types**, and are followed by an **increasing** number of bugs.
- Number of bug fixes **increases after API refactorings** specifically.

Exam angle: Be ready to argue **both sides** — this topic is explicitly framed as contested/contradictory, a good essay/discussion question target.

8.3 Why Refactoring May Not Always Be Beneficial (mechanisms behind the "opposite view")

- Refactoring often needs **coordinated edits across different parts** of a system → hard to locate all relevant locations & apply changes consistently.
 - **Incomplete refactorings** cause errors.
 - **Code churn** (lines added/deleted/modified) correlates with defect density; refactoring introduces large structural changes → raises doubts about net benefit.
-

9. Automated Refactoring

- Modern **IDEs** provide automatic support: rename, move, extract method, etc.
- IDE's refactoring engine handles: **precondition check** → **program transformation** → **postcondition check** — developer just picks a command + supplies info (e.g., new method name).

9.1 LLM-Based Automated Refactoring

- Uses LLMs to analyze, rewrite, modernize code by understanding **intent and context**, not just rigid hardcoded rules.
- **Contrast with traditional AST-based tools**: AST tools only handle predictable patterns (e.g., renaming variables).
- LLMs can additionally: fix **complex logical flaws**, explain **architectural changes**, translate **legacy paradigms** into modern code standards.

Exam angle: Compare traditional AST-based automated refactoring vs. LLM-based — AST = rule-based/pattern-limited; LLM = context/intent-aware, handles semantic-level changes.

10. Real-World Refactoring Practices (empirical findings — likely list/short-answer)

- Most developers refactor **in the context of** bug fixes or feature additions — and these combined changes are generally **not semantics-preserving** (echoes Section 7's nuance).
- Most refactoring is done **manually**, not via automated tools.
- Programmers **don't use automated refactoring** even when aware it's available.
- Complaint: refactoring tools **disrupt workflow** — force switching away from focused coding.

- **Risks reported by developers:** regression bugs, code churn, merge conflicts, time taken away from other tasks, difficulty of code review after refactoring, risk of **overengineering**.

Exam angle: Note the disconnect — tools exist and developers know about them, yet adoption is low due to workflow disruption + trust/risk concerns. This gap is a good discussion-question anchor.

11. Quick-Fire Concept Map

Perfective Change

- ├ Definition: expand existing requirements (new use cases beyond original scope)
- ├ Problem: Tyranny of Dominant Decomposition
 - | → system structured ONE way → other concerns can't be cleanly separated
- ├ → Crosscutting Concerns (logging, error handling, sync, performance)
- | → nonlocalized changes, code duplication/scattering
- └ Solution: AOP (Aspect-Oriented Programming)
 - ├ Concepts: Aspect, Join Point, Advice, Pointcut, Weaving
 - └ Benefits: no duplication, cleaner core code, centralized maintenance

Preventive Change

- ├ Definition: prevent malfunction / improve maintainability
- ├ Motivation: Lehman's Laws → corrective+adaptive+perfective changes
 - | degrade structure & increase entropy over time
- └ Main technique: Refactoring
 - ├ Definition: behavior-preserving transformation (in theory)

- |— Process: Identify → Check → Refactor → Assess → Maintain consistency
 - |— High-level (composite) vs Low-level (primitive)
 - |— Reality check: Fowler's 72 refactorings ≠ guaranteed behavior-preserving;
 - | Johnson: "improves some aspects, doesn't preserve all"
 - |— Contested benefit:
 - | Pro: fewer defects, faster bug-fix time, less tech debt
 - | Con: code churn ↑ defects, large commits ↑ bug risk, incomplete refactorors
 - |— Automated refactoring: IDE (rename/move/extract) vs LLM-based (intent-aware)
 - |— Real-world practice: mostly manual, tools underused, workflow disruption complaints
-

12. Likely Exam Question Types

- Define perfective change and give an example of requirement expansion
- Explain "tyranny of dominant decomposition" and how it causes crosscutting concerns
- Define and give an example scenario for each AOP concept (aspect, join point, advice, pointcut, weaving)
- List and explain the 3 benefits of AOP
- Define preventive change and connect it to Lehman's Laws of software evolution
- List the 5 steps of the refactoring process, in order
- Distinguish high-level vs low-level refactorings with examples
- Argue whether refactoring always preserves behavior — cite Fowler + Johnson
- Present evidence for AND against refactoring's benefits (contested topic — expect an essay-style question)
- Compare traditional IDE/AST-based automated refactoring vs LLM-based refactoring
- List real-world adoption findings/risks reported by developers (why don't they use automated tools?)

Inspecting Changes – Software Inspection & Modern Code Review

1. Code Inspections – Core Idea

Definition: Manual examination of software changes/artifacts by developers to improve quality during evolution.

Purpose: Improve software quality during evolution via **code reviews** – manually examining software changes.

When performed: At the **end of major development phases**, aiming to find **overlooked defects** before moving to the next phase.

Exam angle: Key trigger word – inspections are **phase-gated** (happen at milestones), not continuous. This is the seed distinction that separates them from Modern Code Reviews later.

2. Traditional Code Inspections

2.1 Process

- Software artifacts **circulated a few days in advance**.
- Reviewed and discussed in a **series of meetings**.

2.2 Roles in review meetings (memorize – likely matching/MCQ)

Role	Function
Author	Creator of the artifact under review
Other developers	Assess the artifact
Meeting chair	Moderates the discussion
Secretary	Records the discussion

2.3 Trade-off

- **Proven valuable** for improving software quality over the years.
- **BUT:** cumbersome and time-consuming → hinders **universal adoption** in practice.

Exam angle: The core "problem statement" that motivates the entire rest of the lecture: traditional inspections work, but are too slow/heavy for modern development cadence → motivates Modern Code Reviews (MCR).

3. Modern Code Reviews (MCR)

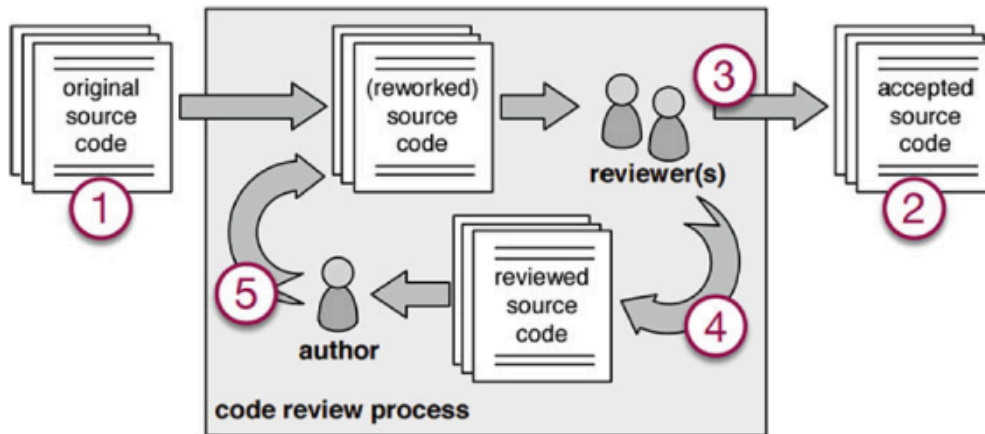
3.1 Definition

A **lightweight, flexible** code review process adopted by most open-source and industrial projects to avoid the inefficiencies of traditional code inspections.

3.2 MCR Process Flow (diagram – 5 numbered steps, likely to be tested as ordering/diagram-labeling)

1. **Original source code** – author submits it for review.

2. (loops through rework) **(Reworked) source code** sent to reviewers.
3. **Reviewer(s)** decide if code meets quality acceptance criteria → if it does, code becomes **accepted source code** (step 2 in diagram numbering – the exit path).
4. If not accepted, reviewers **annotate with comments** → produces **reviewed source code**.
5. **Author** revises code based on comments → cycle repeats (back to reworked source code) until **all reviewers accept**.



Exam angle: Know this is a **cyclical/iterative** process (author ↔ reviewer loop), unlike the one-shot meeting-based structure of traditional inspection.

Contrast with traditional inspection:

Aspect	Traditional Inspection	Modern Code Review
Formality	Formal, meeting-based	Informal, asynchronous
Frequency	End of major phase	Regular, on every change/commit

Change size	Large components	Small, incremental changes
Reviewers	Fixed roles (chair, secretary, etc.)	Small, self-selected group
Tooling	Manual/paper-based	Tool-based (GitHub, Gerrit, etc.)

3.3 Defining Quote (verbatim characterization – good to recognize, don't need to memorize word-for-word but know the gist)

Modern code reviews = *"early, frequent reviews of small, independent, complete contributions conducted asynchronously by a potentially large, but actually small, group of self-selected experts."*

Break this down – 4 defining traits packed into one sentence:

- **Early & frequent** (not phase-gated)
- **Small, independent, complete** contributions (not massive artifacts)
- **Asynchronous** (not scheduled meetings)
- **Self-selected small group** (not everyone participates, even though visible to many)

These 4 traits map directly onto the 4 "Modern Code Review Practices" subsections below.

4. Modern Code Review Practices (4 key empirical characteristics)

4.1 Occur Early, Quickly, and Frequently

- Traditional inspections: happen **after finishing a major component**, last **several weeks**.
- MCR: happen **frequently and quickly**, triggered by each commit/change.
- **Example:** Apache project — review intervals **a few hours to a day**.
- Most reviews across projects are **picked up within a few hours** → reviewers are actively/regularly watching for changes.

4.2 Examine Small Program Changes

- Change size in MCR is **much smaller** than in traditional inspections.
- Smaller changes → developers can **constantly review** and stay up-to-date with peers' activity.

4.3 Conducted by a Small Group of Self-Selected Reviewers

- **OSS projects:** no formal assignment — developers **self-select** changes they're interested in reviewing.
 - Changes/discussions are **broadcast to a large group**, but only a **small number** actually participate.
- **Industrial projects: mixed assignment** — author adds a pool of candidate reviewers; individuals from that pool then self-select based on interest/expertise.

Exam angle: Compare OSS (pure self-selection, no assignment) vs. Industrial (hybrid — candidate pool + self-selection within it).

4.4 Often Tool-Based

- Clear trend toward **review tools** for tasks + communication.
 - **History:** OSS reviews used to be **email-based** (lack of tool support).
 - **Now:** GitHub, BitBucket integrate lightweight review tools — assign reviewers, enter comments, record discussions.
 - **Benefit vs. email/traditional inspection:** tool-based reviews provide **traceability**.
-

5. Do Code Reviews Actually Find Defects? (Nuanced finding — high-value exam topic)

- **Initial/intended purpose:** find defects.

- **Reality (per empirical studies):** practice/outcomes are **less about defect-finding than expected**.
- **Microsoft study finding:** only a **small portion** of review comments were defect-related – and those were mainly **small, low-level logical issues** (not deep architectural bugs).

5.1 What code review *actually* provides (the real value, per findings)

- **Knowledge transfer**
- **Team awareness**
- **Improved solutions** – better practices and readability

Exam angle: This mirrors the "contradicting beliefs on refactoring" nuance from Lecture 7 – a maintenance activity whose *stated purpose* diverges from its *actual observed value*. Good potential essay/compare question: "Refactoring and code review both have purposes that empirical studies show don't fully match reality – discuss." Expect a direct question: "What is the actual benefit of code review found by empirical studies, vs. its intended purpose?"

6. Quick-Fire Concept Map

Inspecting Changes

├─ Code Inspections (Traditional)

| ├─ Phase-gated: happens at end of major dev phases

| ├─ Process: circulate artifact days ahead → meetings

| ├─ Roles: Author, Developers, Meeting Chair, Secretary

| └─ Trade-off: proven valuable BUT slow/cumbersome → low adoption

|

└─ Modern Code Reviews (MCR) – lightweight alternative

├─ Process loop: original code → rework → review →

| (accept) OR (annotate → revise → repeat)

└─ Defining traits: early/frequent, small/independent/complete,

| asynchronous, self-selected small group

└─ 4 Empirical Practices:

| 1. Early/quick/frequent (hours–day turnaround, e.g. Apache)

| 2. Small change sizes (smaller than inspections)

| 3. Small self-selected reviewer group

| (OSS: pure self-select | Industrial: pool + self-select)

| 4. Tool-based (GitHub/BitBucket; traceability benefit)

└─ Actual outcome vs. intended purpose:

Intended: find defects

Reality (MS study): few defect-comments, mostly low-level

Real value: knowledge transfer, team awareness, better practices

7. Likely Exam Question Types

- Define code inspection and explain when in the dev lifecycle it occurs
- List and describe the 4 roles in a traditional code inspection meeting
- Explain why traditional code inspections, despite being valuable, aren't universally adopted
- Walk through the MCR process diagram (author → reviewer → accept/annotate → revise loop)
- Quote/paraphrase and unpack the 4-part definition of modern code review
- Compare traditional code inspections vs. MCR on: timing, change size, participants, tooling
- Explain how OSS reviewer self-selection differs from industrial mixed assignment
- Explain the traceability benefit tool-based review offers over email-based review

- Discuss the gap between code review's intended purpose (finding defects) and its actual empirical benefit (knowledge transfer, team awareness, readability) – cite the Microsoft study finding

Source: SE 811 Software Maintenance, Lecture 8 – Toukir Ahammed

Software Reengineering - Slides / Claude

General Objectives of Reengineering

- Software systems are reengineered by keeping one or more of the following four general objectives in mind:
 - Improving maintainability
 - Migrating to a new technology
 - Improving quality
 - Preparing for functional enhancement.

Levels of Abstraction and Refinement

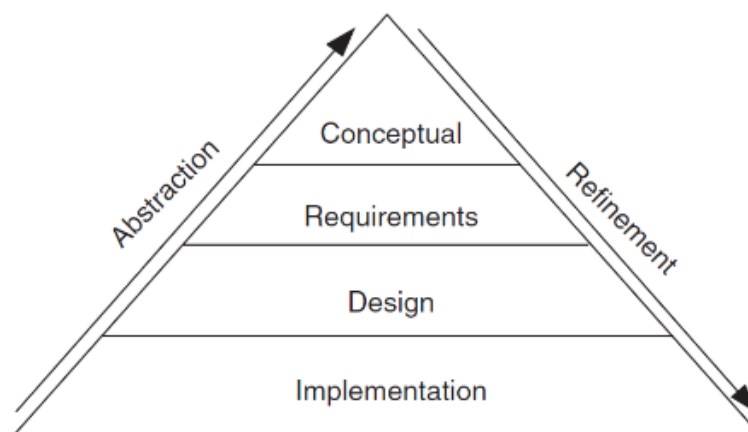


FIGURE 4.1 Levels of abstraction and refinement. From Reference 5. © 1992 IEEE

Forward Engineering

- A new software is created by going downward from the top, highest level of abstraction to the bottom, lowest level.
- This downward movement is known as forward engineering.
- Forward engineering follows a sequence of activities:
 - formulating concepts about the system to identifying requirements to designing the system to implementing the design.

Reverse Engineering

- On the other hand, the upward movement through the layers of abstractions is called reverse engineering.
- Reverse engineering of software systems is a process comprising the following steps:
 - (i) analyze the software to determine its components and the relationships among the components and
 - (ii) represent the system at a higher level of abstraction or in another form.
- Some examples of reverse engineering are:
 - (i) decompilation, in which object code is translated into a high-level program;
 - (ii) architecture extraction, in which the design of a program is derived;
 - (iii) document generation, in which information is produced from, say, source code, for better understanding of the program; and
 - (iv) software visualization, in which some aspect of a program is depicted in an abstract way.

Software Reengineering Model

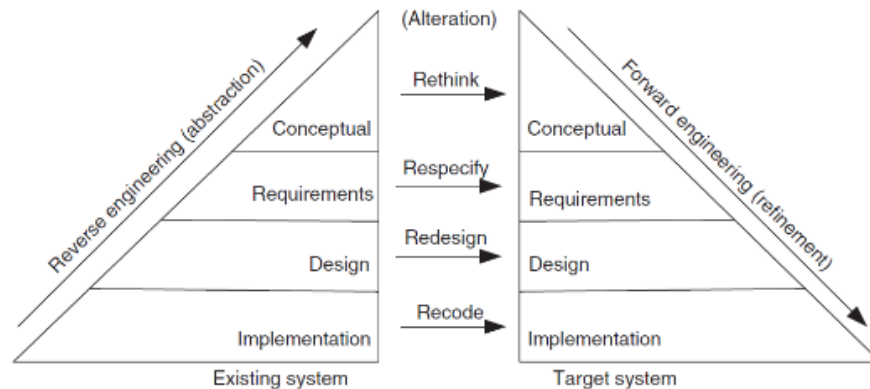


FIGURE 4.3 General model of software reengineering. From Reference 5. © 1992 IEEE

An existing system can be reengineered by following one of four paths.

Objectives Of Reverse Engineering

1. Generating alternative views,
2. Recovering lost information,
3. Synthesizing higher Levels of abstractions,
4. Detecting side effects,
5. Facilitating reuse, and
6. Coping with complexity.

Techniques Used for Reverse Engineering

In order to extract information which is not explicitly available in source code, automated analysis techniques are used. The well-known analysis techniques that facilitate reverse engineering are:

- Lexical analysis
- Syntactic analysis
- Control flow analysis
- Data flow analysis
- Program slicing
- Visualization
- Program metrics

Impact Analysis

1. What is Impact Analysis?

- **Change Request (CR)** triggers maintenance process → maintenance starts with **impact analysis**.
- **Definition (3 equivalent ways to state it):**
 1. Assessment of impact of a change to source code of a module on other modules of the system.
 2. Task of assessing effects of making a set of changes to a software system.
 3. Identifying potential consequences of a modification, OR discovering entities that must be modified to accomplish a change.
- Determines **scope** of change + gives measure of its **complexity**.

Why do it? (Reasons — likely MCQ/short answer)

- Estimate **cost** of executing CR.
- Determine if **critical portions** of system are affected.
- Record **history** of change info for future evaluation.
- Understand how change items relate to **software structure**.

- Determine what portions need **regression testing** after change.

Memory hook: **C**ost, **C**ritical parts, **H**istory, **S**tructure, **R**egression testing (CCHSR)

2. Impact Analysis Techniques (3 classical, dependency-based)

1. **Call-graph-based analysis**
 2. **Static program slicing**
 3. **Dynamic program slicing**
- Example idea: insert change in code → do **forward slicing** from changed point → get set of statements potentially affected.

3. Impact Analysis Process (Figure 6.1 – important diagram)

Flow:

1. Analyze CR + documentation → produces **SIS** (Starting Impact Set)
2. Trace potential impacts → produces **CIS** (Candidate Impact Set)
3. Implement requested changes → produces **AIS** (Actual Impact Set)
4. Feedback loops:
 - **FPIS** (False Positive Impact Set) feeds back into "Trace potential impacts" (things wrongly thought impacted, need re-check)
 - **DIS** (Discovered Impact Set) feeds into "Implement requested changes" (new impacts found during implementation)

Impact Analysis Process

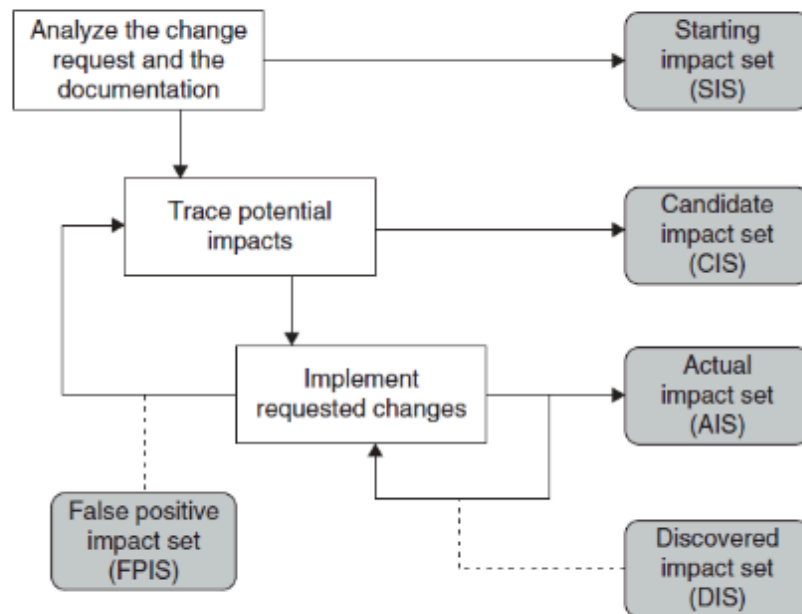


FIGURE 6.1 Impact analysis process. From Reference 6. © 2008 IEEE

Key Terminology (very likely to be asked directly – definitions)

Term	Meaning
SIS – Starting Impact Set	Initial set of components <i>presumed</i> impacted by CR
CIS – Candidate Impact Set	Set of components <i>estimated</i> to be impacted, per some analysis approach
DIS – Discovered Impact Set	New components (not in CIS) found to be impacted <i>while implementing</i> the CR
AIS – Actual Impact Set	Components <i>actually changed</i> as result of the CR

FPIS – False Positive Impact Set	Components estimated impacted but not actually impacted
---	--

Relationship intuition: SIS → (analysis) → CIS → (implementation) → AIS. CIS may overestimate (FPIS = wrongly included) or underestimate (DIS = missed, found later).

4. Evaluating the Impact Analysis Process

- **Recall** = $|CIS \cap AIS| / |AIS|$
 - Fraction of actual impacts that were captured in CIS.
 - Recall = 1 when DIS is empty (nothing was missed).
- **Precision** = $|CIS \cap AIS| / |CIS|$
 - Fraction of candidate impacts that are actually impacted.
 - Precision = 1 when FPIS is empty (no false positives).

Easy trap: Recall relates to **DIS** (missed items), Precision relates to **FPIS** (wrongly included items). Don't mix them up.

(Extra knowledge, standard IR framing: high recall = few false negatives (misses); high precision = few false positives.)

5. Identifying the SIS

- Core challenge = **Concept Assignment Problem**: mapping a CR's human-level "concepts" onto source code components.
- Hard to fully automate because **code and concepts exist at different levels of abstraction** → needs human interaction.
- Methods to identify concepts/features in source code:
 - **grep tool**
 - **Software reconnaissance**
 - **Dependency-graph-based feature location**

Software Reconnaissance (technique detail – could be asked to explain)

Idea: some concepts are "selectable" because execution depends on specific input.

Steps:

1. Execute program **once with** the feature, **once without**.

2. Mark code executed in run 1 but **not** in run 2.
3. Marked code is likely the code implementing that feature.

6. Artifacts Traceability

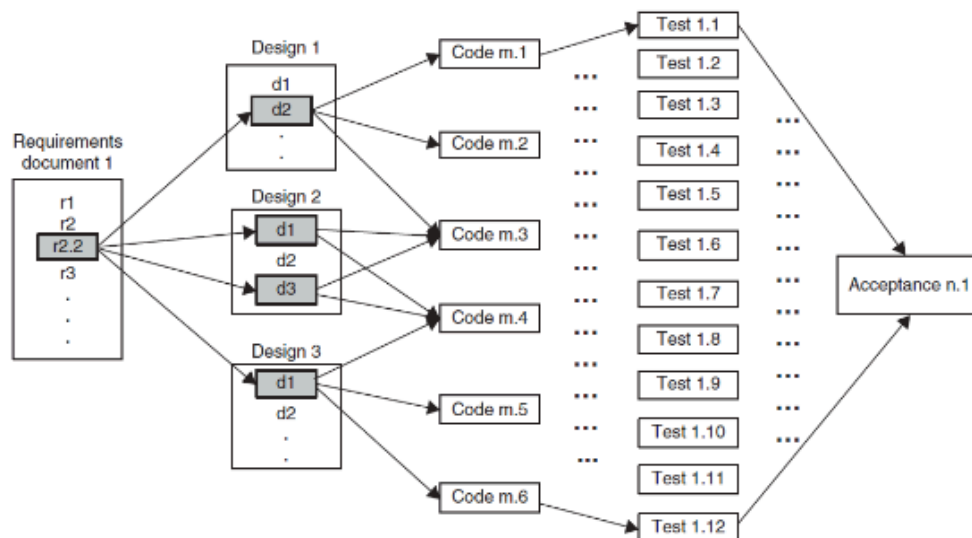
- **Traceability:** ability to describe/follow life of an artifact **forward and backward**; trace between artifacts across the software lifecycle.
- Helps understand relationships among all artifacts in a project.
- After finding high-level docs about a feature, traceability helps maintainer locate entities to change.

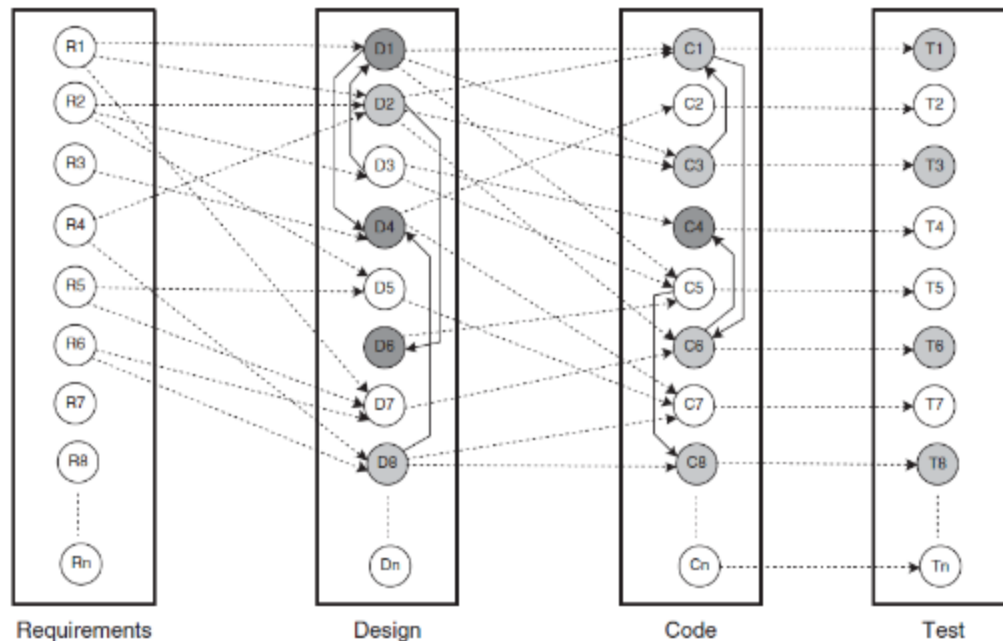
Two kinds of traceability

- **Horizontal (external) traceability** – tracing between **different models** (e.g., requirements → design → code → test).
- **Vertical (internal) traceability** – tracing dependent artifacts **within the same model** (e.g., one requirement to another requirement).

Traceability graph example in slides: Requirements → Design → Code → Test → Acceptance, connected by dotted (trace) links.

Analysis of Traceability Graph





7. Identifying the Candidate Impact Set (CIS)

- SIS is **augmented** with Software Lifecycle Objects (SLOs) likely to change because SIS elements changed.
- Two types of impact between entities:
 - **Direct impact:** exists between two entities related by a **fan-in and/or fan-out** relation (i.e., directly connected in dependency graph).
 - **Indirect impact:** if A directly impacts B, and B directly impacts C, then **A indirectly impacts C** (transitive).

Representing relationships (Tables 6.1–6.3 – conceptual, not memorize numbers)

- **Connectivity matrix:** shows direct relationships only (X = direct edge exists).
- **Reachability matrix:** shows if one SLO can reach another **via any path** (direct or indirect) – much denser than connectivity matrix.
- **Distance matrix:** shows shortest path length (number of hops) between SLOs – used to judge "how far" impact might propagate.

Identifying the Candidate Impact Set

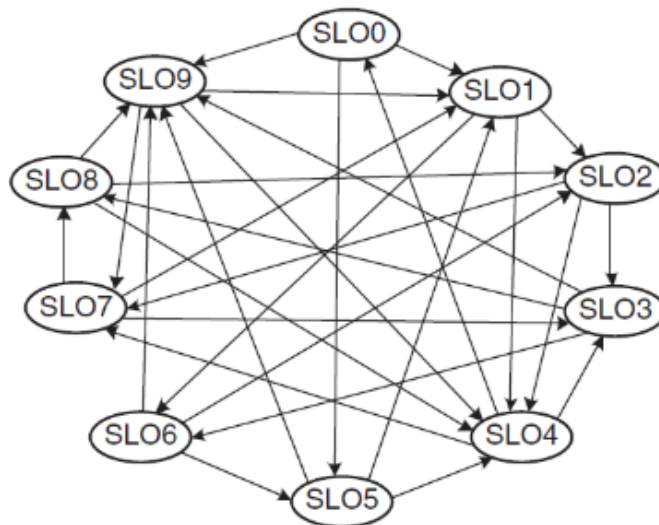


FIGURE 6.5 Simple directed graph of SLOs. From Reference 12. © 2002 IEEE

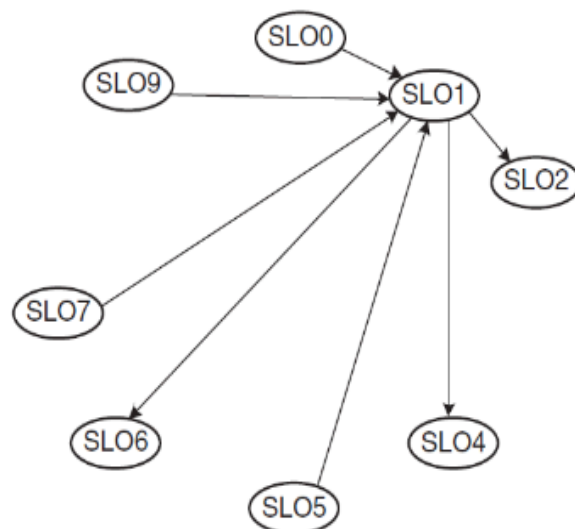


FIGURE 6.6 In-degree and out-degree of SLO1. From Reference 12. © 2002 IEEE

Reducing False Positives – two approaches

1. **Distance-based approach:** SLOs farther than a **threshold distance** are considered NOT impacted (cut off propagation beyond a limit).
2. **Incremental approach:** Build CIS incrementally –

- For each SLO in SIS, look at all SLOs interacting with it.
- Only add SLOs **actually impacted** by the CR.
- Repeat recursively until all impacted SLOs found.

8. Dependency-based Impact Analysis

Call Graph

- **Directed graph**: node = function/component/method; edge $A \rightarrow B$ means **A may invoke B**.
- Used by programmers to understand potential impact of a change (if you change B, anything calling B — directly or transitively — may be affected).
- Execution trace example: sequence like **M B r A C D r E r r r r x** shows actual call/return order (r = return, x = exit) — shows dynamic behavior vs static graph.

Program Dependency Graph (PDG)

- Nodes represent:
 - Each **simple statement**
 - Each **predicate expression** (condition, e.g., **if**)
- Two edge types:
 - **Data dependency edge**: $v_i \rightarrow v_j$ means computation at v_i **directly depends on result** of computation at v_j .
 - **Control dependency edge**: $v_i \rightarrow v_j$ means v_i **may execute based on** the outcome of the condition evaluated at v_j .

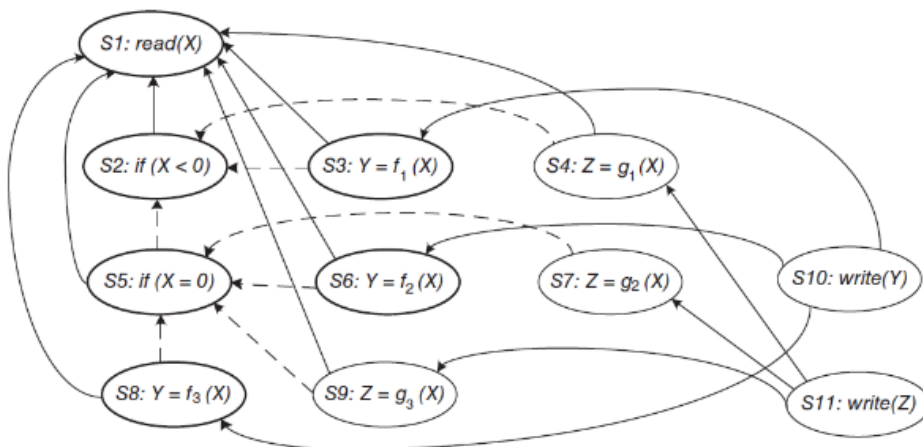
Example Program

```

begin
S1:  read(X)
S2:  if (X < 0)
      then
S3:      Y = f1 (X);
S4:      Z = g1 (X);
      else
S5:      if (X = 0)
            then
S6:      Y = f2 (X);
S7:      Z = g2 (X);
            else
S8:      Y = f3 (X);
S9:      Z = g3 (X);
            end_if;
      end_if;
S10: write(Y);
S11: write(Z);
end.

```

Program Dependency Graph



```

begin
S1:  read(X)
S2:  if (X < 0)
      then
S3:      Y = f1 (X);
S4:      Z = g1 (X);
      else
S5:      if (X = 0)
            then
S6:      Y = f2 (X);
S7:      Z = g2 (X);
            else
S8:      Y = f3 (X);
S9:      Z = g3 (X);
            end_if;
      end_if;
S10: write(Y);
S11: write(Z);
end.

```

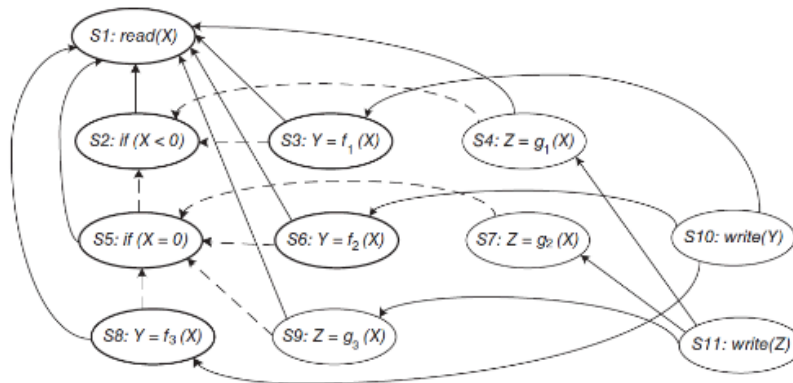
Static Program Slice

- **Definition:** for a variable **var** at node **n**:
 - Find all **reaching definitions** of **var** at **n**.
 - Find all nodes **reachable** from those definition nodes in the PDG.

- Visited nodes = the **slice**.
- Worked example (from slides): variable Y at S10.
 - Reaching definitions of Y at S10: {S3, S6, S8}
 - Nodes reachable from these: {S1, S2, S3, S5, S6, S8} → this is the static slice.
- Key trait: static slice considers **all possible paths/inputs**, independent of any
- specific test/run.

Static Program Slice

- Consider the program and variable Y at S10.
- First, find all the reaching definitions of Y at node S10: {S3, S6, and S8}.
- Next, find the set of all nodes which are reachable from {S3, S6, and S8} {S1, S2, S3, S5, S6, S8}.



Dynamic Program Slice

- Only considers statements executed for a **specific input/test case** — narrower than static slice.
- Method:
 - For the current test, **mark executed nodes** in the PDG.
 - Traverse marked nodes.
- Worked example: $X = -1 \rightarrow$ executed nodes = {S1, S2, S3, S4, S10, S11}.
 - Static slice for Y at S10 = {S3, S6, S8} (all 3 possible definitions).
 - But for $X = -1$, only **S3** actually executes.
 - So dynamic slice narrows down the defect search: if Y is wrong at S10 with $X = -1$, bug is either in **f1 (S3)** or the **condition at S2**.

- **Dynamic slice > static slice for debugging** — because it localizes the fault more precisely (fewer candidate statements to check) for a *given* failing test case.

Dynamic Program Slice

- For the case $X = -1$, the executed nodes are: $\langle S1, S2, S3, S4, S10, S11 \rangle$.

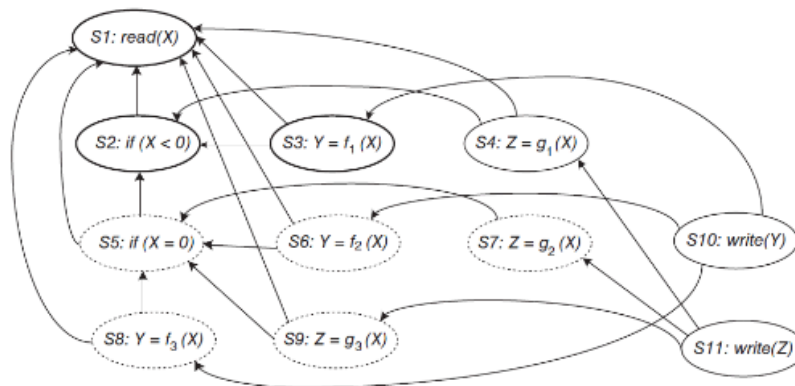


FIGURE 6.11 Dynamic program slice for the code in Figure 6.9, text case $X = -1$, with respect to variable Y

9. Quick Comparison Table (likely conceptual exam Q: "differentiate X and Y")

Concept	Static Slice	Dynamic Slice
Based on	All possible executions/paths	One specific test input
Size	Larger (superset)	Smaller (subset), more precise
Use	General understanding of dependencies	Fault localization for a failing test

Concept	Recall	Precision
Formula	$ CIS \cap AIS / AIS $	$ CIS \cap AIS / CIS $
= 1 when	DIS empty	FPIS empty
Measures	Coverage (did we miss anything?)	Accuracy (did we over-include?)

Concept	Direct impact	Indirect impact
Definition	Fan-in/fan-out relation between 2 entities	Transitive: $A \rightarrow B \rightarrow C$ means A impacts C

10. Likely Conceptual Exam Questions to Prepare

1. Define impact analysis (give 2–3 phrasings).
2. Why is impact analysis performed? (list reasons)
3. Explain the impact analysis process diagram – define SIS, CIS, DIS, AIS, FPIS.
4. Derive/explain Recall and Precision formulas; when do they equal 1?
5. What is the concept assignment problem, and why is it hard to automate?
6. Explain software reconnaissance with steps.
7. Differentiate horizontal vs vertical traceability.
8. Differentiate direct vs indirect impact.
9. Explain distance-based vs incremental approach to reducing false positives in CIS.
10. Define call graph, PDG, data vs control dependency edge.

11. Work through a static slice / dynamic slice example (be ready to trace one given a small program + PDG).
12. Why is dynamic slicing more useful than static slicing for debugging?

Legacy Information Systems

1. What is a Legacy System?

- **Informal definition:** "large software systems that we don't know how to cope with but that are vital to our organization."
- Corresponds to the **Phaseout stage** of the staged model (staged lifecycle model of software evolution).
- **Formal definition:** "any information system that significantly resists modification and evolution to meet new and constantly changing business requirements."
- Also **mission-critical** – essential to the organization's business, must be **operational at all times**.
- **Summary insight:** because skills to maintain it are scarce and the system lacks flexibility, there's a strong urge to **replace it with something modern**.

Key exam phrasing to remember: legacy system = *resists change* + *mission-critical* + *vital but hard to cope with*.

2. Acceptable Features of a Legacy System (checklist – likely to be asked as "list the characteristics")

1. **Large** – millions of lines of code.
2. **Geriatric** – often more than 10 years old.
3. Written in **obsolete programming languages**.
4. **Lack of consistent documentation**.
5. **Poor management of data** – often based on flat-file structures (not proper relational DBs).

6. **Degraded structure** following years of modifications (structural decay — related to software entropy / Lehman's laws of "increasing complexity" and "declining quality" studied elsewhere in the course).
7. Very **difficult, if not impossible, to expand**.
8. Runs on **old processors**, generally much slower.
9. Support team is **totally reliant on expert individuals'** knowledge (bus-factor risk — tribal knowledge, no formal documentation to fall back on).

Memory hook: **L**arge, **G**eriatric, **O**bssolete language, **D**ocumentation missing, **D**ata poorly managed, **S**tructure degraded, **H**ard to expand, **O**ld hardware, **E**xpert-dependent (LGODDSHOE — rough mnemonic, group as you like)

3. Solutions for Legacy Information Systems (LIS) — 6 options

1. **Freeze** — stop making any changes to the system (use as-is, no more maintenance).
2. **Outsource** — hand over maintenance responsibility to an external party.
3. **Carry on maintenance** — keep maintaining it in-house as normal.
4. **Discard and redevelop** — throw away the old system, build a new one from scratch.
5. **Wrap** — build a wrapper/interface around the legacy system so it can interoperate with modern systems without changing its internals.
6. **Migrate** — move the system (schema, data, programs) to a new technology platform while preserving functionality.

Conceptual question likely: "Compare Wrap vs Migrate vs Discard-and-redevelop."

- **Wrap**: cheapest, fastest, keeps legacy core untouched, but doesn't fix underlying problems (technical debt stays).
- **Migrate**: moves to new technology, preserves data/logic, moderate cost/risk.
- **Discard & redevelop**: highest cost/risk, but produces a fully modern system with no legacy baggage.

4. Migration — the 3 Main Steps

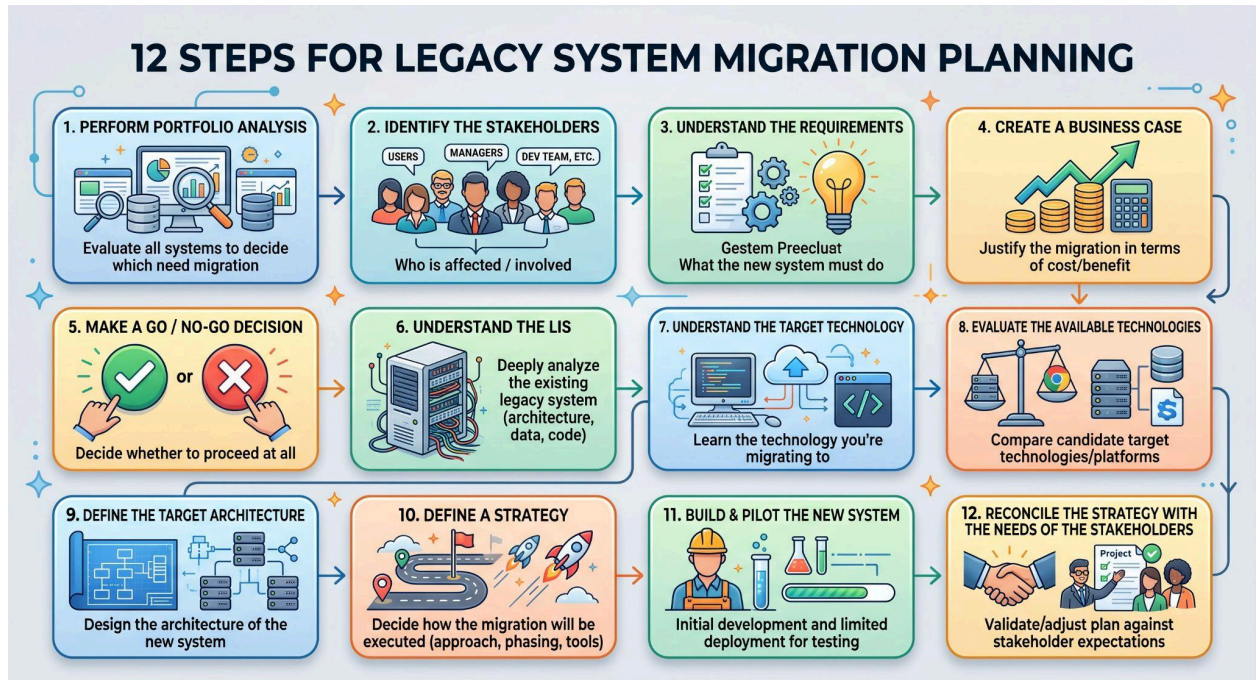
Migration = moving legacy system to a new technology/platform. Comprises:

1. **Schema conversion**
 - Translating the **legacy database schema** into an equivalent structure expressed in the **new technology**.
2. **Data conversion**
 - **Movement of data instances** from legacy database → target database.
 - Requires **three sub-steps: Extract, Transform, Load (ETL)**.
 - **Extract**: pull data out of the legacy database.
 - **Transform**: convert/clean/reshape data to fit the target schema.
 - **Load**: insert transformed data into the target database.
3. **Program conversion**
 - Modifying the program(s) so they **access the migrated (new) database** instead of the legacy data source.

Order matters conceptually: Schema must be defined before data can be transformed into it; programs must be updated to point at the new schema/data.

5. Migration Planning – 12 Steps (likely to be asked to list/explain in order)

1. **Perform portfolio analysis** – evaluate all systems/applications to decide which need migration (see Section 6 below).
2. **Identify the stakeholders** – who is affected/involved (users, managers, dev team, etc.).
3. **Understand the requirements** – what the new system must do.
4. **Create a business case** – justify the migration in terms of cost/benefit.
5. **Make a go or no-go decision** – decide whether to proceed at all.
6. **Understand the LIS** – deeply analyze the existing legacy system (architecture, data, code).
7. **Understand the target technology** – learn the technology you're migrating to.
8. **Evaluate the available technologies** – compare candidate target technologies/platforms.
9. **Define the target architecture** – design the architecture of the new system.
10. **Define a strategy** – decide *how* the migration will be executed (approach, phasing, tools).
11. **Reconcile the strategy with the needs of the stakeholders** – validate/adjust plan against stakeholder expectations.
12. **Determine the resources required** – people, time, budget, tools needed.



- **Decide** (steps 1–5): portfolio analysis → stakeholders → requirements → business case → go/no-go
- **Understand & Design** (steps 6–9): understand LIS → understand target tech → evaluate tech → define architecture
- **Plan Execution** (steps 10–12): define strategy → reconcile with stakeholders → determine resources

6. Portfolio Analysis (Detailed — Figure 5.5, the chi-square/quadrant chart)

- A method to decide **what to do with each application** in an organization's portfolio, based on two axes:
 - **X-axis: Business value**
 - **Y-axis: Technical quality**
- **Four quadrants:**

Perform portfolio analysis

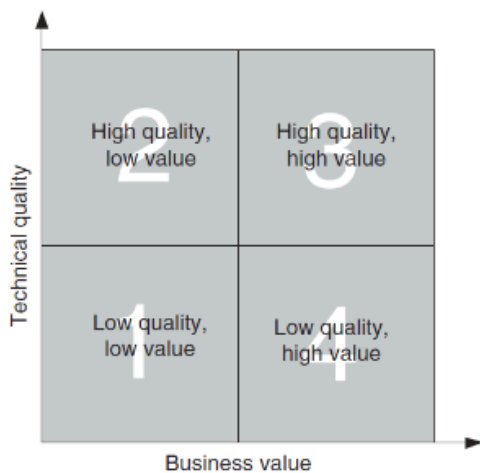


FIGURE 5.5 Portfolio analysis chi-square chart

- *Quadrant 1.* prime candidate for substitution with a commercial product if one is available.
- *Quadrant 2.* need not be replaced, migrated, or restructured.
- *Quadrant 3.* should be actively evolved.
- *Quadrant 4.* good candidate for redevelopment or migration to a new system.

Important nuance: Quadrant naming in the diagram is **not** in simple reading order – pay attention to which quadrant is which:

- Q1 = low quality, low value (bottom-left)
- Q2 = high quality, low value (top-left)
- Q3 = high quality, high value (top-right)
- Q4 = low quality, high value (bottom-right) This is the classic "business value vs technical quality" 2x2 matrix – conceptually similar to BCG matrix style reasoning, so if asked *why* a quadrant gets a certain treatment, reason via combination of (value, quality).

Criteria to Establish Business Value

- Level of **usage**
- Number of **business goals satisfied**
- **Contribution to profit**
- **User satisfaction**
- **Annual revenue**

Criteria for Technical Quality

- **Ease of making changes**
- **Frequency of release notes** (how often updates/releases happen)
- **Accuracy**

- **Performance** of the system
- **Error rate**
- **Cyclomatic complexity** (a code-complexity metric – ties back to earlier maintainability metrics topics)
- **Availability of training** (for staff to work with the system)

Scoring Formula (important – could appear as a numeric problem)

$$r = 1 - \left(\frac{\text{actual measure} - \text{lower_bound}}{\text{upper_bound} - \text{lower_bound}} \right)$$

- This normalizes a raw metric into a **coefficient between 0 and 1**.
- **Simple example from slides:** if max value of a criterion is 100, and an application scores 30 → coefficient = 30/100 = **0.30**.
 - (Note: this simple example is a direct ratio; the general formula above additionally supports non-zero lower bounds, i.e., normalizing between a lower_bound and upper_bound rather than assuming 0.)
- Each criterion gets scored this way, then likely combined (e.g., averaged or weighted) to get overall business value / technical quality scores – placing the application into one of the 4 quadrants.

Watch for a calculation question: "given actual measure, lower bound, upper bound, compute r." Just plug into the formula directly.

7. Quick Recap Table

Concept	Key Point
Legacy system	Resists change, mission-critical, vital but hard to maintain

6 LIS solutions	Freeze, Outsource, Carry on maintenance, Discard & redevelop, Wrap, Migrate
Migration = 3 steps	Schema conversion → Data conversion (ETL) → Program conversion
Migration planning	12 ordered steps (portfolio analysis is Step 1)
Portfolio analysis axes	Business value (x) vs Technical quality (y)
4 Quadrants	Q1 low/low → substitute; Q2 high-quality/low-value → leave; Q3 high/high → evolve; Q4 low-quality/high-value → redevelop/migrate
Coefficient formula	$r = 1 - ((\text{actual} - \text{lower}) / (\text{upper} - \text{lower}))$

8. Likely Conceptual Exam Questions to Prepare

1. Define a legacy system (give both informal and formal definitions).
2. List the acceptable features/characteristics of a legacy system.
3. Why do organizations want to get rid of legacy systems?
4. List and briefly explain the 6 solutions for legacy information systems.
5. Explain the 3 main steps of migration; explain ETL in the context of data conversion.
6. List the 12 steps of migration planning in order.
7. Explain portfolio analysis – draw/label the quadrant chart and explain the recommended action for each quadrant.
8. What criteria are used to establish business value vs technical quality?
9. Given sample values, compute the coefficient r using the formula.

10. Compare Wrap vs Migrate vs Discard & Redevelop as legacy solutions — trade-offs.

Software Maintenance Exam — Solved Answers

Question & Answer Set (Each question — 4 marks)

Question 1: Software Maintenance vs. Software Evolution

What is Software Maintenance? Discuss the differences between software evolution and software maintenance.

Definition

Software Maintenance is the process of modifying a software system or component after delivery to correct faults, improve performance or other attributes, or adapt the product to a changed environment. As defined by IEEE, it includes all activities performed to keep a software system operational, accurate, and up to date after deployment.

Categories of Software Maintenance (IEEE/ISO)

- Corrective — fixing discovered defects/bugs

- Adaptive – modifying the system to work in a new environment (OS, hardware, regulations)
- Perfective – enhancing performance, maintainability, or adding new features
- Preventive – modifying the system to prevent future problems (e.g., refactoring)

Software Evolution vs. Software Maintenance

Aspect	Software Evolution	Software Maintenance
Scope	Broad – long-term, continuous change and growth of a software system over its lifetime	Narrower – specific corrective, adaptive, or enhancement activities after deployment
Purpose	Adapting the system to new requirements, environments, and understanding over time	Fixing defects, improving performance, and keeping the system operational
Nature of change	Planned, strategic, often architecture-level change	Often reactive, tactical, code-level fixes/updates
Timeframe	Spans the entire life of the software, including major redesigns	Typically post-release, ongoing support activities
Theoretical basis	Governed by Lehman's Laws of Software Evolution	Governed by IEEE/ISO maintenance standards and categories (corrective, adaptive, perfective, preventive)

In short: software evolution is the broader, continuous process of a system growing and adapting over its entire life (guided by Lehman's Laws), while software maintenance refers to the specific set of activities – corrective, adaptive, perfective, and preventive – carried out to support and sustain the software after release. Maintenance can be seen as one of the mechanisms through which evolution happens.

Question 2: Software Rejuvenation

Explain the concept of software rejuvenation. Discuss the pros and cons of software rejuvenation.

Concept

Software Rejuvenation is a proactive fault-management technique in which a running software system is periodically stopped, its internal state is cleaned (e.g., clearing memory leaks, resetting internal data structures, garbage collection, restarting processes), and then it is restarted in a fresh, clean state. It is designed to counter "software aging" — the gradual degradation of a long-running system's performance and reliability due to phenomena like memory leaks, data corruption, and resource fragmentation — before the system actually fails.

Pros and Cons

Pros	Cons
Restores system to a healthy state without full replacement, extending useful life	Provides only temporary relief — does not fix root cause of aging (e.g., latent bugs, design decay)
Cheaper and faster than rewriting the system from scratch	Requires downtime or resource overhead for restarts/resets, which can affect availability
Reduces failures caused by "software aging" (e.g., memory leaks, resource exhaustion)	Can mask deeper architectural or design problems that eventually need re-engineering
Improves reliability, performance, and availability of long-running systems	Needs careful scheduling/prediction — poorly timed rejuvenation can itself cause disruption

Question 3: Staged Maintenance Models of FLOSS Systems

Briefly describe the staged maintenance models of FLOSS systems.

FLOSS (Free/Libre and Open Source Software) systems typically follow a staged lifecycle model — adapted from the classic staged model of software evolution — describing how an open-source project matures, stabilizes, and eventually declines or is rejuvenated based on community and contributor activity.

Stage	Description
1. Initiation / Initial Development	Project starts, core architecture and first working version are built, usually by a small founding group of contributors
2. Growth / Evolution	Community expands, features are rapidly added, contributions increase, and the codebase evolves to meet growing user needs
3. Maturity / Saturation	Growth slows down; the system stabilizes; focus shifts from adding new features to refining, optimizing, and fixing existing functionality
4. Decline / Stagnation	Contributor activity drops, fewer updates are released, and the project may lose relevance or community interest
5. Closedown / Death (or Rejuvenation)	The project is either abandoned/archived, or it is revived through rejuvenation, a fork, or a new maintaining team

This staged view helps maintainers and researchers understand where a FLOSS project stands in its lifecycle, predict its future contributor activity, and decide when interventions (like rejuvenation, re-engineering, or forking) are needed.

Question 4: Rule-Based Bug Detection vs. Automated Bug Repair

Explain the major limitations of rule-based bug detection approaches. Discuss how Automated Bug Repair techniques address these limitations.

Limitation of Rule-Based Bug Detection	How Automated Bug Repair (APR) Addresses It
Relies on a fixed, predefined set of rules/patterns — cannot detect novel or unseen bug types	Many APR techniques use learning-based or search-based methods (e.g., genetic programming, ML models) that generalize beyond fixed patterns

Limitation of Rule-Based Bug Detection	How Automated Bug Repair (APR) Addresses It
High false positive rate — flags correct code as buggy due to rigid pattern matching	APR systems validate candidate fixes against test suites, filtering out incorrect "fixes" and reducing noise
Cannot capture context-dependent or semantic bugs (e.g., logic errors specific to business rules)	Techniques like semantic-based or learning-based repair analyze program behavior and context, not just syntax
Rules must be manually written and maintained by experts — doesn't scale well to new languages/frameworks	Data-driven APR (e.g., using large code corpora or LLMs) can adapt automatically without manual rule updates
No repair capability — only detects/flags issues, leaving fixing to developers	APR goes a step further by automatically generating and applying patches, closing the loop from detection to repair

Summary: Rule-based bug detection is fast and interpretable but fundamentally limited by its reliance on fixed, hand-crafted patterns — it cannot generalize, produces false positives, and stops at detection. Automated Bug Repair (APR) techniques (search-based, semantic-based, and learning-based, including modern LLM-based repair) extend beyond detection by automatically generating, validating, and applying candidate patches, addressing novel and context-dependent bugs that static rules would miss.

Question 5: Aspect-Oriented Programming (AOP)

Define Aspect-Oriented Programming (AOP). Briefly explain how AOP handles cross-cutting concerns.

Definition

Aspect-Oriented Programming (AOP) is a programming paradigm that aims to increase modularity by allowing the separation of cross-cutting concerns — functionality that spans multiple modules/classes of a program (e.g., logging, security, transaction management) — into a single unit called an aspect, rather than scattering that code throughout the codebase.

Key AOP Concepts

- Aspect — a module that encapsulates a cross-cutting concern (e.g., a Logging aspect)
- Join Point — a specific point in program execution (e.g., a method call) where an aspect can be applied
- Pointcut — an expression that selects a set of join points where advice should run
- Advice — the actual code that runs at a matched join point (before, after, or around it)
- Weaving — the process of combining aspects with the main business logic code, done at compile-time, load-time, or runtime

How AOP Handles Cross-Cutting Concerns

Without AOP, concerns like logging, security checks, or transaction handling must be manually duplicated inside every relevant method/class, leading to code tangling (mixing business logic with infrastructure logic) and code scattering (the same concern repeated across many files). AOP solves this by letting developers define such concerns once, as an aspect, and specify pointcuts describing where they should apply. During weaving, the AOP framework automatically inserts ("weaves") this behavior into the relevant join points throughout the codebase — without modifying the original business logic classes directly. This keeps core modules clean and focused on their primary responsibility, while cross-cutting behavior stays centralized, reusable, and easier to maintain.

Software Evolution & Reengineering Exam — Solved Answers

All Questions Solved (1–4, all parts)

Question 1(a): Lehman's Laws — Applicability & Modifications

Lehman's laws of software evolution describe patterns in the growth and change of large, proprietary systems. Considering open-source development and agile methodologies, critically analyze their applicability today and propose modifications.

Lehman's Laws of Software Evolution

Law	Short Description
1. Continuing Change	A system must be continually adapted or it becomes progressively less useful
2. Increasing Complexity	As a system evolves, its complexity increases unless work is done to reduce it
3. Self-Regulation	Global system evolution is self-regulating, with growth trends staying fairly stable over time
4. Conservation of Organizational Stability	The average effort per release stays roughly constant over the system's lifetime
5. Conservation of Familiarity	The amount of change per release stays roughly constant so developers/users can keep understanding the system
6. Continuing Growth	Functionality must continually increase to maintain user satisfaction
7. Declining Quality	Quality declines over time unless the system is rigorously maintained and adapted
8. Feedback System	Evolution processes involve multi-loop, multi-agent feedback and must be treated as such to achieve improvement

Critical Analysis: Applicability to Open-Source & Agile

Laws that still largely hold:

- Continuing Change and Continuing Growth remain true — both OSS and agile systems are continuously updated to stay relevant and meet growing user demand.
- Increasing Complexity still applies — even in agile/OSS projects, code entropy increases without deliberate refactoring effort.

- Declining Quality still applies in principle, though agile's continuous testing/CI pipelines slow the decline compared to traditional waterfall systems.

Laws that need reconsideration:

- Self-Regulation and Conservation of Organizational Stability assumed a single, centrally managed organization with fairly constant staffing. In OSS, contributor numbers fluctuate wildly (bursts of activity, contributor churn), so growth is far less "self-regulating" and more volatile.
- Conservation of Familiarity is challenged by agile's short iterative releases — teams intentionally make larger, faster changes per sprint using automated tests as a safety net instead of relying on developers' deep familiarity with the whole codebase.
- The Feedback System law is amplified rather than diminished in OSS/agile — feedback loops are faster and more numerous (CI/CD, code review, issue trackers, user telemetry), so the underlying idea holds but operates on a much shorter cycle time.

Proposed Modifications for Present-Day Software Realities

- Redefine "release" in terms of continuous delivery (CI/CD) instead of discrete versioned releases, since many modern systems deploy multiple times per day.
- Incorporate community/contributor dynamics explicitly for OSS — e.g., a law addressing bursty, decentralized contribution patterns rather than assuming stable organizational effort.
- Account for automated quality safety nets (tests, static analysis, CI) that allow larger, faster changes without the traditional quality decline Lehman assumed.
- Add a law reflecting cloud/microservice architectures, where evolution happens at the service level independently rather than as one monolithic system.

Question 1(b): Software Maintenance vs. Software Reengineering

Differentiate between software maintenance and software reengineering.

Aspect	Software Maintenance	Software Reengineering
Goal	Keep the existing system functional, fix defects, and make small enhancements	Restructure/rebuild the system to improve quality while preserving functionality
Scope of change	Small, incremental, localized changes	Large-scale, systematic transformation of design/architecture
Approach	Reactive — driven by reported bugs or new requirements	Proactive — driven by the need to reduce technical debt or modernize
Output	Same system, patched/updated	A restructured or re-architected version of the system (same behavior, better internals)
Typical activities	Corrective, adaptive, perfective, preventive maintenance	Reverse engineering, restructuring, forward engineering
Risk & cost	Lower cost per change, but costs accumulate over time (technical debt)	Higher upfront cost/risk, but reduces long-term maintenance burden

Question 2(a): Legacy Information System

What is a Legacy Information System? Describe the characteristics of a Legacy Information System.

Definition

A Legacy Information System is a software system that continues to be used because it fulfills a critical business need, even though it was built using outdated technology, languages, platforms, or design practices, and is typically difficult and costly to maintain, extend, or replace.

Characteristics of a Legacy Information System

- Built with outdated technology, programming languages, or hardware/software platforms
- Poor or missing documentation, making the system hard to understand
- Original developers are often no longer available, resulting in lost domain/design knowledge
- Highly business-critical — the organization depends heavily on it despite its age
- Rigid, tightly-coupled, monolithic architecture that resists change
- High cost and risk associated with maintenance, modification, or replacement
- Difficult to integrate with modern systems and technologies
- Accumulated technical debt from years of ad-hoc patches and fixes

Question 2(b): Solutions to the Legacy Problem

Discuss different types of solutions to the legacy problem.

Solution Type	Description
Freeze / Discard	Stop maintaining the legacy system; freeze it as-is or discard it entirely if it's no longer valuable
Wrap / Encapsulate	Wrap the legacy system with a new interface (e.g., API/middleware) so it can interoperate with modern systems without modifying internals

Migrate	Move the legacy system to a new platform/environment while largely preserving its functionality and structure
Re-engineer (Restructure)	Analyze and transform the internal structure/architecture of the legacy system to improve maintainability, keeping behavior the same
Rewrite / Redevelop	Completely rebuild the system from scratch using modern technologies, replicating or improving on required functionality
Replace (Buy/COTS)	Replace the legacy system entirely with a new or commercial off-the-shelf (COTS) product

Question 2(c): Technical Quality Coefficient Calculation

Calculate coefficient ratio for the following technical quality criteria.

Formula used: Coefficient = (Maximum – Actual) / (Maximum – Minimum). A coefficient closer to 1 indicates better quality (actual value close to the best/minimum extreme); a coefficient closer to 0 indicates poorer quality (actual value close to the worst/maximum extreme).

Criterion	Actual	Max	Min	Coefficient = (Max – Actual) / (Max – Min)
(i) Error rate (per 10k LOC)	50	60	20	$(60 - 50) / (60 - 20) = 10 / 40 = 0.25$
(ii) Cyclomatic complexity (per procedure)	30	40	15	$(40 - 30) / (40 - 15) = 10 / 25 = 0.40$

(i) **Error rate coefficient** = 0.25 → since the actual error rate (50) is closer to the maximum (worst) value of 60, the coefficient is low, indicating relatively poor quality on this criterion.

(ii) **Cyclomatic complexity coefficient** = 0.40 → the actual complexity (30) is somewhat closer to the midpoint between best and worst, indicating moderate quality on this criterion.

Question 3(a): Impact Analysis

A software change in a large system can have unintended consequences. Describe in detail the process of impact analysis and discuss its limitations. Provide scenarios where (i) traceability analysis alone fails, (ii) dependency analysis alone fails. Explain how a combination of these approaches can mitigate those limitations.

Process of Impact Analysis

Impact analysis is the process of identifying the potential consequences (direct and indirect) of a proposed software change before it is implemented, so that the scope, cost, and risk of the change can be properly estimated.

- Step 1 — Change Identification: Clearly define what is being changed (a requirement, a module, an interface, etc.)
- Step 2 — Traceability Analysis: Use traceability links (requirements → design → code → tests) to identify all artifacts formally connected to the changed element
- Step 3 — Dependency Analysis: Examine code-level dependencies (call graphs, data flow, control flow) to find components that structurally depend on the changed element
- Step 4 — Ripple Effect Analysis: Trace how the change propagates through the dependent components to identify secondary/indirect effects
- Step 5 — Impact Estimation & Reporting: Compile the affected set of artifacts/components, estimate the cost/effort/risk, and report to stakeholders for decision-making

Limitations of Impact Analysis

- Can be incomplete if traceability links are outdated, missing, or inconsistently maintained
- Static analysis may miss dynamic/runtime dependencies (e.g., reflection, dynamic dispatch, configuration-driven behavior)
- Large systems produce a huge dependency graph, making manual analysis time-consuming and error-prone
- Cannot fully predict emergent behavior or interactions that only manifest at runtime under specific conditions

Scenario (i): Traceability Analysis Alone Fails

Traceability analysis relies on documented links between requirements, design, and code. If a developer directly modifies code to fix an urgent production bug without updating requirement/design documentation (a common real-world occurrence), the traceability links become stale. A later change to the "documented" requirement will fail to reveal the undocumented code dependency, so the impacted component is missed entirely — even though it is functionally affected.

Scenario (ii): Dependency Analysis Alone Fails

Dependency analysis examines structural/code-level relationships (e.g., function calls, variable usage). However, it cannot capture business or requirement-level relationships that aren't reflected in code structure. For example, two modules may have no direct code dependency but are linked by a shared business rule (e.g., a tax calculation rule used independently in a billing module and a reporting module, implemented as separate, non-referencing code). A code-only dependency graph would miss this business-level connection, so a change to the tax rule in one module would silently produce inconsistent results in the other.

Combining Both Approaches

Using traceability analysis and dependency analysis together compensates for each other's blind spots: traceability analysis captures requirement/design-level, documented business relationships, while dependency analysis captures actual structural/code-level relationships that may not be documented at all.

Cross-referencing both result sets produces a more complete impact set — any component flagged by either method is included, significantly reducing the chance of missed impacts — and discrepancies between the two (e.g., a traced link with no code dependency, or vice versa) can flag areas needing documentation updates or further manual investigation.

Question 3(b): Rewrite vs. Rework vs. Replace

Compare rewrite, rework, and replace using cost-risk models.

Strategy	Cost	Risk	When It's the Right Choice	Trade-off Summary
Rework (in-place repair/refactor)	Low–Medium (incremental spend)	Low — existing behavior/tests already validated	System is mostly sound but has localized quality issues	Cheapest, safest, but limited improvement ceiling — technical debt can persist
Rewrite (rebuild from scratch)	Very High (full redevelopment effort)	High — new bugs, requirement gaps, long delivery time	System is fundamentally unfit (wrong architecture /tech stack) and long-term ROI justifies	Highest potential quality gain, but highest cost and schedule/business risk

			the investment	
Replace (buy COTS/SaaS)	Medium (licensing/subscription + integration cost)	Medium – vendor lock-in, integration/compatibility risk	A suitable off-the-shelf product exists that meets business needs	Fast and often lower risk than rewrite, but less customizable and creates dependency on vendor

Question 4(a): Program Comprehension

What is meant by program comprehension? Why is it important to understand programs?

Definition

Program comprehension is the cognitive process by which a developer builds a mental model of what a piece of software does, how it is structured, and why it behaves the way it does, by reading and analyzing its source code, documentation, and behavior.

Why It's Important

- Maintenance and debugging cannot happen safely without first understanding the existing code – most maintenance time (often over 50%) is spent on comprehension rather than actual coding
- Enables accurate impact analysis – you cannot predict the consequences of a change without understanding how components relate
- Reduces the risk of introducing new defects when modifying unfamiliar code

- Supports knowledge transfer, especially when original developers have left the project
- Essential for code review, refactoring, and re-engineering decisions

Image Processing

Image Processing(incomplete)

Pattern (Basic Concept)

-A **pattern** is a set of features or attributes that describe an object and allow it to be identified or classified.

Represented using **feature vectors**

Can be:

- Physical (e.g., color, shape)
- Mathematical (data patterns)

Example:

- Face → features like eyes, nose, distance
- Speech → frequency, pitch

Pattern Recognition is the process by which a machine identifies patterns in data using algorithms and classifies them based on learned knowledge.

Core idea: **Classification + Learning**

Uses **machine learning**

Converts raw data → meaningful classes

Approaches of Pattern Recognition

1. Statistical Approach 2. Structural Approach 3. Neural Approach

Image (Basic Concept)

-An image is a **2D representation of visual data** made up of pixels.

Pixel

- Smallest unit of an image
- Stores intensity/color value

Digital Image Processing is the use of computer algorithms to process images for enhancement or extracting information.

Levels

1. Low-level

- Input & output = images
- Example: noise removal

2. Mid-level

- Output = features
- Example: edge detection

3. High-level

- Output = understanding
- Example: object recognition

Types of Images

1. Binary Image

- Only **0 and 1**
- Black & white

2. Black & White Image

- Only two colors (no gray)

3. Grayscale Image (8-bit)

- 256 shades (0–255)
- 0 = black, 255 = white

4. Color Image (16-bit / RGB)

- Uses **Red, Green, Blue** channels
- Thousands of colors

Fundamental Steps of Image Processing

- Image acquisition
- Preprocessing
- Segmentation
- Feature extraction
- Recognition / interpretation

What is Digital Image Processing?

Digital Image

— a two-dimensional function $f(x, y)$
x and y are spatial coordinates

The amplitude of f is called **intensity** or **gray level** at the point (x, y)

Digital Image Processing

— process digital images by means of a computer, it covers low-, mid-, and high-level processes

low-level: inputs and outputs are images

mid-level: outputs are attributes extracted from input images

high-level: an ensemble of recognition of individual objects

Pixel

— the elements of a digital image

Sources of image:

- Electromagnetic (EM) energy spectrum
- Acoustic
- Ultrasonic
- Electronic
- Synthetic images produced by computer

Major uses

Gamma-ray imaging: nuclear medicine and astronomical observations

X-rays: medical diagnostics, industry, and astronomy, etc.

Ultraviolet: lithography, industrial inspection, microscopy, lasers, biological imaging, and astronomical observations

Visible and infrared bands: light microscopy, astronomy, remote sensing, industry, and law enforcement

Microwave band: radar

Radio band: medicine (such as MRI) and astronomy

Chromatic light bands: 0.43 to 0.79 μm

The quality of a chromatic light source:

Radiance: total amount of energy

Luminance (Im): the amount of energy an observer perceives from a light source

Brightness: a subjective descriptor of light perception that is impossible to measure. It embodies the achromatic notion of intensity and one of the key factors in describing color sensation.

Illumination

Lumen — A unit of light flow or luminous flux

Lumen per square meter (lm/m^2) — The metric unit of measure for illuminance of a surface

IMAGE ACQUISITION (Digital Image Processing)

1. Definition

Image acquisition is the **first step in digital image processing**, where a real-world scene is captured and converted into a digital image.

It involves:

1. **Sensing the scene (capturing light/energy)**
2. **Converting it into an electrical signal**
3. **Digitizing the signal into a computer-readable image**

Image acquisition = Physical scene → Sensor → Electrical signal → Digital image

This process converts continuous information (real world) into a discrete digital format.

Key Processes in Image Acquisition

1. Sampling (Spatial Discretization)

- Divides image into a grid of pixels
- Determines resolution

Example:

More pixels → Higher detail

2. Quantization (Intensity Discretization)

- Converts continuous intensity into discrete levels
- Determines gray levels (brightness)

Example:

- 8-bit image → 256 intensity levels

Output of Image Acquisition

A digital image is represented as: $f(x,y)$

Where:

- x,y = spatial coordinates
- $f(x,y)$ = intensity (gray level)

1. Resolution

- Number of pixels in an image
- Higher resolution → better detail

2. Gray Levels

- Number of intensity values
- Higher levels → smoother image

3. Noise (Important)

- Unwanted disturbance during acquisition
- Comes from: Sensors, Environment

Image Interpolation

Interpolation — Process of using known data to estimate unknown values

e.g., zooming, shrinking, rotating, and geometric correction

Interpolation (sometimes called *resampling*) — an imaging method to increase (or decrease) the number of pixels in a digital image.

Some digital cameras use interpolation to produce a larger image than the sensor captured or to create digital zoom

BASIC RELATIONSHIPS BETWEEN PIXELS

1. Neighborhood of a Pixel

A pixel at coordinates (x,y) has neighboring pixels that define spatial relationships.

Types:

- **4-Neighbors $N_4(p)$:**
 $(x-1,y), (x+1,y), (x,y-1), (x,y+1)$
Only horizontal and vertical neighbors.
- **Diagonal Neighbors $N_D(p)$:**
 $(x-1,y-1), (x+1,y+1), (x+1,y-1), (x-1,y+1)$
- **8-Neighbors $N_8(p)$:**
 $N_8(p) = N_4(p) \cup N_D(p)$

Defines how pixels interact in image processing such as segmentation and filtering.

2. Adjacency

Adjacency defines whether two pixels are connected based on intensity values.
Let V be the set of allowed intensity values.

Adjacency

Let V be the set of intensity values used to define adjacency

4-adjacency: Two pixels p and q with values from V are 4-adjacent if q is in the set $N_4(p)$.

8-adjacency: Two pixels p and q with values from V are 8-adjacent if q is in the set $N_8(p)$.

m-adjacency: Two pixels p and q with values from V are m-adjacent if

(i) q is in the set $N_4(p)$, or

(ii) q is in the set $\mathbf{N_D(p)}$ and the set $N_4(p) \cap N_4(q)$ has no pixels whose values are from V .

3. Path

A (digital) path (or curve) from pixel p with coordinates (x_0, y_0) to pixel q with coordinates (x_n, y_n) is a sequence of distinct pixels with coordinates

$$(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$$

Where (x_i, y_i) and (x_{i-1}, y_{i-1}) are adjacent for $1 \leq i \leq n$.

Here n is the *length* of the path.

If $(x_0, y_0) = (x_n, y_n)$, the path is **closed** path.

We can define 4-, 8-, and m-paths based on the type of adjacency used.

4. Connectivity

Two pixels are connected if a path exists between them within a set SSS.

Concept: Forms the basis for region detection and segmentation.

5. Region and Boundary

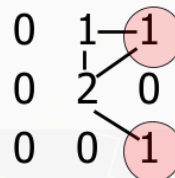
- **Region:** A set of connected pixels.
- **Boundary:** Pixels in region R that have at least one neighbor outside R .

Concept: Used in edge detection and object extraction.

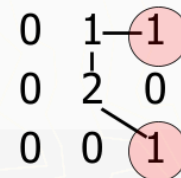
Examples: Adjacency and Path

$$V = \{1, 2\}$$

0 _{1,1}	1 _{1,2}	1 _{1,3}
0 _{2,1}	2 _{2,2}	0 _{2,3}
0 _{3,1}	0 _{3,2}	1 _{3,3}



8-adjacent



m-adjacent

The 8-path from $(1,3)$ to $(3,3)$:

- (i) $(1,3), (1,2), (2,2), (3,3)$
- (ii) $(1,3), (2,2), (3,3)$

The m-path from $(1,3)$ to $(3,3)$:

- $(1,3), (1,2), (2,2), (3,3)$

Boundary (or border)

The boundary of the region R is the set of pixels in the region that have one or more neighbors that are not in R.

If R happens to be an entire image, then its boundary is defined as the set of pixels in the first and last rows and columns of the image.

Distance Measures

The following are the different Distance measures:

a. Euclidean Distance :

$$D_e(p, q) = [(x-s)^2 + (y-t)^2]^{1/2}$$

b. City Block Distance:

$$D_4(p, q) = |x-s| + |y-t|$$

c. Chess Board Distance:

$$D_8(p, q) = \max(|x-s|, |y-t|)$$

		2		
	2	1	2	
2	1	0	1	2
	2	1	2	
		2		

2	2	2	2	2
2	1	1	1	2
2	1	0	1	2
2	1	1	1	2
2	2	2	2	2

MATHEMATICAL OPERATIONS IN DIGITAL IMAGE PROCESSING

1. Array vs Matrix Operations

- **Array operations:** Element-wise operations
- **Matrix operations:** Follow linear algebra rules

2. Linear and Nonlinear Operations

A transformation H is linear if:

- Additivity: $H(f+g)=H(f)+H(g)$

- Homogeneity: $H(af) = aH(f)H(af)$

If these are not satisfied, the operation is nonlinear.

ARITHMETIC OPERATIONS ON IMAGES

$$s(x,y) = f(x,y) + g(x,y)$$

$$d(x,y) = f(x,y) - g(x,y)$$

$$p(x,y) = f(x,y) \times g(x,y)$$

$$v(x,y) = f(x,y) \div g(x,y)$$

Applications

Noise Reduction

- Corrupted image:
- Noise is reduced by averaging multiple noisy images.

Used in low-light imaging such as astronomy.

Image Subtraction

$$g(x,y) = f(x,y) - h(x,y)$$

Application: Medical imaging (e.g., radiography).

An Example of Image Subtraction: Mask Mode Radiography

Mask $h(x,y)$: an X-ray image of a region of a patient's body

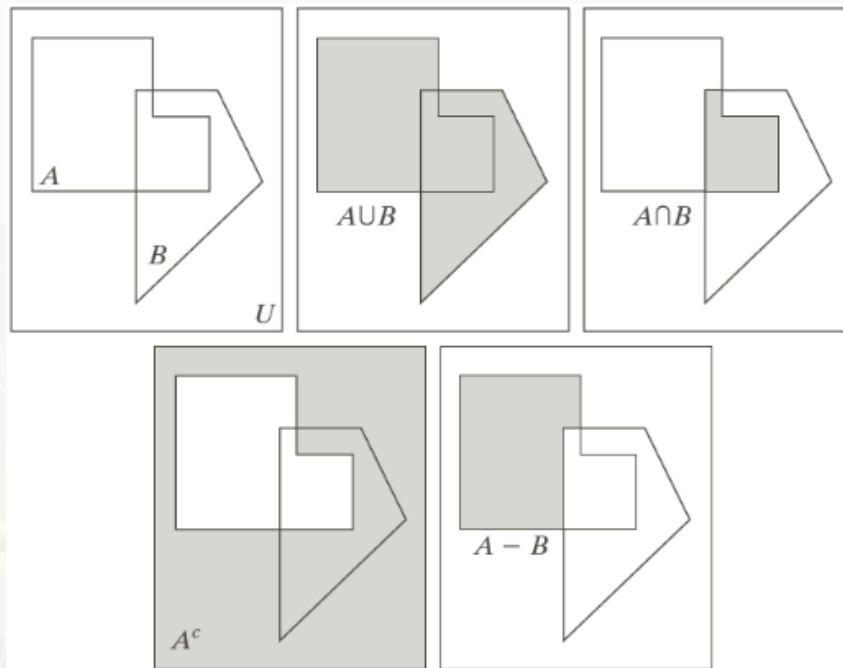
Live images $f(x,y)$: X-ray images captured at TV rates after injection of the contrast medium

Enhanced detail $g(x,y)$

$$g(x,y) = f(x,y) - h(x,y)$$

The procedure gives a movie showing how the contrast medium propagates through the various arteries in the area being observed.

Set and Logical Operations



a	b	c
d	e	

FIGURE 2.31

(a) Two sets of coordinates, A and B , in 2-D space. (b) The union of A and B . (c) The intersection of A and B . (d) The complement of A . (e) The difference between A and B . In (b)–(e) the shaded areas represent the member of the set operation indicated.

Set and Logical Operations



FIGURE 2.33

Illustration of logical operations involving foreground (white) pixels. Black represents binary 0s and white binary 1s. The dashed lines are shown for reference only. They are not part of the result.

SPATIAL OPERATIONS

Single-Pixel Operations

Modify pixel intensity independently.

Example: brightness adjustment.

GEOMETRIC TRANSFORMATIONS

Definition

Spatial transformation of coordinates with intensity interpolation.

Example: rotation, scaling.

Affine Transformation

Preserves:

- Straight lines
- Parallelism

INTENSITY ASSIGNMENT

Forward Mapping-Maps input pixels to output coordinates.

Issue: multiple pixels may map to same location.

Inverse Mapping-Maps output pixels back to input image.

Preferred because it avoids gaps.

Image Registration

Input and output images are available but the transformation function is unknown.

Goal: estimate the transformation function and use it to register the two images.

One of the principal approaches for image registration is to use *tie points* (also called *control points*)

The corresponding points are known precisely in the input and output (**reference**) images.

IMAGE TRANSFORM

- Represent image in another domain (e.g., frequency domain)
- Original image can be recovered using inverse transform

Intensity Transformation & Spatial Filtering

— Exam Notes

1. Spatial Domain vs Transform Domain

- **Spatial domain:** operate directly on pixel intensity values of the image plane.
General form: $g(x, y) = T[f(x, y)]$
 - $f(x, y)$ = input image
 - $g(x, y)$ = output image
 - T = operator on f , defined over a **neighborhood** of point (x, y) (e.g. a 3×3 window centered at (x, y))
 - If the neighborhood is 1×1, T becomes a simple **intensity transformation function**: $s = T(r)$, where r is input intensity, s is output intensity.
- **Transform domain:** process transform coefficients (e.g. Fourier coefficients), not the raw pixel intensities directly.

Conceptual question to expect: "Why does neighborhood size matter?" — A 1×1 neighborhood gives point processing (only depends on that pixel's own value). Larger neighborhoods enable spatial filtering (smoothing, sharpening) because output depends on surrounding pixels too.

2. Basic Intensity Transformation Functions

All defined for intensity range $[0, L-1]$ (e.g., $L=256$ for 8-bit images).

2.1 Image Negative

$$s = L - 1 - r$$

- Reverses intensity: dark $\leftrightarrow$ light.
- Useful for enhancing white/gray detail embedded in large dark regions (e.g., medical images — mammograms, since a lesion in a dark background becomes clearer against inverted background).

Worked example: 8-bit image ($L=256$), pixel $r = 60$. $s = 255 - 60 = 195$

2.2 Log Transformation

$$s = c \cdot \log(1 + r)$$

- c is a scaling constant, typically $c = (L-1)/\log(1+R)$ where R = max intensity in image, so output stays in range.
- **Effect:** Expands dark pixel values (low r), compresses bright/high pixel values.
- Used to display **Fourier spectra** (huge dynamic range – a few very bright points, rest dark) so both faint and dominant components become visible.
- The **inverse log** (exponential) transform does the opposite – expands high values, compresses low.

Worked example: $c=1$, $r=1000$ (typical Fourier magnitude): $s = \log(1001) \approx 6.9$ – massively compressed vs raw linear display.

2.3 Power-Law (Gamma) Transformation

$$s = c \cdot r^\gamma$$

- c, γ positive constants.
- $\gamma < 1$ (e.g. 0.4, 0.67): maps a narrow range of dark input values into a wider range of output → expands dark regions, similar look to log transform. Used to correct **displays with $\gamma > 1$** (CRT monitors have $\gamma \approx 1.8-2.5$, so images are pre-corrected/gamma-corrected with $\gamma < 1$, $\sim 1/2.5$, before display).
- $\gamma > 1$ (e.g. 3, 4, 5): expands bright values, compresses dark values – darkens the image overall. Used when an image is washed out (too much dynamic range in bright areas), e.g. aerial/satellite images.
- $\gamma = 1$: identity transform (no change) when $c=1$.
- This is also called **gamma correction**.

Worked example: $c=1$, $r=0.5$ (normalized 0–1 scale).

- $\gamma=0.4$: $s = 0.5^{0.4} = 0.758$ (brightened)
- $\gamma=2.5$: $s = 0.5^{2.5} = 0.177$ (darkened)

Key exam distinction – Log vs Gamma:

	Log	Power-law $\gamma < 1$
Shape	Fixed, always compresses high values	Family of curves, tunable via γ
Use	Fixed dynamic-range compression (e.g. Fourier spectra)	Tunable contrast/brightness (display gamma correction)

3. Piecewise-Linear Transformations

Unlike the above (defined by one simple formula), these are built from **multiple linear segments** – more complex, so implementation requires specifying breakpoints (not just one equation).

3.1 Contrast Stretching

- **Definition of contrast** = difference between max and min pixel intensity.
- **Goal:** expand the range of intensity levels in a low-contrast image to span the full display range.
- Typical transformation: pick two points (r_1, s_1) and (r_2, s_2) on the r - s plane; connecting $(0, 0) \rightarrow (r_1, s_1) \rightarrow (r_2, s_2) \rightarrow (L-1, L-1)$ with straight lines defines $T(r)$.
- **Thresholding** is the limiting/extreme case: $r_1 = r_2 = k$ (a single threshold value), $s_1 = 0, s_2 = L-1 \rightarrow$ produces a **binary image** (step function).

3.2 Intensity-Level Slicing

- **Goal:** highlight a specific range of intensities $[A, B]$ of interest (e.g., highlighting blood vessels containing a contrast medium in an X-ray/angiogram).
- Two common implementations:
 1. Display chosen range $[A, B]$ as one high value, everything else as one low value (loses background detail).
 2. Display chosen range $[A, B]$ as high value, but **preserve all other original intensities** (keeps background context).
- Application example: highlighting major blood vessels to study flow of contrast medium and detect blockages; measuring flow over a time series of images.

3.3 Bit-Plane Slicing

- An 8-bit image can be decomposed into **8 binary bit-planes** (plane 1 = least significant bit / LSB, plane 8 = most significant bit / MSB).
- **Key fact:** Higher-order bit planes (especially plane 8, the MSB) contain the majority of visually significant data; lower-order planes contribute subtle detail and noise.
- **Useful for image compression** (can discard/quantize lower bit planes with less visual loss) and for **watermarking/steganography** (hide data in LSBs).
- Reconstructing from top planes only (e.g. planes 8+7, or 8+7+6, or 8+7+6+5) progressively approximates the original image – the more MSB planes included, the closer to the original.

Conceptual question: "Which bit-plane should you keep if you can only keep one, to best preserve the image?" → Plane 8 (MSB) – since it carries the coarse intensity structure.

4. Histogram Processing

4.1 Histogram Basics

$$h(r_k) = n_k$$

- r_k = k-th intensity value, n_k = number of pixels in image with intensity r_k .

Normalized histogram:

$$p(r_k) = n_k / (MN)$$

- $M \times N$ = total number of pixels. $p(r_k)$ is an estimate of the probability of occurrence of intensity r_k . Sum of all $p(r_k) = 1$.

Reading histograms (conceptual):

- Dark image → histogram concentrated at low intensity (left side).
- Light image → histogram concentrated at high intensity (right side).
- Low contrast image → histogram narrow, clustered in a small range.
- High contrast image → histogram spread widely across the full range.

4.2 Histogram Equalization

Idea: find transformation $T(r)$ that converts an arbitrary distribution $pr(r)$ into a **uniform** distribution $ps(s)$ – spreads out the most frequent intensity values, improving contrast globally.

Continuous form:

$$s = T(r) = (L-1) \int_0^r pr(w) dw$$

Result: $ps(s)$ is uniform = $1/(L-1)$ over $[0, L-1]$, regardless of the shape of $pr(r)$. (This is the CDF transform / probability integral transform.)

Discrete form (what you actually compute):

$$sk = T(rk) = (L-1) \sum_{j=0}^k pr(rj) = (L-1)/(MN) \sum_{j=0}^k nj \quad \text{for } k = 0, 1, \dots, L-1$$

- i.e., $sk = (L-1) \times$ **cumulative normalized histogram** up to level k .
- After computing sk , **round to nearest integer** to get valid discrete intensity levels.

Steps to solve any histogram equalization problem:

1. Build table of $rk, nk, pr(rk) = nk/MN$.
2. Compute cumulative sum of pr up to each k .
3. Multiply cumulative sum by $(L-1)$.
4. Round to nearest integer $\rightarrow$ this is sk , the new (equalized) intensity for old intensity rk .
5. (Optional) Merge levels that round to the same sk , sum their nk to get new histogram ns , then $ps(sk) = ns/MN$.

Fully worked example (3-bit image, $L=8$, size 64×64 , $MN=4096$):

rk	nk	$pr(rk)=nk/MN$	cumulative pr	$sk=7 \times \text{cumulative}$ e	rounded sk
0	790	0.19	0.19	1.33	1
1	1023	0.25	0.44	3.08	3

2	850	0.21	0.65	4.55	5
3	656	0.16	0.81	5.67	6
4	329	0.08	0.89	6.23	6
5	245	0.06	0.95	6.65	7
6	122	0.03	0.98	6.86	7
7	81	0.02	1.00	7.00	7

New (equalized) histogram – merge rk's mapping to same sk:

- $s_0=1$: $n_k=790$
- $s_1=3$: $n_k=1023$
- $s_2=5$: $n_k=850$
- $s_3=6$: $n_k=656+329=985$
- $s_4=7$: $n_k=245+122+81=448$

So final levels used are only $\{1,3,5,6,7\}$ – note histogram equalization of a discrete image does **not** produce a perfectly flat histogram (that's only exact in the continuous case); it produces the **best achievable approximation** given quantization.

Is histogram equalization always good? → NO.

- It can **over-enhance noise** in relatively flat/low-contrast regions (amplifies small differences into big contrast jumps).
- It's a **global**, automatic, non-interactive method – you can't target a specific region or desired look.
- It can wash out or distort images that already had a reasonable/desired histogram shape.

4.3 Histogram Matching (Histogram Specification)

Idea: instead of forcing a uniform histogram, generate an output image whose histogram matches a **specified** target PDF $p_z(z)$ (useful when you want a particular look, not just "flat").

Method:

1. Equalize the input: $s = T(r) = (L-1) \int_{[0,r]} p_r(w) dw$

2. Define desired output variable z via its own equalization function: $G(z) = \frac{1}{(L-1)} \int_{[0,z]} p_z(t) dt = s$
3. Then $z = G^{-1}(s) = G^{-1}[T(r)]$ – this is the mapping applied to each pixel.
 - In discrete practice: compute equalized s values from input histogram, compute the equalized mapping $G(z)$ from the *target* histogram, then for each s find the z whose $G(z)$ is closest to s (invert G).

Exam tip: Histogram equalization is a **special case** of histogram matching, where the specified target PDF is the uniform distribution.

4.4 Local Histogram Processing

- Global histogram equalization applies ONE transformation to the entire image – details in small local regions that don't significantly affect the global histogram may not be enhanced.
 - **Local processing:** slide a neighborhood (e.g. 3×3) across the image; at each pixel location, compute the histogram of pixels **within that neighborhood only**, derive the local equalization (or specification) transformation, and use it to map only the **center pixel's** intensity. Move to the next pixel, repeat.
 - Reveals small details embedded within regions that are nearly uniform in the global view (e.g., small squares with faint embedded shapes invisible after global equalization but visible after local equalization).
 - Much more computationally expensive than global equalization (histogram recomputed per pixel).
-

5. Spatial Filtering Fundamentals

A **spatial filter** = (a) a neighborhood (filter mask/kernel/template/window), + (b) a predefined operation. **Uses a mask to modify pixels based on neighborhood**

- Filtering produces a new pixel value placed at the coordinates of the **center** of the neighborhood.

5.1 Linear Spatial Filtering (general formula)

For filter w of size $m \times n$ (with $m=2a+1$, $n=2b+1$, i.e. odd dimensions so there's a well-defined center) applied to image f of size $M \times N$:

$$g(x,y) = \sum_{s=-a \text{ to } a} \sum_{t=-b \text{ to } b} w(s,t) f(x+s, y+t)$$

This is technically **correlation**.

5.2 Correlation vs Convolution

- **Correlation:** $w(x,y) \star f(x,y) = \sum_s \sum_t w(s,t) f(x+s, y+t)$ — slide the filter over the image, no flipping.
- **Convolution:** $w(x,y) \star f(x,y) = \sum_s \sum_t w(s,t) f(x-s, y-t)$ — filter is **rotated 180°** (flipped both horizontally and vertically) before sliding.
- **Key relationship:** if the filter mask w is **symmetric** (as most smoothing/sharpening masks are — e.g. averaging masks, Laplacian, Gaussian), correlation and convolution give the **identical result**, since flipping a symmetric mask doesn't change it.
- **Exam trap:** correlation with a unit impulse just copies the filter (not flipped) into the image at the impulse location; convolution with a unit impulse copies the **flipped (rotated 180°)** filter into the image.

Worked example: filter $w = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ applied to an image that is all zero except a single 1 at the position aligned with the filter's center.

Correlation result (at that neighborhood) = the filter itself, unflipped:

1 2 3 4 5 6 7 8 9

Convolution result = filter rotated 180°:

9 8 7 6 5 4 3 2 1

6. Smoothing Spatial Filters

Used for **blurring** and **noise reduction**; blurring is also used to remove small details / bridge small gaps in lines. Two categories: **linear** (averaging) and **nonlinear** (order-statistic).

6.1 Linear Smoothing (Averaging) Filters

General weighted average formula:

$$g(x,y) = \left[\sum_s \sum_t w(s,t) f(x+s,y+t) \right] / \left[\sum_s \sum_t w(s,t) \right]$$

- Dividing by the sum of coefficients normalizes output to stay in the valid intensity range (this is what makes it an "average").

Two classic 3×3 masks:

Box/standard average: all coefficients = 1, multiplier = 1/9.

$$1/9 \times \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

- Treats all 9 pixels equally.

Weighted/Gaussian-like average: center-weighted, multiplier = 1/16.

$$1/16 \times \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

- Reduces blurring compared to box filter while still smoothing, because closer pixels contribute more.

Effect of increasing mask size (m): larger mask → more blurring, more noise reduction, but more loss of fine detail/edges. This trade-off (noise reduction vs detail preservation) is a classic exam question.

Application — "gross representation of objects": heavy averaging (e.g. 15×15 mask) blurs small bright objects (e.g. stars/galaxies in astronomical images) into blobs, then thresholding isolates them as regions — useful for detecting/counting objects rather than fine analysis.

6.2 Order-Statistic (Nonlinear) Filters

- Based on **ranking (ordering)** the pixel values within the filter neighborhood; replace the center pixel with a value determined by rank.
- **Median filter:** replace center with the **median** of the neighborhood. Excellent for removing **salt-and-pepper (impulse) noise** while preserving edges much better than a linear averaging filter (averaging blurs edges and doesn't fully remove impulse noise; median filter removes outlier/impulse values directly since they get ranked to the extremes and discarded).
- **Max filter:** replace center with the **maximum** value in the neighborhood — finds brightest points, used to reduce "pepper" (dark) noise; also used in CNN operations (max pooling analog).

- **Min filter:** replace center with the **minimum** value — reduces "salt" (bright) noise.

Key comparison (common exam Q): averaging filter vs median filter on salt-and-pepper noise:

- **Averaging filter:** blurs noise into surrounding pixels — noise is reduced but smeared, image also gets blurred overall.
 - **Median filter:** directly eliminates isolated extreme-value noise pixels (they are outliers in the ranked list, so median ignores them), preserving sharp edges much better.
-

7. Sharpening Spatial Filters

Goal: highlight fine detail (opposite of smoothing) — based on spatial **differentiation** (derivatives enhance edges/discontinuities and de-emphasize slowly varying regions).

7.1 Foundation: Derivative Definitions (1-D, digital)

First-order derivative:

$$\partial f / \partial x = f(x+1) - f(x)$$

Second-order derivative:

$$\partial^2 f / \partial x^2 = f(x+1) + f(x-1) - 2f(x)$$

Behavior comparison (crucial conceptual point):

Feature	1st derivative	2nd derivative
Flat/constant region	zero	zero
Ramp (gradual transition)	nonzero (constant) along ramp	zero except at ramp start/end (fine "double-edge" spike)

Step edge	nonzero at the transition	produces a strong response, changes sign across the edge (zero-crossing at the edge)
Isolated noise point	nonzero	much stronger/more sensitive response (since 2nd derivative amplifies rapid changes more)
Line thickness	thick edges (responds along the whole ramp)	thinner, more precise edges

Exam takeaways:

- 1st-order derivatives → thick edges, used for edge **detection** (gradient magnitude).
- 2nd-order derivatives → much stronger response to fine detail/noise, produces a "double line" effect at edges, more sensitive but thinner responses; used for **sharpening** (Laplacian).

7.2 The Laplacian (2nd-order, isotropic)

$$\nabla^2 f = \partial^2 f / \partial x^2 + \partial^2 f / \partial y^2$$

$$= f(x+1,y) + f(x-1,y) + f(x,y+1) + f(x,y-1) - 4f(x,y)$$

- **Isotropic** = rotation-invariant (responds equally to edges in any direction) — valid for 90° rotation increments with the standard mask, and for all directions (isotropic in the continuous sense) when diagonal terms included.

Standard 3×3 masks:

Basic (4-neighbor): Extended (8-neighbor, incl. diagonals):

$$\begin{array}{ccc} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{array} \quad \begin{array}{ccc} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{array}$$

(Or the sign-inverted versions with -1's around a +4 or +8 center — mathematically equivalent, just negated.)

Sharpening using the Laplacian:

$$g(x,y) = f(x,y) + c \cdot [\nabla^2 f(x,y)]$$

- $c = -1$ if the mask used has a **negative** center coefficient (e.g. center = -4 or -8, as shown above)
- $c = +1$ if the mask used has a **positive** center coefficient (i.e., the sign-inverted mask)
- Reasoning: the Laplacian mask's output already carries a sign; you need the *original edge information added back with the correct sign* so that edges are reinforced (brightened where they should brighten, darkened where they should darken) rather than cancelled.

Worked example: center pixel $f(x,y)=100$, its 4-neighbors are 90, 95, 105, 110. Using basic mask (center -4):

$$\nabla^2 f = 90+95+105+110 - 4(100) = 400 - 400 = 0 \quad (\text{flat-ish region} \rightarrow \text{little sharpening added})$$

If instead neighbors were 60,60,140,140 (sharp local variation):

$$\nabla^2 f = 60+60+140+140 - 400 = 400-400 = 0$$

(still balances out — try an asymmetric case) neighbors 60,60,60,60, center=100:

$$\nabla^2 f = 240 - 400 = -160$$

With $c=-1$: $g = 100 + (-1)(-160) = 100+160 = 260 \rightarrow$ clipped to 255 (shows strong brightening/sharpening at this edge-like pixel).

7.3 Unsharp Masking & Highboost Filtering

Steps:

1. Blur the original image $\rightarrow$ get $f^-(x, y)$.
2. Subtract blurred from original to get the **mask**:
 $g_{\text{mask}}(x,y) = f(x,y) - f^-(x,y)$

3. Add a weighted portion of the mask back to the original:

$$g(x,y) = f(x,y) + k \cdot g_mask(x,y), \quad k \geq 0$$

- $k = 1$ → standard **unsharp masking**.
- $k > 1$ → **highboost filtering** – amplifies the high-frequency (edge) content even more strongly than standard unsharp masking, producing a more aggressively sharpened image.
- $k < 1$ → de-emphasizes sharpening contribution.

1-D intuition (classic exam figure): Original step-like signal → blurred version rounds off the step → mask (original – blurred) is near zero in flat regions but shows a positive overshoot then negative undershoot right at the edge (like a derivative-ish spike) → adding this mask back to the original produces a signal with **overshoot/undershoot spikes at edges**, which is what makes edges look "sharper" to the human eye.

7.4 First-Order-Derivative (Gradient) Sharpening – nonlinear

Gradient vector:

$$\nabla f = [g_x; g_y] = [\partial f / \partial x; \partial f / \partial y]$$

Magnitude (this is what's typically called the "gradient image" $M(x,y)$):

$$M(x,y) = \text{mag}(\nabla f) = \sqrt{g_x^2 + g_y^2}$$

- Nonlinear because of the square root (though often approximated linearly for speed):

$$M(x,y) \approx |g_x| + |g_y|$$

Using the standard 3×3 neighborhood naming:

z1 z2 z3

z4 z5 z6

z7 z8 z9

Simplest 2×2 approximation:

$$M(x,y) = |z_8 - z_5| + |z_6 - z_5|$$

Roberts Cross-Gradient Operators (diagonal differences, 2×2 masks):

$$M(x,y) \approx |z_9 - z_5| + |z_8 - z_6|$$

Masks:

$$\begin{array}{cc} -1 & 0 \\ 0 & 1 \end{array} \quad \begin{array}{cc} 0 & -1 \\ 1 & 0 \end{array}$$

Sobel Operators (3×3, most common gradient/edge operator, includes smoothing to reduce noise sensitivity):

$$\begin{array}{cc} gx: & \begin{array}{ccc} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{array} & gy: & \begin{array}{ccc} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{array} \end{array}$$

$$M(x,y) \approx |(z_7+2z_8+z_9) - (z_1+2z_2+z_3)| + |(z_3+2z_6+z_9) - (z_1+2z_4+z_7)|$$

- All coefficients of Sobel/Roberts masks sum to **zero** — expected of any derivative operator (a constant/flat region should give zero response).
- The Sobel operator's extra weighting (the "2") gives more importance to the center row/column, providing some smoothing → less noise-sensitive than Roberts.
- Gradient-based sharpening (Sobel especially) is widely used for **edge detection** (not just enhancement) — e.g., detecting defects on the boundary of a manufactured part (contact lens example), because it strongly highlights boundaries/discontinuities while suppressing flat regions.

Worked example — Sobel gx component: Neighborhood:

$$\begin{array}{ccc} 10 & 10 & 10 \\ 10 & 50 & 90 \\ 10 & 90 & 90 \end{array}$$

$z1..z9 = 10, 10, 10, 10, 50, 90, 10, 90, 90$

$gx\text{-part} = (z7+2z8+z9) - (z1+2z2+z3) = (10+180+90) - (10+20+10) = 280 - 40 = 240$

$gy\text{-part} = (z3+2z6+z9) - (z1+2z4+z7) = (10+180+90) - (10+20+10) = 280-40 = 240$

$M \approx |240| + |240| = 480 \rightarrow \text{clipped to } 255 \text{ (strong edge detected)}$

7.5 Laplacian vs Gradient – Exam Comparison

	Laplacian (2nd order)	Gradient (1st order, e.g. Sobel)
Linearity	Linear	Nonlinear (magnitude involves sqrt or abs)
Sensitivity	Very sensitive to noise & fine detail	Less noise-sensitive (Sobel has built-in smoothing)
Best use	Fine detail/general sharpening	Edge detection, prominent edge extraction
Directionality	Isotropic (rotation-invariant)	Directional components (gx, gy) combined into isotropic magnitude

8. Combining Spatial Enhancement Methods

Real enhancement pipelines often **chain multiple techniques** since no single method handles every issue. Classic example (bone scan enhancement):

1. Start with original image (a).
2. Compute **Laplacian** (b) to capture fine detail/edges.
3. Add Laplacian back to original $\rightarrow$ sharpened image (c) = (a)+(b). (But this also amplifies noise.)
4. Compute **Sobel gradient** (d) of the original – highlights strong edges, robust to noise.
5. **Smooth** the Sobel image with an averaging filter (e.g. 5×5) (e) – turns edges into broader regions usable as a mask (removes noisy speckling from the gradient image).

6. Form a **mask image** (f) = product of the Laplacian-sharpened detail (c) and the smoothed gradient mask (e) – this multiplies fine detail by "how strong an edge is nearby," so detail is enhanced mainly **near edges**, suppressing noise amplification in flat regions.
7. **Sharpened image** (g) = original (a) + mask (f).
8. Apply a final **power-law (gamma)** transformation to (g) $\rightarrow$ (h), to adjust overall contrast/brightness to taste.

Exam takeaway: This demonstrates *why* you combine methods – Laplacian alone sharpens but amplifies noise everywhere; multiplying by a smoothed gradient mask restricts the sharpening boost to regions that actually contain edges, giving a cleaner, more visually pleasing enhancement than any single method alone.

9. Quick-Reference Formula Sheet

Technique	Formula
Negative	$s = L - 1 - r$
Log	$s = c \cdot \log(1 + r)$
Power-law/Gamma	$s = c \cdot r^\gamma$
Normalized histogram	$p(r_k) = n_k / (MN)$
Histogram equalization (discrete)	$s_k = (L - 1) / (MN) \cdot \sum_{j=0..k} n_j$
Correlation	$g(x, y) = \sum \sum w(s, t) f(x + s, y + t)$
Convolution	$g(x, y) = \sum \sum w(s, t) f(x - s, y - t)$
Weighted average filter	$g = [\sum \sum w \cdot f] / [\sum \sum w]$
Laplacian	$\nabla^2 f = f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)$
Laplacian sharpening	$g = f + c \cdot \nabla^2 f$ ($c = -1$ or $+1$ depending on mask sign)
Unsharp/highboost	$g = f + k \cdot (f - \hat{f})$, $k=1$ unsharp, $k>1$ highboost

Gradient magnitude	$M = \sqrt{g_x^2 + g_y^2} \approx g_x + g_y $
Roberts	$M \approx z_9 - z_5 + z_8 - z_6 $
Sobel	$M \approx (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3) + (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7) $

10. Practice Problems (with worked solutions)

Q1. For an 8-bit image ($L=256$), apply the negative transform to pixel value 220.

$$s = 255 - 220 = \mathbf{35}$$

Q2. A 4-bit image ($L=16$) has intensity distribution: $r=0 \rightarrow n=100$, $r=5 \rightarrow n=300$, $r=10 \rightarrow n=400$, $r=15 \rightarrow n=200$ ($MN=1000$). Compute the histogram-equalized value for $r=5$.

$$\begin{aligned} \text{pr}(0)=0.1, \text{pr}(5)=0.3 \rightarrow \text{cumulative up to } r=5 &= 0.1+0.3 = 0.4 \\ s &= (L-1) \times 0.4 = 15 \times 0.4 = 6.0 \rightarrow \mathbf{s=6} \end{aligned}$$

Q3. Given 3×3 neighborhood:

```
20 20 20
20 100 20
20 20 20
```

Compute the Laplacian response at the center using the basic 4-neighbor mask, then compute the sharpened center value using $g = f + c \cdot \nabla^2 f$ with the mask having center $= -4$ (so $c = -1$).

$$\begin{aligned} \nabla^2 f &= (20+20+20+20) - 4(100) = 80 - 400 = -320 \\ g &= 100 + (-1)(-320) = 100+320 = 420 \rightarrow \text{clipped to } \mathbf{255} \end{aligned}$$

Q4. Using the same neighborhood as Q3, compute the simple gradient-magnitude approximation $M \approx |z_8 - z_5| + |z_6 - z_5|$ ($z_5 = \text{center} = 100$, $z_6 = 20$, $z_8 = 20$).

$$M = |20 - 100| + |20 - 100| = 80 + 80 = \mathbf{160}$$

Q5. Why does a 5×5 averaging filter blur an image more than a 3×3 averaging filter, and what is the trade-off?

A larger mask averages over more neighboring pixels, so rapid intensity changes (edges, fine detail, noise) get smoothed out more aggressively. Trade-off: better noise suppression but greater loss of fine detail and edge sharpness — larger masks are chosen only when heavy smoothing (e.g. gross object detection) is the goal.

Q6. True or False: Correlation and convolution always give the same result. Justify.

False in general — convolution flips the kernel 180° before sliding, correlation does not. They give the **same** result only when the kernel is symmetric (as most smoothing, Laplacian, and Gaussian kernels are), since flipping a symmetric kernel doesn't change it.

Q7. An image has salt-and-pepper noise. Compare choosing a 3×3 averaging filter vs a 3×3 median filter to clean it.

Averaging filter smears each noise spike's extreme value into its neighbors — noise is reduced in magnitude but spread out, and the whole image becomes blurred. Median filter replaces each pixel with the median of its neighborhood; since salt-and-pepper values are extreme outliers, they get pushed to the top/bottom of the ranked list and excluded — noise is removed with minimal blurring, and edges are much better preserved.

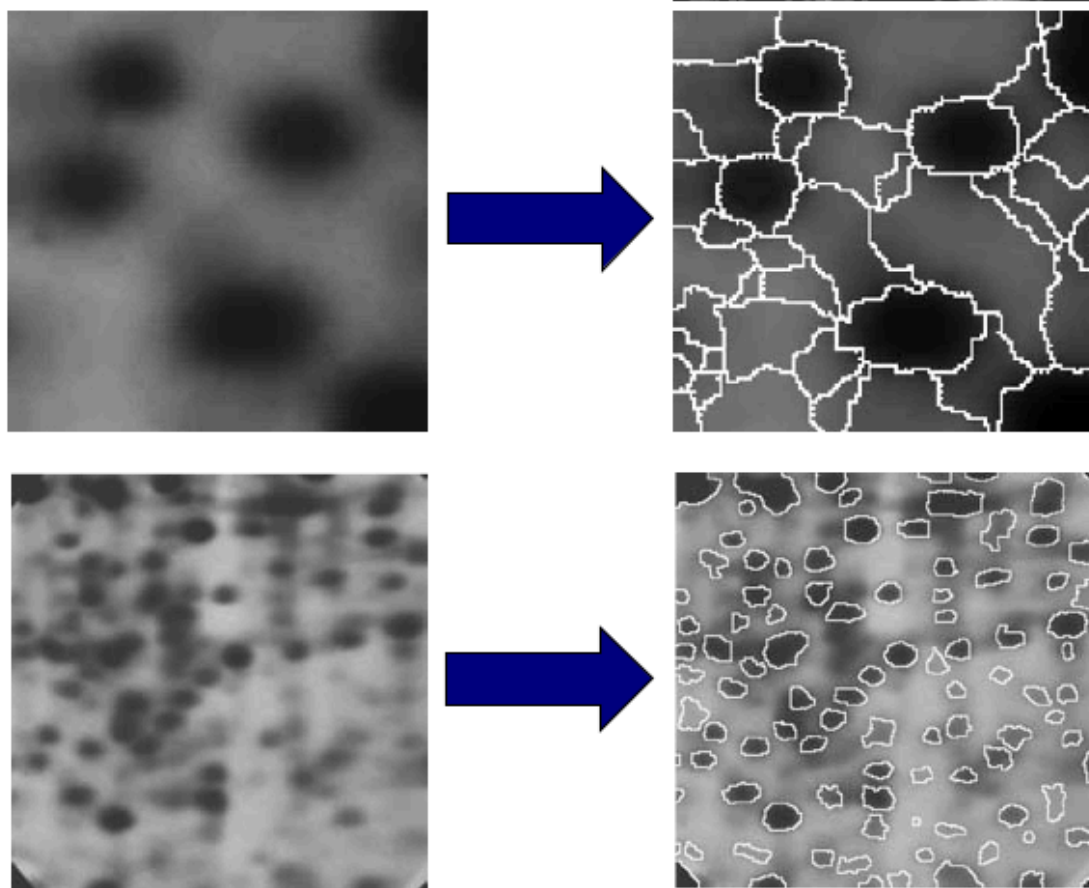
Image Segmentation — Edge Detection & Thresholding — Exam Notes

Question-setter lens: mask/kernel identification (given numbers $\rightarrow$ name operator), formula derivations, worked numeric problems (gradient at Z5, Otsu calc), "why does X fail" scenario Qs, algorithm-steps ordering.

1. Image Segmentation — Definition

Def: process partitioning image into constituent parts/objects. Target: isolate objects/parts of interest. **First step in automated CV pipeline.**

Likely Q: Why is segmentation called first step of CV app? **Answer:** downstream tasks (recognition, measurement, tracking) need isolated objects first — can't classify/measure until region is separated from background.



2. Two Basic Approaches (core dichotomy — very high yield)

Gray-level segmentation based on:

1. **Discontinuity** — partition based on abrupt gray-level changes → edge-based
2. **Similarity** — group pixels alike → thresholding, region growing, region splitting/merging

Likely Q (scenario): Given noisy image with clear object/background gray split, which family of technique fits better — discontinuity or similarity based? **Answer:** similarity-based (thresholding) — abrupt-change detectors (discontinuity) are

noise-sensitive; if regions differ mainly by overall gray level not sharp boundary, thresholding suits better.

3. Detection of Discontinuities – 3 Types

Points, Lines, Edges — found via mask + correlation.

Correlation formula (memorize exact): $R_{z5} = w_1z_1 + w_2z_2 + \dots + w_9z_9 = \sum w_i \cdot z_i$ (i=1 to 9)

mask = w_i 's (3×3), image window = z_i 's, response computed at center pixel z_5 .

Likely Q (numeric): given 3×3 mask weights + 3×3 image patch, compute R at center.

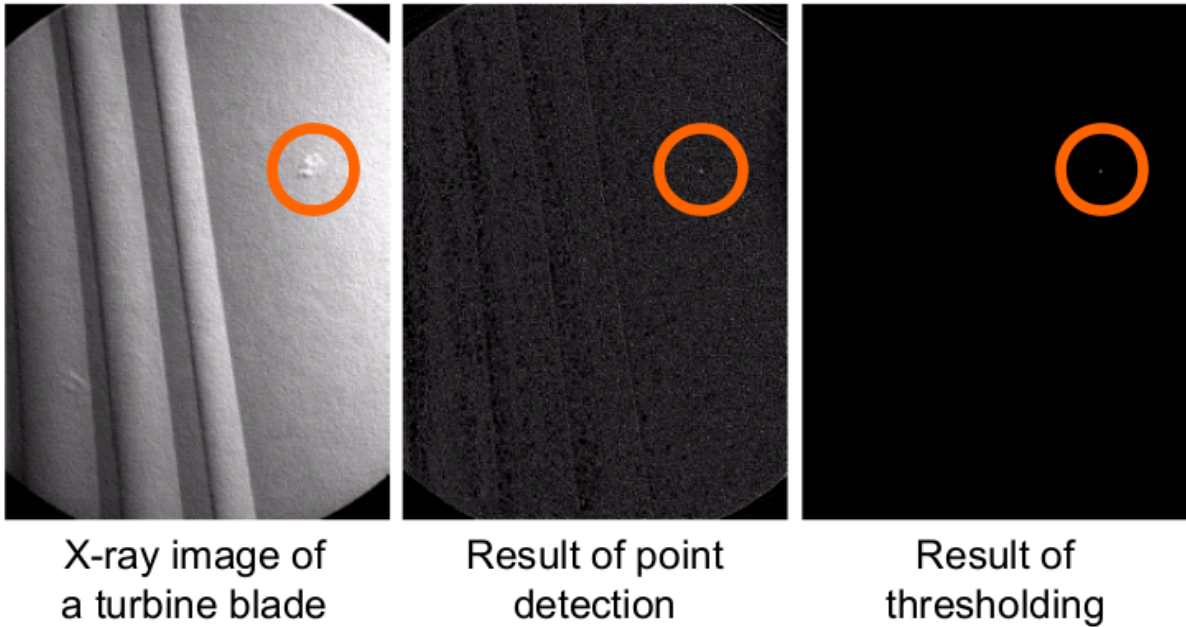
Method: multiply corresponding cells, sum all 9 products.

4. Point Detection

Mask (memorize exact):

```
-1 -1 -1
-1  8 -1
-1 -1 -1
```

Point detected where filtered response **exceeds a threshold**.



Likely Q (conceptual): why center weight = 8, others = -1? **Answer:** sum of weights = 0 (flat/homogeneous region → zero response); isolated bright/dark point against uniform background → large response, so setting threshold isolates true points.

Likely Q (scenario): X-ray turbine blade defect detection – why point detection + thresholding used together (not point detection alone)? **Answer:** raw point-detector output has noise/small responses everywhere; thresholding isolates only responses above cutoff = true defect points.

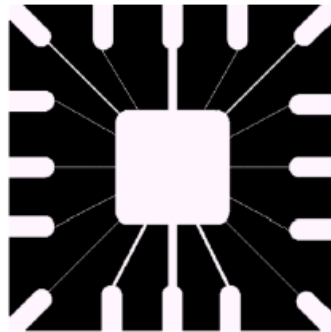
5. Line Detection

4 masks – **Horizontal**, **+45°**, **Vertical**, **-45°** – each extracts 1-pixel-thick lines in that direction.

Pattern: row/diagonal of 2's in the line's direction, -1's elsewhere. Weights sum to 0 (same flat-response-zero logic as point mask).

-1	-1	-1	-1	-1	2	-1	2	-1	2	-1	-1
2	2	2	-1	2	-1	-1	2	-1	-1	2	-1
-1	-1	-1	2	-1	-1	-1	2	-1	-1	-1	2
Horizontal			+45°			Vertical			-45°		

Binary image of a wire bond mask



After processing with -45° line detector



Result of thresholding filtering result



Likely Q (identify-the-mask, common MCQ): given a mask matrix, name the direction it detects. **Trick:** locate row of 2's — that's the line direction it responds most strongly to.

6. Edge Detection — Core Definition

Edge = jump in intensity = discontinuity/abrupt intensity change. Formally: **set of connected pixels lying on boundary between two regions.**

Edges → characterize object boundaries → useful for segmentation, registration, identification.

Edge detection considers the intensity change that occurs in the pixel at the boundary (or edges) of an image.



a b c

FIGURE 10.8

From left to right, models (ideal representations) of a step, a ramp, and a roof edge, and their corresponding intensity profiles.

An edge is a set of connected pixels that lie on the boundary between two regions.

3 ideal edge models (Figure 10.8 – memorize names):

- **Step edge** – instant jump
- **Ramp edge** – gradual transition
- **Roof edge** – spike (rises then falls, e.g. thin line)

Likely Q (scenario): real camera images rarely show pure step edges – why? **Answer:** optical blur/sampling → sharp discontinuities smoothed into ramp edges in practice.

7. Derivative Behavior at Edges (high-yield conceptual)

- **1st derivative:** local **maximum** at edge → tells **where** edge is (nonzero across ramp width)
- **2nd derivative:** produces **zero crossing** at edge center → tells **edge direction/fine localization**

Derivative filters → high intensity at edges, low intensity in homogeneous regions.

Likely Q (scenario): given a ramp edge's intensity profile, sketch expected 1st and 2nd derivative shape. **Answer:** 1st derivative = plateau/pulse over ramp width (nonzero

constant during transition); 2nd derivative = positive spike at ramp start, negative spike at ramp end, crossing zero in between (the "zero crossing" is the precise edge location).

8. 1st & 2nd Derivative Formulas (exact — common fill-in-blank)

1st derivative: $\partial f / \partial x = f(x+1) - f(x) \rightarrow$ "difference between subsequent values, measures rate of change"

2nd derivative: $\partial^2 f / \partial^2 x = f(x+1) + f(x-1) - 2f(x) \rightarrow$ "takes into account values both before and after current value"

Likely Q (numeric): given $f(x)$ values at $x-1$, x , $x+1$, compute 1st and 2nd derivative at x .
Method: plug into formulas directly.

9. Edge Detection Operator Families (memorize categorization — high MCQ yield)

Order	Category	Operators
1st derivative	Gradient operators	Sobel, Prewitt, Roberts
2nd derivative	Laplacian operator	—
Combined	LoG (Laplacian of Gaussian)	"Mexican hat"

Likely Q: classify: "Which operator uses 2nd derivative and combines noise smoothing?"
Answer: LoG — Gaussian for noise removal + Laplacian for edge detection (2nd derivative).

10. Gradient Operator – Formula Chain

The gradient operator of an image $f(x,y)$ at location x, y can be expressed by it's magnitude as:

$$G[f(x,y)] = [G_x^2 + G_y^2]^{1/2}$$
$$= \sqrt{\left(\frac{df}{dx}\right)^2 + \left(\frac{df}{dy}\right)^2}$$

For practical reasons this can be simplified as:

$$\nabla f \approx |G_x| + |G_y|$$

Likely Q (conceptual): why use $|G_x|+|G_y|$ instead of the true magnitude formula in practice? **Answer:** computational efficiency – avoids square/square-root ops, good enough approximation for edge magnitude.

11. Roberts Operator (2×2 – smallest gradient kernel)

♣ Consider the 3x3 region shown below and approximate the gradient at z_5 .

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

$$df/dx \approx z_5 - z_6$$

$$df/dy \approx z_5 - z_8$$

$$|\nabla f| \approx |z_5 - z_6| + |z_5 - z_8|$$

♣ We can thus define a kernel to implement the gradient approximation as follows, which is known as **Robert operator**.

1	-1
0	0

1	0
-1	0

Likely Q (numeric, very common): given a 3x3 pixel grid ($z_1..z_9$ values), compute Roberts gradient magnitude at z_5 . **Method:** $|z_5 - z_6| + |z_5 - z_8|$, plug in given values.

12. Sobel Operator (weight-2 center row/col – smoothing built in)

We can again approximate the magnitude of the gradient at the center of a 3x3 region and obtain the following. Based on these equations we can derive the **Sobel Operators**.

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

$$\nabla f \approx \left| (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3) \right| + \left| (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7) \right|$$

The weight value 2 is to achieve smoothing by giving more important to the center point

-1	-2	-1
0	0	0
1	2	1

-1	0	1
-2	0	2
-1	0	1

To detect edges of an image it is filtered using both operators the results of which are added together.

Key exam point: weight 2 at center row/column = smoothing, gives more importance to center point (reduces noise sensitivity vs Prewitt).

To detect edges: filter with **both** kernels (horizontal + vertical), results **added together**.

13. Prewitt Operator (same structure as Sobel, no center weighting)

We can again approximate the magnitude of the gradient at the center of a 3x3 region as shown by the following eq^{ns}:

Based on these equations we can derive the **Prewitt Operators**.

$$|\nabla f| \cong \left| (z_7 + z_8 + z_9) - (z_1 + z_2 + z_3) \right| + \left| (z_3 + z_6 + z_9) - (z_1 + z_4 + z_7) \right|$$

-1	-1	-1
0	0	0
1	1	1

-1	0	1
-1	0	1
-1	0	1

Likely Q (compare, very common exam pattern): Sobel vs Prewitt vs Roberts – key structural difference? **Answer:** Roberts = 2x2, diagonal differences, most noise-sensitive, simplest/fastest. Prewitt = 3x3, uniform weight ± 1 . Sobel = 3x3, center row/col weighted ± 2 for smoothing $\rightarrow$ less noise-sensitive than Prewitt. All are 1st-derivative/gradient family.

Likely Q (numeric): given 3x3 patch, compute Sobel Gx, Gy, combined magnitude $\approx |G_x| + |G_y|$.

14. Gradient Vector – Edge Strength & Direction (Fig 10.12)

Gradient vector points in direction of **greatest rate of intensity increase**. **Edge direction is perpendicular to gradient vector direction** – memorize this relationship, common trick question.

Edge direction = $\alpha - 90^\circ$ where α = gradient vector angle.

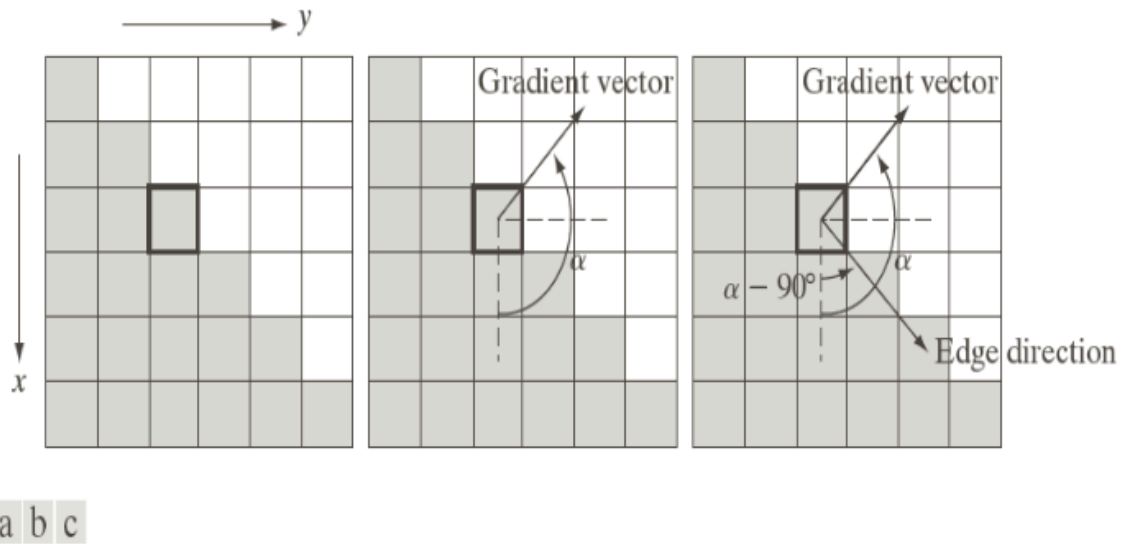


FIGURE 10.12 Using the gradient to determine edge strength and direction at a point. Note that the edge is perpendicular to the direction of the gradient vector at the point where the gradient is computed. Each square in the figure represents one pixel.

Likely Q (scenario): if gradient vector at a point makes angle α with horizontal, what's the edge orientation? **Answer:** perpendicular to gradient vector $\rightarrow$ edge angle = $\alpha - 90^\circ$.

15. Laplacian Operator (2nd derivative, isotropic)

So, the Laplacian can be given as follows:

$$\nabla^2 f = [f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1)] - 4f(x, y)$$

0	1	0
1	-4	1
0	1	0

Key property: strong **zero crossing** at edges → Laplacian edge detector locates zeros of 2nd derivative rather than peaks of 1st derivative.

Likely Q (conceptual): Laplacian vs gradient operator — fundamental difference in how each locates an edge? **Answer:** Gradient (1st deriv) finds edge via local **maximum** magnitude; Laplacian (2nd deriv) finds edge via **zero crossing** between a positive and negative spike.

16. Derivatives & Noise (critical weakness — recurring exam theme)

Derivative-based edge detectors are extremely sensitive to noise. Fix: **smooth image before edge detection.**

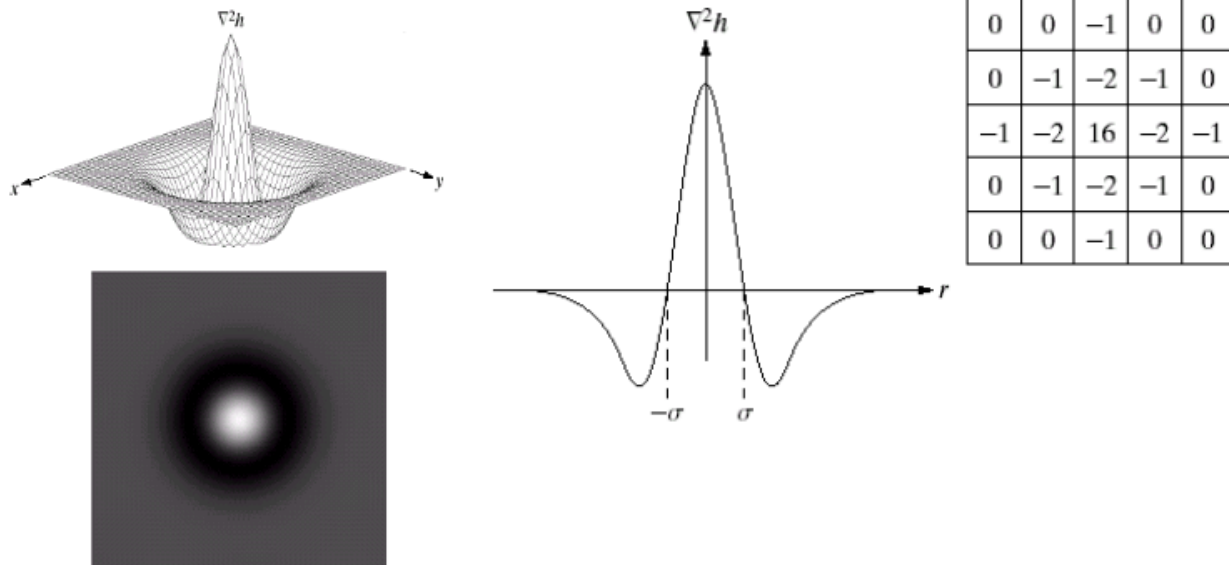
Likely Q (scenario, high probability): noisy image run through Sobel gives edge map full of false positives everywhere, not just true object boundaries. Diagnose + fix. **Answer:** derivative operators amplify high-frequency noise (noise = local intensity jumps, same signature as edges) → false edges. Fix: apply smoothing (e.g. Gaussian blur) before differentiation — leads directly into LoG concept.

17. Laplacian of Gaussian (LoG / "Mexican Hat")

Combines: **Gaussian (noise removal) + Laplacian (edge detection, 2nd derivative)** in one filter — solves the noise-sensitivity problem of raw derivatives.

Example 5×5 LoG-style kernel (memorize shape, not necessarily exact numbers):

The Laplacian of Gaussian (LoG or Mexican hat) filter uses the Gaussian for noise removal and the Laplacian for edge detection.



Likely Q (conceptual): why does LoG outperform plain Laplacian on noisy images?

Answer: Gaussian pre-smoothing suppresses high-frequency noise before the noise-sensitive 2nd-derivative (Laplacian) step runs — reduces false zero-crossings from noise.

18. Applications of Edge Detection (list — quick recall Q)

Industrial inspection (3-D) · 3-D object measurement · autonomous vehicles/robotics · medical/biomedical imaging · transport/traffic scene analysis · 3-D urban/town-planning databases.

19. Thresholding — Basic (Single Value)

Single value thresholding can be given mathematically as follows:

$$g(x, y) = \begin{cases} 1 & \text{if } f(x, y) > T \\ 0 & \text{if } f(x, y) \leq T \end{cases}$$

Formula (exact): $g(x,y) = 1$ if $f(x,y) > T$ $g(x,y) = 0$ if $f(x,y) \leq T$

Simple binary segmentation by comparing pixel intensity to threshold T .

Likely Q (scenario, poker-robot example from slides): thresholding used to isolate card symbols from background – what happens if T set too low vs too high? **Answer:** T too low → background noise/shadow treated as foreground (extra black regions/speckle). T too high → parts of actual object lost (foreground merges with dark background/shadow areas, image mostly black). Both distort segmentation – **threshold selection is critical.**

20. Basic Global Thresholding Algorithm (ITERATIVE – sequence/steps question very likely)

1. Select initial estimate for T (typically average gray level of image)
2. Segment image using $T \rightarrow G1$ (pixels $> T$), $G2$ (pixels $\leq T$)
3. Compute average gray level of $G1 \rightarrow \mu_1$, of $G2 \rightarrow \mu_2$
4. Compute new threshold: $T = (\mu_1 + \mu_2) / 2$
5. Repeat steps 2–4 until difference in T between iterations $<$ predefined parameter dT

Works well when: clear valley between histogram modes (object vs background separable).

Likely Q (numeric, high probability): given initial gray-level groups/means, walk through 1–2 iterations computing new T. **Method:** apply formula $T = (\mu_1 + \mu_2)/2$ iteratively till convergence $< dT$.

Likely Q (conceptual): why does this algorithm fail on unimodal or multimodal (non-bimodal) histograms? **Answer:** algorithm assumes 2 separable groups around a valley; single value thresholding **only works for bimodal histograms** — needs a clear valley to converge meaningfully.

21. Problems With Single-Value Thresholding

- Only works for **bimodal histograms**
- Multimodal histograms need multiple thresholds ($T_1, T_2, \dots$)
- **Uneven illumination** severely disrupts single-value schemes — same object can straddle multiple gray levels across the image due to lighting gradient, breaking the clean bimodal separation

Likely Q (scenario): object photographed under a light gradient (brighter on one side) — single global threshold produces a result where part of object is missed/misclassified as background. Explain + name the fix category. **Answer:** uneven illumination shifts local gray levels so a single global T can't separate object/background everywhere → need **adaptive thresholding** (see below).

22. Basic Adaptive Thresholding

Idea: divide image into sub-images, threshold each individually — threshold depends on pixel's **location** → "adaptive."

Process: if a sub-image still gives poor results (mixed success), **further subdivide** the troublesome region and re-threshold.

Likely Q (scenario, exam favorite — matches slide example directly): global threshold on unevenly-lit polygon image gives mixed/partial segmentation. Propose next step. **Answer:** apply adaptive thresholding — split image into regions, threshold each region separately using local statistics; further subdivide any sub-region that still segments poorly.

23. Otsu's Thresholding – Concept (HIGH-YIELD, numeric worked example likely)

General automatic global thresholding steps:

1. Process input image
2. Obtain image histogram
3. Compute threshold value $T \leftarrow$ **this is where Otsu differs from other algorithms**
4. Replace pixels: white where intensity $> T$, black otherwise

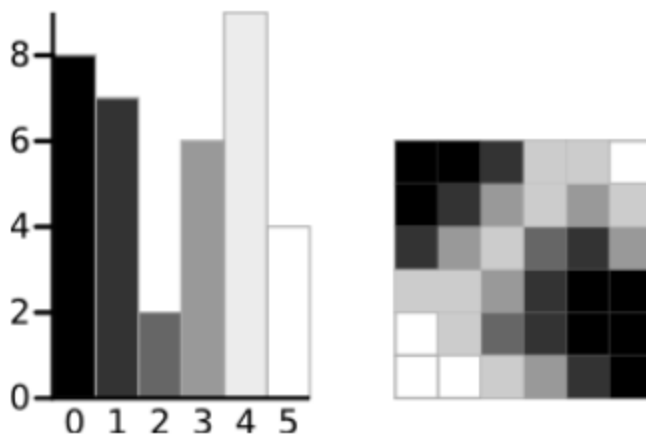
Otsu's core idea: separate histogram into 2 clusters (background, foreground) via threshold that **minimizes within-class (weighted) variance**:

$$\sigma^2w = W_b \cdot \sigma^2_b + W_f \cdot \sigma^2_f$$

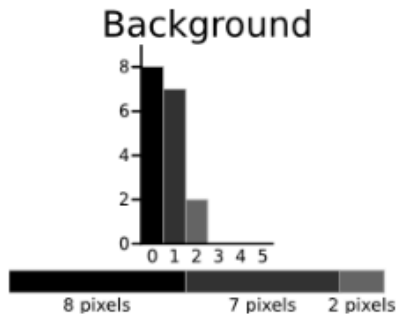
where W_b, W_f = proportion of pixels in background/foreground classes; σ^2_b, σ^2_f = variance within each class.

Algorithm (conceptual): try every possible T , compute σ^2w for each, pick T that gives **minimum** σ^2w .

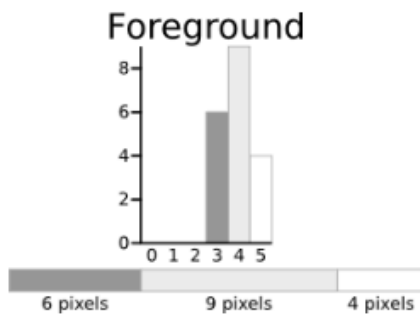
Worked numeric example (from slides – memorize the method, not just the numbers)



Histogram over levels 0–5, e.g. counts: 0→8, 1→7, 2→2, 3→6, 4→9, 5→4 (total = 36 pixels).



$$\begin{aligned} \text{Weight } W_b &= \frac{8 + 7 + 2}{36} = 0.4722 \\ \text{Mean } \mu_b &= \frac{(0 \times 8) + (1 \times 7) + (2 \times 2)}{17} = 0.6471 \\ \text{Variance } \sigma_b^2 &= \frac{((0 - 0.6471)^2 \times 8) + ((1 - 0.6471)^2 \times 7) + ((2 - 0.6471)^2 \times 2)}{17} \\ &= \frac{(0.4187 \times 8) + (0.1246 \times 7) + (1.8304 \times 2)}{17} \\ &= 0.4637 \end{aligned}$$



$$\begin{aligned} \text{Weight } W_f &= \frac{6 + 9 + 4}{36} = 0.5278 \\ \text{Mean } \mu_f &= \frac{(3 \times 6) + (4 \times 9) + (5 \times 4)}{19} = 3.8947 \\ \text{Variance } \sigma_f^2 &= \frac{((3 - 3.8947)^2 \times 6) + ((4 - 3.8947)^2 \times 9) + ((5 - 3.8947)^2 \times 4)}{19} \\ &= \frac{(4.8033 \times 6) + (0.0997 \times 9) + (4.8864 \times 4)}{19} \\ &= 0.5152 \end{aligned}$$

$$\begin{aligned} \text{Within Class Variance } \sigma_W^2 &= W_b \sigma_b^2 + W_f \sigma_f^2 = 0.4722 * 0.4637 + 0.5278 * 0.5152 \\ &= 0.4909 \end{aligned}$$

For candidate T=2 (background = levels 0,1,2; foreground = levels 3,4,5):

- $W_b = (8+7+2)/36 = 0.4722$
- $\mu_b = (0 \times 8 + 1 \times 7 + 2 \times 2)/17 = 0.6471$

- $\sigma^2b = \Sigma(\text{level} - \mu_b)^2 \times \text{count} / 17 = 0.4637$
- $W_f = (6+9+4)/36 = 0.5278$
- $\mu_f = (3 \times 6 + 4 \times 9 + 5 \times 4)/19 = 3.8947$
- $\sigma^2f = 0.5152$
- $\sigma^2w = W_b \cdot \sigma^2b + W_f \cdot \sigma^2f = 0.4722 \times 0.4637 + 0.5278 \times 0.5152 = 0.4909$

Repeat for every T (0 through 5), tabulate σ^2w , pick **T with minimum σ^2w** → in the slide example, **T=3 gives lowest σ^2w (0.4909)** → chosen threshold.

Likely Q (numeric, near-certain exam question): given a small histogram (5–8 gray levels + pixel counts), compute W_b , μ_b , σ^2b , W_f , μ_f , σ^2f , and σ^2w for a given candidate T; or find the T that minimizes σ^2w across all candidates. **Method (memorize step order):**

1. Split histogram at candidate T into background/foreground
2. Weight = class pixel count / total pixels
3. Mean = $\Sigma(\text{level} \times \text{count}) / \text{class total count}$
4. Variance = $\Sigma(\text{level} - \text{mean})^2 \times \text{count} / \text{class total count}$
5. Within-class variance = $W_b \cdot \sigma^2b + W_f \cdot \sigma^2f$
6. Repeat across all T, choose T with **minimum** within-class variance

Likely Q (conceptual): why "minimize within-class variance" (equivalent to maximizing between-class variance) rather than just splitting at the histogram's midpoint? **Answer:** goal is to make each class (background/foreground) as internally homogeneous as possible — minimum within-class variance = classes are tight/well-separated = best possible bimodal split, adapts to actual histogram shape rather than assuming midpoint works.

24. Global Thresholding — Issue With Noise (ties back to earlier theme)

Even Otsu (global thresholding) fails on noisy images — noisy histogram lacks a clean bimodal shape, corrupting threshold selection.

Fix (same pattern as edge detection): smooth the image first (e.g., 5×5 averaging mask) → cleans up histogram into clear bimodal shape → then apply Otsu's method successfully.

Likely Q (scenario, ties multiple topics together – strong final-exam candidate): noisy image thresholded directly with Otsu gives poor/speckled segmentation. Explain why, and describe the two-step fix, referencing what you know about derivatives-and-noise from edge detection. **Answer:** noise broadens/blurs the histogram, destroying the bimodal valley Otsu depends on – parallel to how noise corrupts derivative-based edge detection. Fix: **pre-smooth image** (e.g., averaging/Gaussian mask) to sharpen histogram peaks and restore bimodal shape → then run Otsu on the smoothed image/histogram for clean thresholding.

Quick-Fire Formula Sheet (cram list)

Concept	Formula
Mask response	$R = \sum w_i \cdot z_i$
1st derivative	$f(x+1) - f(x)$
2nd derivative	$f(x+1) + f(x-1) - 2f(x)$
Gradient magnitude (exact)	$\sqrt{((df/dx)^2 + (df/dy)^2)}$
Gradient magnitude (approx)	$ G_x + G_y $
Roberts	$ z_5 - z_6 + z_5 - z_8 $
Sobel	$ (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3) + (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7) $
Prewitt	$ (z_7 + z_8 + z_9) - (z_1 + z_2 + z_3) + (z_3 + z_6 + z_9) - (z_1 + z_4 + z_7) $
Laplacian	$f(x+1,y) + f(x-1,y) + f(x,y+1) + f(x,y-1) - 4f(x,y)$
Global threshold (basic)	$T = (\mu_1 + \mu_2) / 2$, iterate
Otsu within-class variance	$\sigma^2_w = W_b \cdot \sigma^2_b + W_f \cdot \sigma^2_f$

Exam Strategy Notes

- Highest-probability numeric Qs: Roberts/Sobel/Prewitt gradient computation on a given 3×3 patch; Otsu within-class-variance table calc for a small histogram; basic global threshold iteration.
- Highest-probability conceptual Qs: gradient (1st deriv, max-based) vs Laplacian (2nd deriv, zero-crossing-based); why derivatives fail under noise + how LoG/smoothing fixes it; why single-value thresholding fails on uneven illumination/multimodal histograms → leads to adaptive thresholding.
- Mask-identification trick: sum of weights = 0 for point/line/Sobel/Prewitt/Laplacian masks — useful quick check when asked "is this a valid edge/point mask?"
- Recurring narrative thread across whole lecture: **noise sensitivity** → **smoothing as the fix** — appears in derivatives (LoG) AND in thresholding (smooth before Otsu). Great link to draw in a long-answer scenario question.

Image Segmentation (Canny Edge Detection)— Lec 5

1. What is Image Segmentation?

Partitioning image into meaningful regions/objects (separating foreground from background, or grouping pixels by similarity). Main families:

Category	Idea	Examples
Threshold-based	Pick intensity cutoff(s)	Otsu's Threshold, Mean Shift
Region-based	Grow/merge/split pixel groups	Region Growing, Region Split & Merge
Edge-based	Find boundaries via gradients	Canny
Watershed	Treat image as topographic surface	Watershed, Marker-Controlled Watershed
Clustering-based	Group pixels in feature space	K-Means, Fuzzy C-Means
Neural network-based	Learned segmentation (CNNs, etc.)	—

Likely exam Q: "Name segmentation categories" or "classify method X under correct category" – memorize the tree diagram above.

2. Region Growing (Region-based)

Core idea: Start from seed pixel(s) (can be random), then grow outward by absorbing neighboring pixels that are similar enough.

Formal rule: Given input image $f(x,y)$, seed image $S(x,y)$, threshold T :

- If $|f(x_i, y_i) - S(x,y)| \leq T \rightarrow f(x_i, y_i)$ belongs to region $S_i(x,y)$
- Else $\rightarrow$ does not belong

Algorithm steps:

1. Pick seed pixel.
2. Compare neighboring pixels to seed/region model using threshold T (based on intensity or distance).
3. Aggregate similar neighbors into the region; update the cluster's model (e.g., running average) with newly added pixels.
4. Repeat until region stops growing.
5. Start new seed elsewhere, continue until image covered.

Key facts:

- It's a **generalized flood fill**.
- Regions grown **one at a time, not simultaneously** – so threshold choice matters a lot: if T is poorly chosen, an early region can "dominate" and swallow pixels that should belong to a later region.
- Demonstrated on lightning image: smaller T (e.g., 190–255) keeps segmentation tight/sparse; larger range (125–255) captures much more (including noise/less relevant branches).

Common exam trap: Confusing this with clustering – region growing is *spatially* constrained (must connect to existing region), clustering is not.

3. K-Means Clustering (Clustering-based)

Core idea: Partition all pixels into k clusters based on intensity/color similarity (not spatial adjacency).

Algorithm:

1. Choose initial k cluster centers $m_j, j = 1 \dots k$ (values within image intensity range).
2. Assign each pixel $f(x_i, y_i)$ to nearest cluster by Euclidean distance: assign to cluster j that minimizes $\|f(x_i, y_i) - m_j\|$
3. Update cluster means (recompute m_j as average of pixels assigned to it).
4. Repeat steps 2–3 until convergence (max iterations reached, or means stop changing).

Key facts:

- Unsupervised, iterative, converges to local optimum (result depends on initial centers).
- Works in feature space (e.g., RGB or grayscale intensity), not necessarily spatially contiguous regions.
- Example shown: landscape photo segmented into sky/vegetation/water clusters by color similarity.

Exam trap: K-Means requires specifying k in advance; it does NOT guarantee spatially connected regions like Region Growing does.

4. Canny Edge Detection (Edge-based)

Definition: Multi-stage algorithm detecting a wide range of edges. **5 steps** (very likely to be asked in order):

Step 1 – Noise Reduction (Gaussian Blur)

- Blurs image before further processing to reduce noise sensitivity.
- Typical params: $\sigma = 1.4$, kernel size 5×5 (found empirically/trial-and-error).
- Why: gradient calculation (next step) is sensitive to noise; blurring first prevents false edges.

Step 2 – Gradient Calculation

- Computed using a **Sobel filter** (in x and y directions).

- Edge = where pixel intensity changes sharply → gradient magnitude and direction (θ) computed at every pixel.
- Gradient magnitude $\approx \sqrt{G_x^2 + G_y^2}$; direction = $\text{atan2}(G_y, G_x)$.

Step 3 – Non-Maximum Suppression (NMS)

- Goal: thin out thick edges → produce 1-pixel-wide edges.
- Method: at each pixel, compare gradient magnitude to neighbors **along the gradient direction** (rounded to one of the 8 principal directions/angles like $-\pi$, $-\pi/4$, $\pi/2$, etc.). Keep pixel only if it's a local maximum in that direction; suppress (zero out) otherwise.

Step 4 – Double Thresholding

- Classifies pixels into 3 categories using two thresholds (High, Low):
 - **Strong pixels** – intensity above High Threshold → definitely part of an edge.
 - **Weak pixels** – intensity between Low and High Threshold → possibly part of an edge.
 - **Non-relevant pixels** – intensity below Low Threshold → discarded, not an edge.

Step 5 – Edge Tracking by Hysteresis

- Converts weak pixels into strong ones **only if** at least one neighboring pixel (in local neighborhood) is already a strong pixel.
- If no strong pixel nearby → weak pixel is suppressed (removed).
- This "connects" real edges and removes noise-induced weak responses that aren't touching a genuine edge.

Summary table (good for quick recall):

Step	Purpose	Tool/Concept
1. Noise reduction	Reduce noise before edge detection	Gaussian blur, $\sigma=1.4$, 5×5 kernel
2. Gradient calculation	Find intensity change magnitude & direction	Sobel filter

3. Non-max suppression	Thin edges to 1px	Compare along gradient direction
4. Double threshold	Classify strong/weak/non-relevant	High & Low threshold
5. Hysteresis	Keep weak pixels connected to strong ones	Neighbor-based promotion

Likely exam Qs: "List the 5 steps of Canny in order," "Why Gaussian blur first?", "What's the difference between strong/weak/irrelevant pixel," "Why is hysteresis needed after double thresholding?" (Answer: double thresholding alone would either miss real-but-faint edges or keep too much noise; hysteresis links weak-but-genuine edge pixels to strong ones while dropping isolated noise.)

5. Otsu's Thresholding (Threshold-based, part of general auto-thresholding)

General automatic global thresholding steps:

1. Process input image.
2. Obtain image histogram (pixel intensity distribution).
3. Compute threshold value T . (*Different algorithms differ mainly in this step.*)
4. Replace pixels: white if intensity $> T$, black otherwise.

Otsu's specific idea:

Separate histogram into two classes (background & foreground) at threshold t , choosing t that **minimizes the within-class (weighted) variance**:

$$\sigma^2_w = W_b \cdot \sigma^2_b + W_f \cdot \sigma^2_f$$

Where:

- W_b, W_f = probability (proportion) of pixels in background/foreground class, split by threshold t (t ranges 0–255).
- σ^2_b, σ^2_f = variance of intensities within each class.
- μ_b, μ_f = mean intensity within each class (needed to compute variance).

Worked example logic (from slides, using toy histogram with bins 0–5):

For a candidate threshold $T=3$ (background = bins 0,1,2 $\rightarrow$ 8,7,2 pixels; foreground = bins 3,4,5 $\rightarrow$ 6,9,4 pixels):

Background ($t=3$):

- $W_b = (8+7+2)/36 = 0.4722$
- $\mu_b = [(0 \times 8) + (1 \times 7) + (2 \times 2)] / 17 = 0.6471$
- $\sigma^2_b = [((0-0.6471)^2 \times 8) + ((1-0.6471)^2 \times 7) + ((2-0.6471)^2 \times 2)] / 17 = 0.4637$

Foreground ($t=3$):

- $W_f = (6+9+4)/36 = 0.5278$
- $\mu_f = [(3 \times 6) + (4 \times 9) + (5 \times 4)] / 19 = 3.8947$
- $\sigma^2_f = [((3-3.8947)^2 \times 6) + ((4-3.8947)^2 \times 9) + ((5-3.8947)^2 \times 4)] / 19 = 0.5152$

Within-class variance: $\sigma^2_w = (0.4722 \times 0.4637) + (0.5278 \times 0.5152) = \mathbf{0.4909}$

Algorithm (in practice):

- Compute σ^2_w for every possible threshold T (0 to 255).
- Pick T that gives the **minimum** σ^2_w .
- In the slide's toy example, comparing $T=0..5$, **$T=3$ gives lowest σ^2_w (0.4909)** $\rightarrow$ so $T=3$ is chosen as Otsu's threshold.
- Apply threshold: pixels $\leq T \rightarrow$ black, pixels $> T \rightarrow$ white $\rightarrow$ produces binary segmented image.

Real example shown: boat-on-water photo $\rightarrow$ Otsu's method cleanly separates dark boat/people (foreground) from bright sky/water (background).

Exam trap: Remember Otsu **minimizes within-class variance** (equivalently maximizes between-class variance — same optimum). Know how to compute W_b , μ_b , σ^2_b (and same for foreground) from a small pixel-count histogram given a specific T , and be able to pick the best T by comparing σ^2_w across candidates.

Quick Comparison Table (great last-minute review)

Method	Category	Needs seed/init?	Spatially constrained?	Output
Region Growing	Region-based	Yes (seed pixel)	Yes	Grown region(s)
K-Means	Clustering-based	Yes (k, initial centers)	No	k clusters
Canny	Edge-based	No	N/A (edge map)	Binary edge map
Otsu	Threshold-based	No	No	Binary image (2 classes)

High-Yield Formulas to Memorize

- Region growing rule: $|f(x_i, y_j) - S(x, y)| \leq T$
- K-Means assignment: $\min_j ||f(x_i, y_i) - m_j||$
- Otsu within-class variance: $\sigma^2_w = W_b \cdot \sigma^2_b + W_f \cdot \sigma^2_f$

Morphological Image Processing – Slide 6

1. What Is Morphology?

Morphology = form/structure of an object, or arrangement/interrelationship between parts of an object.

Morphological image processing = techniques dealing with the *shape* of objects in an image.

- Purpose: remove imperfections introduced during **segmentation** (noise, small gaps, broken edges).
- Typically operates on **binary images** (foreground = 1/on, background = 0/off), but grayscale versions exist too.
- **Exam trap:** Some textbook examples (letter "A" figures) use convention where **1 = black pixel** – always check the convention stated on the slide before answering.

2. Structuring Elements (SE), Hits & Fits

A **structuring element** is a small shape (any size/type) with a defined **origin**, used to probe the image.

- Common simplification: rectangular SEs with origin at the **middle pixel**.
- Example SEs: 3×3 all-ones square, cross ("+") shape, etc.

Fit: ALL "on" pixels of the SE cover "on" pixels of the image (SE completely contained in foreground). **Hit:** AT LEAST ONE "on" pixel of the SE covers an "on" pixel of the image (any overlap).

Key exam distinction:

- Dilation uses **HIT** logic (any overlap → output = 1)
- Erosion uses **FIT** logic (full containment → output = 1)

All morphological operations are built from these two ideas.

3. The Two Fundamental Operations

Morphological ops slide the SE over every pixel of the input image, comparing the SE's neighborhood to produce a new pixel in the output image (same size as input).

3.1 Dilation ($\oplus$)

Definition: Thickens/enlarges objects in a binary image.

$$g(x,y) = f \oplus s$$

Rule: SE origin placed at (x,y). $g(x,y) = \begin{cases} 1 & \text{if } s \text{ hits } f \\ 0 & \text{otherwise} \end{cases}$

Set-theoretic definition: $A \oplus B = \{z \mid (\hat{B})_z \cap A \neq \varnothing\}$
(Translate reflected B to point z; if it intersects A at all → z belongs to the dilation.)

Effects / uses:

- Repairs breaks (e.g., closes a gap in a ring/letter "O")
- Repairs intrusions (fills small notches on a boundary)
- **Watch out: dilation always enlarges objects** (bigger SE → more growth, e.g. 3×3 vs 5×5 dilation of letter "A")

Grayscale dilation: output pixel = **maximum** value in the neighborhood defined by the SE.

Worked step-by-step logic (from slide example, cross-shaped SE): At each foreground pixel position, if SE "hits" (any overlap with foreground) → mark **HIT**, add pixel(s) to output. If no overlap → **MISS**, nothing added at that location (but pixel itself may already be covered from a neighboring position). Over all positions this accumulates a thicker shape.

- Original: 37 pixels → After dilation: 48 pixels → **+11 pixels (~30% increase)** (from worked example)

3.2 Erosion (⊖)

Definition: Shrinks/thins objects in a binary image.

$$g(x,y) = f \ominus s$$

Rule: SE origin placed at (x,y). $g(x,y) = \begin{cases} 1 & \text{if } s \text{ fits in } f \text{ at } (x,y) \\ 0 & \text{otherwise} \end{cases}$

Set-theoretic definition: $A \ominus B = \{z \mid (B)_z \subseteq A\}$ (Translate B to point z; z belongs to erosion only if the ENTIRE B fits inside A.)

Effects / uses:

- Splits apart joined/touching objects
- Strips away small extrusions/spurs on a boundary
- **Watch out: erosion always shrinks/thins objects**

Grayscale erosion: output pixel = **minimum** value in the neighborhood defined by the SE.

Worked example numbers: Original 37 pixels → After erosion: 20 pixels → **-17 pixels (~46% decrease)**

Key symmetry to remember:

	Dilation	Erosion
Logic	HIT (any overlap)	FIT (full containment)
Effect	grow / thicken	shrink / thin
Grayscale equivalent	max filter	min filter
Border padding (see §4)	pad with min value (0)	pad with max value (255 or 1)
Repairs	breaks, intrusions	joins objects apart, removes extrusions

4. Border/Padding Behavior (common exam question)

At image edges, part of the SE neighborhood extends beyond the image – these "virtual" pixels need assigned values:

Operation	Padding rule	Binary	Grayscale
Dilation	pixels beyond border = minimum value the data type allows	0	0
Erosion	pixels beyond border = maximum value the data type allows	1	255

Why this matters conceptually: padding with the "neutral"/worst-case value prevents the operation from artificially growing (dilation) or artificially surviving (erosion) at the border due to fabricated data.

5. Mathematical Notation Table (memorize these)

Operation	Equation	Meaning
-----------	----------	---------

Translation	$(B)_z = \{w \mid w = b+z, b \in B\}$	Moves origin of B to point z
Reflection	$\hat{B} = \{w \mid w = -b, b \in B\}$	Reflects all elements of B about the origin
Complement	$A^c = \{w \mid w \notin A\}$	Points NOT in A
Difference	$A - B = \{w \mid w \in A, w \notin B\} = A \cap B^c$	Points in A but not in B
Dilation	$A \oplus B = \{z \mid (\hat{B})_z \cap A \neq \emptyset\}$	Expands the boundary of A
Erosion	$A \ominus B = \{z \mid (B)_z \subseteq A\}$	Contracts the boundary of A
Opening	$A \circ B = (A \ominus B) \oplus B$	Smooths contours, breaks narrow isthmuses, eliminates small islands/sharp peaks
Closing	$A \bullet B = (A \oplus B) \ominus B$	Smooths contours, fuses narrow breaks/gulfs, eliminates small holes, fills gaps

Note: dilation's definition uses the *reflected* SE ($\hat{B}$), erosion's does not – this reflection distinction is a classic exam trick question.

6. Compound Operations

6.1 Opening ($\circ$)

$$f \circ s = (f \ominus s) \oplus s$$

= **Erosion followed by Dilation** (same SE both times).

Effect:

- Removes small protrusions, thin connections, isolated small objects/noise specks
- Smooths object contours from the *outside* (removes narrow isthmuses/peaks sticking out)
- Object **does not grow back** to original size after erosion+dilation because thin/small parts were eliminated by the erosion step and can't be regenerated by dilation.
- Idempotent: opening an already-opened image gives the same result.

Example takeaway: In the worked example, small noise spots (dots) and a jagged/zigzag connector are removed after opening; leftover shapes are just the "core" solid bodies.

6.2 Closing (●)

$$f \bullet s = (f \oplus s) \ominus s$$

= **Dilation followed by Erosion** (same SE both times).

Effect:

- Fills small holes, gaps, narrow breaks within objects
- Smooths contours from the *inside* (fills narrow gulfs/notches)
- Joins objects that are close together
- Also idempotent.

Quick memory trick:

- **Opening** = **O**ut first (Erosion strips outer bits) → shrink then grow → removes small external bits
- **Closing** = "**C**ombine"/**C**onnect first (Dilation joins/fills) → grow then shrink → fills small internal gaps

6.3 Combined real workflow example (fingerprint slide)

$A \ominus B \rightarrow$ then $A \oplus B$ gives opening $A \circ B$; further $(A \circ B) \oplus B$ then $A \ominus B$ gives closing of the opening $(A \circ B) \bullet B$ – commonly used together (open-then-close) to clean up both noise specks AND small holes/gaps in a segmented image (e.g., fingerprint ridge cleanup).

7. Morphological Algorithms

7.1 Boundary Extraction

$$\beta(A) = A - (A \ominus B)$$

Logic: Erode A to get the "inner core" (shrunk slightly), then subtract that from the original A. What remains is just the **outline/boundary** ring.

- Boundary thickness depends on SE size (bigger SE $\rightarrow$ thicker extracted boundary).
- Very useful for finding object outlines cheaply.

7.2 Region Filling

Goal: Given ONE pixel known to be inside a closed boundary, fill the entire enclosed region with foreground (1s).

Key iterative equation: $X_k = (X_{k-1} \oplus B) \cap A^c, \quad k = 1, 2, 3, \dots$

- X_0 = the single starting seed point inside the boundary
- B = simple SE (e.g., cross)
- A^c = complement of the original boundary image A (so filling never crosses OUTSIDE the boundary line – it clips growth at the boundary each iteration)
- Iterate until $X_k = X_{k-1}$ (no more change/converged)
- **Final step: union the result with the original boundary A** $\rightarrow X_k \cup A$

Why it works: Each iteration dilates the current filled region by one SE step, then the intersection with A^c prevents it from spilling past the boundary pixels (since boundary pixels are NOT in A^c). This grows the fill outward until it hits the walls on all sides.

7.3 Other named algorithms (know they exist, general idea only)

- **Extraction of connected components** – isolate/label separate blobs
- **Thinning / Thickening** – reduce/increase object width iteratively
- **Skeletonization** – reduce all objects to their central "stick figure" / medial-axis lines while preserving essential topology/structure (connectivity, branch points). Based on **Lantuéjoul's formula**. Useful for e.g. reducing circuit board traces or shapes to 1-pixel-wide skeletons for analysis.

- **Pruning** – removes small spurious branches/spurs, often used after skeletonization to clean up artifacts.

8. Quick Comparison Cheat-Sheet

Concept	One-line definition
Fit	entire SE lands on foreground
Hit	any part of SE lands on foreground
Dilation	grow objects (hit-based, uses reflected SE)
Erosion	shrink objects (fit-based)
Opening	erosion→dilation; removes small external protrusions/noise
Closing	dilation→erosion; fills small internal holes/gaps
Boundary extraction	$A - (A \ominus B)$
Region filling	iterative dilation clipped by complement of boundary, seeded from interior point
Skeletonization	reduce shape to topologically-equivalent thin lines

9. Likely Exam Question Types

1. Given a small binary grid + SE, manually compute dilation or erosion output (trace HIT/FIT position by position).
2. Explain why dilation uses reflected B in its set definition but erosion does not.
3. State/derive opening and closing formulas from erosion+dilation.
4. Given an image with noise specks + small holes, decide: apply opening first or closing first, and why.
5. Explain padding rules at borders for dilation vs erosion and justify why (avoid artificial growth/survival).

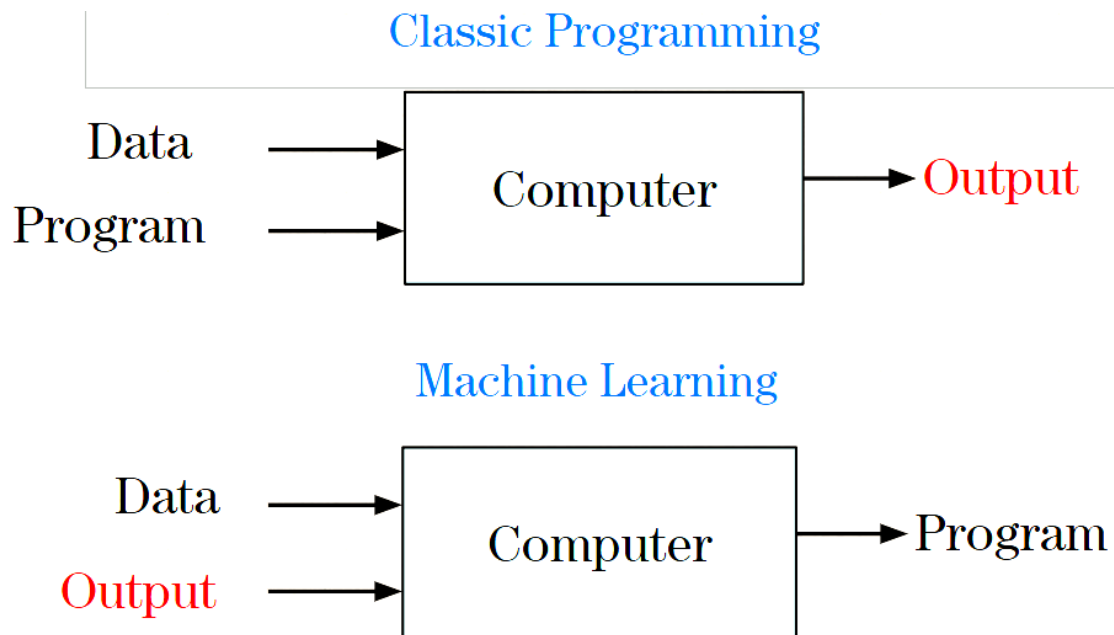
6. Region filling: given seed point + boundary, walk through 2–3 iterations of X_k equation.
7. Boundary extraction: compute $A - (A \ominus B)$ for a small grid.
8. Conceptual: why does opening not restore original shape after dilation step (irreversibility argument — erosion is not invertible; thin parts removed can't come back).

Machine Learning

Machine Learning

What is machine learning (ML)?

- Many AI researchers consider the ultimate goal of AI can be achieved by imitating the way humans learn
- **Machine Learning** – is the scientific study of algorithms and statistical models that computer systems use to learn from observations, without being explicitly programmed
- In this context, **learning** refers to:
 - recognizing complex patterns in data
 - making intelligent decisions based on data observations



"A computer program is said to learn from experience **E** with respect to some class of tasks **T** and performance measure **P**, if its performance at tasks in **T**, as measured by **P**, improves with experience **E**." – Tom Mitchell

(*) implies a certain degree of accuracy

- **Every machine learning algorithm has three components:**
 - **Representation / Model Class**
 - **Evaluation / Objective Function**
 - **Optimization**
- **Standard learning paradigms:**
 - Supervised learning
 - Unsupervised learning
 - Semi-supervised learning
 - Reinforcement learning
- **Non-standard paradigms:**
 - Active learning
 - Transfer learning
 - Transductive learning

Supervised Learning Models -

- Naive Bayes
- k-Nearest Neighbors
- Decision trees and random forests
- Support Vector Machines
- Kernel methods
- Kernel Ridge Regression
- Neural networks
- Many others...

Unsupervised Learning Models -

- K-means clustering
- DBScan
- Hierarchical clustering
- Principal Component Analysis
- t-Distributed Stochastic Neighbor Embedding
- Hidden Markov Models

The correlation coefficient is a measure of the strength and the direction of a linear relationship between two variables. The symbol r represents the sample correlation coefficient. The formula for r is

$$r = \frac{n \sum xy - (\sum x)(\sum y)}{\sqrt{n \sum x^2 - (\sum x)^2} \sqrt{n \sum y^2 - (\sum y)^2}}.$$

The range of the correlation coefficient is -1 to 1 . If x and y have a strong positive linear correlation, r is close to 1 . If x and y have a strong negative linear correlation, r is close to -1 . If there is no linear correlation or a weak linear correlation, r is close to 0 .

A **regression line**, also called a **line of best fit**, is the line for which the sum of the squares of the residuals is a minimum.

The Equation of a Regression Line

The equation of a regression line for an independent variable x and a dependent variable y is

$$\hat{y} = mx + b$$

where $\hat{y}$ is the predicted y -value for a given x -value. The slope m and y -intercept b are given by

$$m = \frac{n \sum xy - (\sum x)(\sum y)}{n \sum x^2 - (\sum x)^2} \text{ and } b = \bar{y} - m\bar{x} = \frac{\sum y}{n} - m \frac{\sum x}{n}$$

where $\bar{y}$ is the mean of the y -values and $\bar{x}$ is the mean of the x -values. The regression line always passes through $(\bar{x}, \bar{y})$.

Means: $\bar{X}$, $\bar{Y}$

Sums of products (least-squares formulas):

$$b = \frac{\sum (X - \bar{X})(Y - \bar{Y})}{\sum (X - \bar{X})^2}, \quad a = \bar{Y} - b \cdot \bar{X}$$

Linear Regression with Linear Algebra

$$y = m \cdot x + c$$

$$m \cdot x_1 + c = y_1$$

$$m \cdot x_2 + c = y_2$$

$$\vdots$$

$$m \cdot x_n + c = y_n$$

$$\begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

$$\mathbf{A} \cdot \mathbf{b} = \mathbf{y}$$

$$(\mathbf{A}^T \mathbf{A}) \mathbf{b} = \mathbf{A}^T \mathbf{y}$$

$$(\mathbf{A}^T \mathbf{A})^{-1} (\mathbf{A}^T \mathbf{A}) \mathbf{b} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{y}$$

$$\mathbf{b} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{y}$$

$$\mathbf{A} = [\mathbf{x} \quad \mathbf{1}] \quad \mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} m \\ c \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

$$\mathbf{A}^T \mathbf{A} = \begin{bmatrix} \sum x^2 & \sum x \\ \sum x & n \end{bmatrix}$$

$$\mathbf{A}^T \mathbf{y} = \begin{bmatrix} \sum xy \\ \sum y \end{bmatrix}$$

What is p-value

Prob of seeing result this extreme (or more), assuming null hypothesis true (i.e. assuming "nothing special happening," just random chance).

Why need it

Data always show *some* pattern, even from pure randomness. p-value test: is pattern real effect, or just noise? Low p-value → unlikely due to chance alone → reject null hypothesis.

Null hypothesis (H0): default assumption, e.g. "coin fair," "no difference between groups," "drug no effect."

Scenario 1: Coin flip

H0: coin fair (p=0.5 heads).

Flip twice, observe 2 heads.

p-value = P(this result) + P(equally extreme) + P(more extreme)

= 0.25 (HH) + 0.25 (TT, equally extreme other side) + 0 (nothing more extreme)

= 0.5

High p-value (0.5) → 2 heads totally plausible under fair coin → don't reject H0.

If you flipped 10 times, got 10 heads:

P(10 heads | fair coin) = $0.5^{10} \approx 0.001$

p-value tiny → strong evidence against "fair coin" H0.

Slide example

Where values come from

- y_i = actual observed Y (house price) for house i — straight from your data table.
 - $\hat{y}_i$ ("y-hat") = predicted Y from regression equation, plug in that house's X.
 $\hat{y}_i = 98.24833 + 0.10977 \times X^T$
 - $\bar{y}$ = mean of all observed Y values (average house price across sample).
-

Worked example with your data

House	X (sq ft)	y (actual)	$\hat{y} = 98.248 + 0.10977X$	$y - \bar{y}$	$\hat{y} - \bar{y}$	$y - \hat{y}$
1	1400	245	251.93	-41.5	-34.57	-6.93
2	1600	312	273.88	25.5	-12.62	38.12
3	1700	279	284.86	-7.5	-1.64	-5.86
4	1875	308	304.06	21.5	17.56	3.94
5	1100	199	219.00	-87.5	-67.5	-20.00
6	1550	219	268.39	-67.5	-18.11	-49.39
7	2350	405	356.21	118.5	69.71	48.79
8	2450	324	367.19	37.5	80.69	-43.19
9	1425	319	254.67	32.5	-31.83	64.33
10	1700	255	284.86	-31.5	-1.64	-29.86

$\bar{y} = 286.5$ (average of the 10 house prices — sum/10)

Now the sums of squares

SST (Total variation) = $\sum (y_i - \bar{y})^2$

→ how far each *actual* price is from overall average, squared, summed.

= $(-41.5)^2 + (25.5)^2 + \dots = 32600.5$ (matches ANOVA table Total row)

SSR (Regression/explained variation) = $\sum(\hat{y}_i - \bar{y})^2$

→ how far each *predicted* price (from the line) is from overall average, squared, summed. This is the part your model explains using X.

= 18934.9348 (matches ANOVA Regression row)

SSE (Error/residual variation) = $\sum(y_i - \hat{y}_i)^2$

→ how far actual price is from *predicted* price — leftover error the model missed.

= 13665.5652 (matches ANOVA Residual row)

Check: SST = SSR + SSE → 18934.93 + 13665.57 = 32600.5 ✓

Degrees of freedom (df)

- Regression df = k = number of predictors = 1 (just square feet)
- Residual df = n - k - 1 = 10 - 1 - 1 = 8
- Total df = n - 1 = 9

MS (Mean Square) = SS ÷ df

- MSR = SSR/k = 18934.9348/1 = 18934.9348
- MSE = SSE/(n-k-1) = 13665.5652/8 = 1708.1957

F statistic

F = MSR/MSE = 18934.9348/1708.1957 = 11.0848

This F tests: "does the regression as a whole explain significant variation?" Compared against F-distribution with df(1,8) → gives Significance F = p-value = 0.01039.

Standard Error of regression = $\sqrt{\text{MSE}} = \sqrt{1708.1957} = 41.33$ (matches "Standard Error" in Excel output — average size of residual/error).

R² = SSR/SST = 18934.9348/32600.5 = 0.58082 (58% of variation in price explained by sq ft; rest is SSE, unexplained).

Logistic Regression

Used when Y is binary (0/1, e.g. default/no default). Instead of predicting Y directly (unbounded line doesn't fit 0-1 range), predicts **log-odds** of Y=1 as linear function of X:

$$\log[P/(1-P)] = a + bX$$

Model outputs probability via sigmoid, always between 0 and 1 — unlike linear regression which can predict outside 0-1.

Odds

Odds = $P/(1-P)$ — ratio of "event happens" to "event doesn't happen." E.g. $P=0.75 \rightarrow$ odds = $0.75/0.25 = 3$ (event 3× more likely than not).

Different from probability: probability capped 0-1, odds range 0 to ∞ .

Logistic model says **log(odds)** is linear in X. So each 1-unit increase in X multiplies odds by e^b (not adds — multiplicative effect).

Given: $\log[P/(1-P)] = -4 + 1.2X$

(i) Odds & probability at $X=3$

$$\text{logit} = -4 + 1.2(3) = -4 + 3.6 = -0.4$$

$$\text{Odds} = e^{(-0.4)} = \mathbf{0.6703}$$

$$P = \text{odds}/(1+\text{odds}) = 0.6703/1.6703 = \mathbf{0.4013} \text{ } (\approx 40.1\% \text{ default probability})$$

(ii) X for $P = 0.5$

At $P=0.5$, odds = $0.5/0.5 = 1$, so $\log(\text{odds}) = \log(1) = 0$

$$0 = -4 + 1.2X$$

$$1.2X = 4$$

$$X = \mathbf{3.333}$$

(iii) Change in odds per unit increase in X

Coefficient $b = 1.2 \rightarrow$ each +1 unit in X multiplies odds by e^b :

$$e^{1.2} = \mathbf{3.3201}$$

So odds increase ~232% (multiply by 3.32) for every 1-unit rise in X — not additive, multiplicative.

- b) Suppose data is collected for a group of students in a Software Analytics class with variables X_1 = "hours studied", X_2 = "undergrad GPA", and Y = "receives an A". We fit a "logistic regression" and produce estimated coefficient, $\beta_0 = -5$, $\beta_1 = 0.1$, $\beta_2 = 1$. 5
- Estimate the probability of getting an A for a student in the class who studies for 40 hours and has an undergrad GPA of 3.7.
 - How many hours would the student need to study to have a 70% chance of getting an A in the class?

Regression Mathematics with Multiple Variables

When working with multiple independent variables ($X_1, X_2, \dots, X_p$), regression models combine them into a single linear combination score, often called z or log-odds:

$$z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p$$

- β_0 : Intercept (base value when all $X_i = 0$)
- $\beta_1, \beta_2, \dots, \beta_p$: Coefficients representing the weight or effect of each predictor variable X_i .

Multiple Logistic Regression

When predicting binary outcomes (like passing/failing or receiving an A/not receiving an A), linear regression cannot be used directly because its output can be any real number, whereas probabilities must stay between 0 and 1.

Logistic regression solves this by passing z through the **sigmoid function**:

$$p(X) = \sigma(z) = \frac{1}{1+e^{-z}} = \frac{e^z}{1+e^z}$$

To solve for an input variable X_k when a desired probability p is given, take the natural logarithm ($\ln$) to invert the logistic equation into the **logit equation**:

$$\ln\left(\frac{p}{1-p}\right) = z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p$$

Step-by-Step Solution to the Example Problem

Given Parameters

- Independent Variables:
 - X_1 = hours studied
 - X_2 = undergrad GPA
- Dependent Variable: Y = receives an A
- Estimated Coefficients:
 - $\beta_0 = -5$
 - $\beta_1 = 0.1$
 - $\beta_2 = 1.0$

Part 1: Estimate the probability of getting an A for $X_1 = 40$ hours and $X_2 = 3.7$ GPA

Step 1: Compute the linear combination score (z)

$$\begin{aligned} z &= \beta_0 + \beta_1 X_1 + \beta_2 X_2 \\ z &= -5 + (0.1 \times 40) + (1.0 \times 3.7) \\ z &= -5 + 4 + 3.7 = 2.7 \end{aligned}$$

Step 2: Convert z to probability $p(X)$ via the sigmoid function

$$\begin{aligned} p &= \frac{1}{1+e^{-z}} \\ p &= \frac{1}{1+e^{-2.7}} \end{aligned}$$

$$p = \frac{1}{1+0.0672055} \approx \frac{1}{1.0672055} \approx \mathbf{0.9370} \quad (\text{or } \mathbf{93.70\%})$$

Part 2: How many hours (X_1) does a student with $X_2 = 3.7$ GPA need to study to have a 70% chance ($p = 0.70$) of getting an A?

Step 1: Calculate the log-odds (z) for $p = 0.70$

$$odds = \frac{p}{1-p} = \frac{0.70}{1-0.70} = \frac{0.70}{0.30} \approx 2.3333$$

$$z = \ln(2.3333) \approx 0.8473$$

Step 2: Set up the linear equation and solve for X_1

$$\begin{aligned} z &= \beta_0 + \beta_1 X_1 + \beta_2 X_2 \\ 0.8473 &= -5 + 0.1(X_1) + 1.0(3.7) \\ 0.8473 &= -5 + 0.1(X_1) + 3.7 \\ 0.8473 &= -1.3 + 0.1(X_1) \\ 0.8473 + 1.3 &= 0.1(X_1) \\ 2.1473 &= 0.1(X_1) \end{aligned}$$

$$X_1 = \frac{2.1473}{0.1} = 21.473 \text{ hours}$$

Final Verdict:

1. Probability of getting an A with 40 hours studied and a 3.7 GPA is **\$93.70\%**.
2. A student with a 3.7 GPA needs to study approximately 21.47 **hours** to have a 70% chance of getting an A.

Gradient Descent

Iterative optimization algorithm to find parameter values that minimize a cost function (e.g., minimize error between predictions and actual data). Instead of solving directly (closed-form), takes small steps downhill on the cost curve until reaching minimum.

Core idea

At any point, gradient (derivative) tells slope direction. Move opposite to gradient (downhill) to reduce cost. Step size controlled by **learning rate (α)**.

Update rule:

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \cdot (d\text{Cost}/d\theta)$$

Example: minimize simple function

Cost function: $f(x) = (x - 3)^2$

(Imagine this is a simplified error function; minimum at $x=3$, $f=0$)

Derivative: $f'(x) = 2(x-3)$

Start: $x_0 = 0$, learning rate $\alpha = 0.1$

Iteration 1:

$$f'(0) = 2(0-3) = -6$$

$$x_1 = x_0 - \alpha \cdot f'(x_0) = 0 - 0.1(-6) = 0 + 0.6 = 0.6$$

Iteration 2:

$$f'(0.6) = 2(0.6-3) = -4.8$$

$$x_2 = 0.6 - 0.1(-4.8) = 0.6 + 0.48 = 1.08$$

Iteration 3:

$$f'(1.08) = 2(1.08-3) = -3.84$$

$$x_3 = 1.08 - 0.1(-3.84) = 1.08 + 0.384 = 1.464$$

Iteration 4:

$$f'(1.464) = 2(1.464-3) = -3.072$$

$$x_4 = 1.464 + 0.3072 = 1.7712$$

... continuing this, x slowly converges toward 3 (the true minimum), each step getting smaller since gradient shrinks as x approaches 3.

Applied to actual regression (real use case)

For linear regression $Y = b_0 + b_1 \cdot X$, cost function (MSE):

$$J(b_0, b_1) = (1/n) \sum (y_i - (b_0 + b_1 x_i))^2$$

Gradients:

$$\partial J / \partial b_0 = -(2/n) \sum (y_i - \hat{y}_i)$$

$$\partial J / \partial b_1 = -(2/n) \sum (y_i - \hat{y}_i) \cdot x_i$$

Update both simultaneously each iteration:

$$b_0 = b_0 - \alpha \cdot \partial J / \partial b_0$$

$$b_1 = b_1 - \alpha \cdot \partial J / \partial b_1$$

Repeat until gradients ≈ 0 (cost stops decreasing) — that's convergence, b_0/b_1 settle near OLS solution.

Why use it over OLS closed-form?

OLS needs matrix inversion $(X^T X)^{-1}$ — expensive/impossible for huge datasets or many features. Gradient descent scales better, works for models without closed-form solution (logistic regression, neural nets).

Key hyperparameter: learning rate α

- Too small $\rightarrow$ converges very slowly
- Too large $\rightarrow$ overshoots, may diverge (never reach minimum)

Decision Tree — ID3 Algorithm (Entropy & Information Gain)

1. What is ID3?

ID3 (Iterative Dichotomiser 3) builds a decision tree by picking, at every node, the attribute that gives the **highest Information Gain (IG)**. IG is computed from **Entropy** (a measure of impurity/disorder in the data).

- Entropy = 0 $\rightarrow$ set is pure (all one class)
- Entropy = 1 $\rightarrow$ set is maximally mixed (50/50 for binary class)

2. Dataset — "Play Tennis?"

14 days of weather data, target = whether tennis was played (Yes/No).

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes

D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Class distribution: **9 Yes, 5 No** (out of 14).

3. Step 1 — Entropy of the whole dataset

Formula:

$$\text{Entropy}(S) = -\sum p(i) * \log_2(p(i))$$

$$p(\text{Yes}) = 9/14 = 0.643$$

$$p(\text{No}) = 5/14 = 0.357$$

$$\begin{aligned}
 \text{Entropy}(S) &= -(0.643 * \log_2(0.643)) - (0.357 * \log_2(0.357)) \\
 &= -(0.643 * -0.637) - (0.357 * -1.485) \\
 &= 0.409 + 0.530 \\
 &= 0.940
 \end{aligned}$$

Entropy(S) $\approx$ 0.940 — quite mixed, close to max (1.0).

4. Step 2 — Entropy + Information Gain for each attribute

General IG formula:

$$\text{IG}(S, A) = \text{Entropy}(S) - \sum (|S_v| / |S|) * \text{Entropy}(S_v)$$

where S_v = subset of S where attribute A has value v.

4a. Attribute: Outlook

Value	Count	Yes	No	Entropy
Sunny	5	2	3	see below
Overcast	4	4	0	0
Rain	5	3	2	see below

$$\text{Entropy}(\text{Sunny}) = -(2/5)\log_2(2/5) - (3/5)\log_2(3/5) = 0.971$$

$$\text{Entropy}(\text{Overcast}) = 0 \quad (\text{pure — all Yes})$$

$$\text{Entropy}(\text{Rain}) = -(3/5)\log_2(3/5) - (2/5)\log_2(2/5) = 0.971$$

$$\begin{aligned} \text{Weighted entropy} &= (5/14)*0.971 + (4/14)*0 + (5/14)*0.971 \\ &= 0.347 + 0 + 0.347 \\ &= 0.694 \end{aligned}$$

$$\text{IG}(\text{Outlook}) = 0.940 - 0.694 = 0.246$$

4b. Attribute: Humidity

Value	Count	Yes	No	Entropy
High	7	3	4	0.985
Normal	7	6	1	0.592

$$\text{Entropy}(\text{High}) = -(3/7)\log_2(3/7) - (4/7)\log_2(4/7) = 0.985$$

$$\text{Entropy}(\text{Normal}) = -(6/7)\log_2(6/7) - (1/7)\log_2(1/7) = 0.592$$

$$\begin{aligned} \text{Weighted entropy} &= (7/14)*0.985 + (7/14)*0.592 \\ &= 0.492 + 0.296 \\ &= 0.788 \end{aligned}$$

$$\text{IG}(\text{Humidity}) = 0.940 - 0.788 = 0.152$$

4c. Attribute: Wind

Value	Count	Yes	No	Entropy
Weak	8	6	2	0.811
Strong	6	3	3	1.000

$$\text{Entropy}(\text{Weak}) = -(6/8)\log_2(6/8) - (2/8)\log_2(2/8) = 0.811$$

$$\text{Entropy}(\text{Strong}) = -(3/6)\log_2(3/6) - (3/6)\log_2(3/6) = 1.000$$

$$\begin{aligned}\text{Weighted entropy} &= (8/14)*0.811 + (6/14)*1.000 \\ &= 0.463 + 0.429 \\ &= 0.892\end{aligned}$$

$$\text{IG}(\text{Wind}) = 0.940 - 0.892 = 0.048$$

4d. Attribute: Temperature

Value	Count	Yes	No	Entropy
Hot	4	2	2	1.000
Mild	6	4	2	0.918
Cool	4	3	1	0.811

$$\begin{aligned}\text{Weighted entropy} &= (4/14)*1.000 + (6/14)*0.918 + (4/14)*0.811 \\ &= 0.286 + 0.393 + 0.232 \\ &= 0.911\end{aligned}$$

$$\text{IG}(\text{Temperature}) = 0.940 - 0.911 = 0.029$$

5. Step 3 — Compare all IG values, pick the winner

$$\text{IG}(\text{Outlook}) = 0.246 \leftarrow \text{highest}$$

$$\text{IG}(\text{Humidity}) = 0.152$$

$$\text{IG}(\text{Wind}) = 0.048$$

$$\text{IG}(\text{Temperature}) = 0.029$$

Outlook wins → becomes the root node.

6. Step 4 — Split by Outlook, recurse on impure branches

Root: Outlook?

- Overcast → Entropy = 0 → PURE → LEAF: "Yes"
- Sunny → Entropy = 0.971 → impure → recurse
- Rain → Entropy = 0.971 → impure → recurse

Recurse on "Sunny" branch (5 rows: D1,D2,D8,D9,D11)

Day	Temperature	Humidity	Wind	PlayTennis
D1	Hot	High	Weak	No
D2	Hot	High	Strong	No
D8	Mild	High	Weak	No
D9	Cool	Normal	Weak	Yes
D11	Mild	Normal	Strong	Yes

Base entropy of this subset (3 No, 2 Yes):

Entropy(Sunny) = 0.971 (already computed above)

Check Humidity here:

High (D1,D2,D8): 0 Yes, 3 No $\rightarrow$ Entropy = 0

Normal (D9,D11): 2 Yes, 0 No $\rightarrow$ Entropy = 0

Weighted entropy = $(3/5)*0 + (2/5)*0 = 0$

IG(Humidity | Sunny) = 0.971 - 0 = 0.971 $\leftarrow$ perfect split!

Humidity perfectly separates the Sunny branch. No need to check Wind/Temp — Humidity gives IG = full entropy, the max possible.

Sunny branch:

```

└── Humidity=High  → LEAF: "No" (pure)
└── Humidity=Normal → LEAF: "Yes" (pure)

```

Recurse on "Rain" branch (5 rows: D4,D5,D6,D10,D14)

Day	Temperature	Humidity	Wind	PlayTennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
D10	Mild	Normal	Weak	Yes
D14	Mild	High	Strong	No

Base entropy (3 Yes, 2 No) = 0.971 (already computed above).

Check Wind here:

Weak (D4,D5,D10): 3 Yes, 0 No $\rightarrow$ Entropy = 0

Strong (D6,D14): 0 Yes, 2 No $\rightarrow$ Entropy = 0

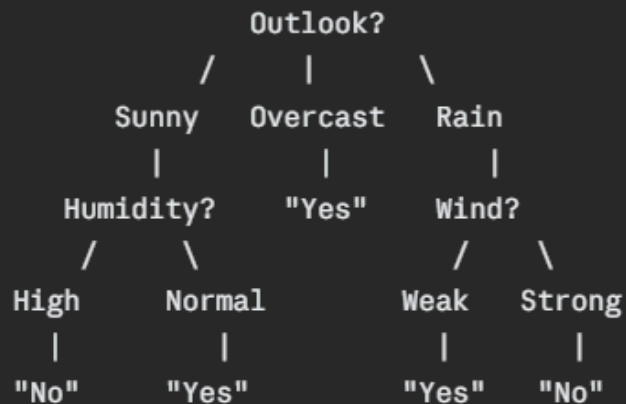
Weighted entropy = 0

$IG(\text{Wind} \mid \text{Rain}) = 0.971 - 0 = 0.971 \leftarrow$ perfect split!

Rain branch:

- └ Wind=Weak $\rightarrow$ LEAF: "Yes" (pure)
- └ Wind=Strong $\rightarrow$ LEAF: "No" (pure)

7. Final Tree



8. ID3 Algorithm — General Pseudocode

function ID3(data, attributes):

 if all rows same class:

 return leaf(that class)

 if attributes is empty:

 return leaf(majority class in data)

 for each attribute A in attributes:

 compute Entropy for each value-subset of A

 compute $IG(\text{data}, A) = \text{Entropy}(\text{data}) - \text{weighted_avg}(\text{Entropy}(\text{subsets}))$

 best = attribute with MAX IG

```

create node(best)
remove best from attributes list

for each value v of best:
    subset = rows where best == v
    if subset is empty:
        attach leaf(majority class in data)
    else:
        attach ID3(subset, attributes) # recurse

return node

```

9. Key Takeaways

- **Entropy** measures disorder in a set (0 = pure, 1 = max mixed for binary).
- **Information Gain** = how much entropy drops after splitting on an attribute.
- ID3 always picks the attribute with **highest IG** at each node.
- Recursion stops when a branch is **pure** (entropy = 0) or **no attributes left**.
- Same core logic as Gini-based CART trees — only the impurity formula differs (log-based entropy vs squared-probability Gini).

K-means clustering

Analogy first

K-means = grouping people at party by where they standing. Pick K "meeting points" (centroids). Everyone join nearest meeting point. Meeting point moves to center of its group. Repeat till nobody switches groups.

Example: 6 points, K=2

Point	X	Y
A	1	1
B	1.5	2
C	3	4

D	5	7
E	3.5	5
F	4.5	5

Step 1: Pick initial centroids (random, or first K points)

$C1 = A = (1, 1)$

$C2 = D = (5, 7)$

Step 2: Assign each point to nearest centroid (Euclidean distance)

distance = $\sqrt{(x1-x2)^2 + (y1-y2)^2}$

Point	Dist to C1(1,1)	Dist to C2(5,7)	Assign
A(1,1)	0	7.21	C1
B(1.5,2)	1.12	6.10	C1
C(3,4)	3.61	3.61	C1 (tie→pick C1)
D(5,7)	7.21	0	C2
E(3.5,5)	4.72	2.5	C2
F(4.5,5)	5.32	2.06	C2

Groups: $C1 = \{A,B,C\}$, $C2 = \{D,E,F\}$

Step 3: Recalculate centroids = mean of each group

new $C1 = \text{mean}(A,B,C) = ((1+1.5+3)/3, (1+2+4)/3) = (1.83, 2.33)$

new $C2 = \text{mean}(D,E,F) = ((5+3.5+4.5)/3, (7+5+5)/3) = (4.33, 5.67)$

Step 4: Re-assign with new centroids

Point	Dist to C1(1.83,2.33)	Dist to C2(4.33,5.67)	Assign
A	1.57	5.83	C1
B	0.42	4.68	C1
C	2.14	2.14	tie→C1

D	6.11	1.79	C2
E	3.07	0.97	C2
F	3.65	0.68	C2

Groups unchanged: {A,B,C} and {D,E,F} → centroids stop moving → **converged**.

Verify with code

Verify k-means result with sklearn

Verify k-means result with sklearn

Matches hand calc exactly: labels [0 , 0 , 0 , 1 , 1 , 1] = {A,B,C} and {D,E,F}, centroids (1.83, 2.33) and (4.33, 5.67).

Algorithm summary

1. Pick K initial centroids (random or first K points)
2. Assign each point to nearest centroid (Euclidean distance)
3. Recompute centroid = mean of points in that group
4. Repeat steps 2-3 until assignments stop changing (converged)

Note: result depends on initial centroid choice — bad start can converge to worse local optimum. Real-world fix: run multiple times with different starts (`n_init` param), keep best result (lowest total within-cluster distance).

Naive Bayes

Analogy first

Naive Bayes = doctor guessing disease from symptoms. Doesn't check symptom combos together (that's "naive" part) — treats each symptom as independent clue, multiplies probabilities, picks most likely disease.

Formula (Bayes' Theorem)

$$P(\text{Class} | \text{Features}) = P(\text{Class}) * P(\text{Feature1}|\text{Class}) * P(\text{Feature2}|\text{Class}) * \dots / P(\text{Features})$$

Denominator same for all classes → ignore it, just compare numerators, pick biggest.

Example: same PlayTennis dataset (14 rows)

Day	Outlook	Temperat	Humidity	Wind	Play
-----	---------	----------	----------	------	------

		ure			
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

New day to predict: Outlook=Sunny, Temperature=Cool, Humidity=High, Wind=Strong
→ Play?

Step 1: Prior probabilities

$$P(\text{Yes}) = 9/14 = 0.643$$

$$P(\text{No}) = 5/14 = 0.357$$

Step 2: Likelihood — P(each feature | class)

Outlook=Sunny:

$$P(\text{Sunny}|\text{Yes}) = 2/9 = 0.222 \quad (\text{Sunny appears 2 times in 9 Yes rows: D9,D11})$$

$$P(\text{Sunny}|\text{No}) = 3/5 = 0.600 \quad (3 \text{ times in 5 No rows: D1,D2,D8})$$

Temperature=Cool:

$$P(\text{Cool}|\text{Yes}) = 3/9 = 0.333 \quad (\text{D5,D7,D9})$$

$$P(\text{Cool}|\text{No}) = 1/5 = 0.200 \quad (\text{D6})$$

Humidity=High:

$$P(\text{High}|\text{Yes}) = 3/9 = 0.333 \quad (\text{D3,D4,D12})$$

$$P(\text{High}|\text{No}) = 4/5 = 0.800 \quad (\text{D1,D2,D8,D14})$$

Wind=Strong:

$$P(\text{Strong}|\text{Yes}) = 3/9 = 0.333 \quad (\text{D7,D11,D12})$$

$$P(\text{Strong}|\text{No}) = 3/5 = 0.600 \quad (\text{D2,D6,D14})$$

Step 3: Multiply everything (naive independence assumption)

$$\begin{aligned} \text{Score}(\text{Yes}) &= P(\text{Yes}) * P(\text{Sunny}|\text{Yes}) * P(\text{Cool}|\text{Yes}) * P(\text{High}|\text{Yes}) * P(\text{Strong}|\text{Yes}) \\ &= 0.643 * 0.222 * 0.333 * 0.333 * 0.333 \\ &= 0.643 * 0.222 * 0.333 * 0.333 * 0.333 \\ &\approx 0.00529 \end{aligned}$$

$$\begin{aligned} \text{Score}(\text{No}) &= P(\text{No}) * P(\text{Sunny}|\text{No}) * P(\text{Cool}|\text{No}) * P(\text{High}|\text{No}) * P(\text{Strong}|\text{No}) \\ &= 0.357 * 0.600 * 0.200 * 0.800 * 0.600 \\ &\approx 0.02057 \end{aligned}$$

Step 4: Normalize (optional, for actual probability %) and compare

$$\text{Total} = 0.00529 + 0.02057 = 0.02586$$

$$P(\text{Yes}|\text{features}) = 0.00529 / 0.02586 \approx 0.205 \quad (20.5\%)$$

$$P(\text{No}|\text{features}) = 0.02057 / 0.02586 \approx 0.795 \quad (79.5\%)$$

Score(No) > Score(Yes) → predict "No" (don't play tennis).

Makes sense: Sunny+High Humidity+Strong Wind historically favors No in the data.

Decision Tree vs Naive Bias

Decision Tree

Pros:

- Handle non-linear, complex boundaries well
- No independence assumption on features
- Interpretable (visual, follow path)
- Handle mixed data types (categorical + numeric) easy
- No need feature scaling

Cons:

- Overfit easy, high variance
- Small data change $\rightarrow$ different tree (unstable)
- Weak on small datasets, many features
- Struggle with correlated/redundant features (still ok, but bias toward biased splits)

Naive Bayes

Pros:

- Fast train + predict, low compute
- Work well small dataset
- Handle high-dimensional data well (text, spam filter classic use)
- Less overfit, low variance

Cons:

- Independence assumption often false $\rightarrow$ hurt accuracy if features correlated
- Bad continuous feature modeling unless distribution assumed correct (Gaussian NB assumes normal)
- Can't capture feature interactions
- Zero-frequency problem (need smoothing)

Which better, which case:

- Text classification, spam detection, high-dim sparse data $\rightarrow$ Naive Bayes better (fast, works despite naive assumption)
- Small dataset, need quick baseline $\rightarrow$ Naive Bayes
- Structured tabular data, need interpretability, non-linear relations $\rightarrow$ Decision Tree
- Features highly correlated $\rightarrow$ avoid pure Naive Bayes, Decision Tree handle better
- Need robust production model $\rightarrow$ neither alone ideal, Decision Tree usually ensembled (Random Forest/GBM) beats standalone NB in most tabular tasks

Neural Networks

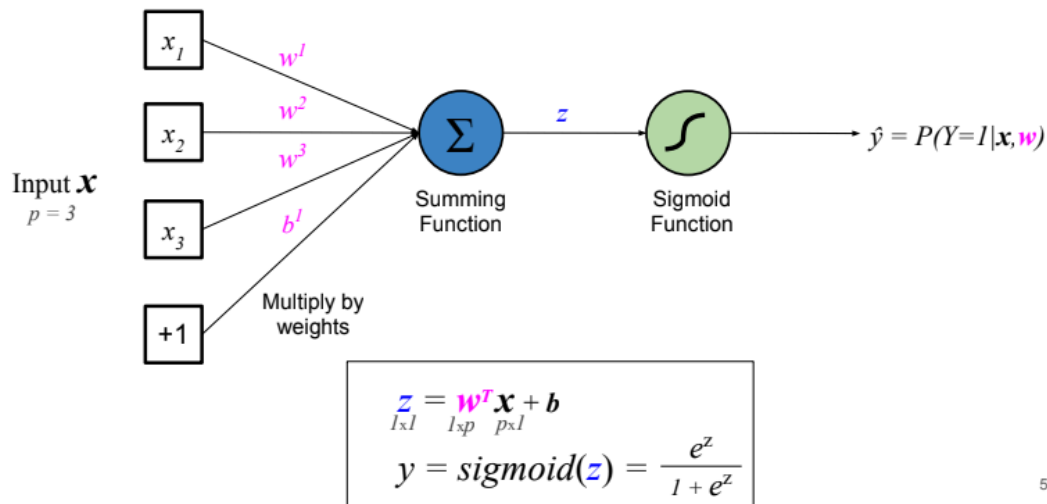
1. Intro: What is Neural Network

NN = stack of differentiable, parameterized layers. Goal: learn function mapping input $x \rightarrow$ output $\hat{y}$ that approx true target t .

Single neuron (logistic regression view):

$$\begin{matrix} x_1 \\ x_2 \\ x_3 \end{matrix} \left. \vphantom{\begin{matrix} x_1 \\ x_2 \\ x_3 \end{matrix}} \right\} \rightarrow \Sigma(w \cdot x + b) = z \rightarrow \text{sigmoid}(z) = \hat{y}$$

Expanded Logistic Regression



5

- Input: x (vector, size p)
- Weights: w (size p), Bias: b
- Linear step: $z = w^T x + b$
- Nonlinearity (activation): $\hat{y} = \sigma(z)$

Sigmoid (logistic) function:

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

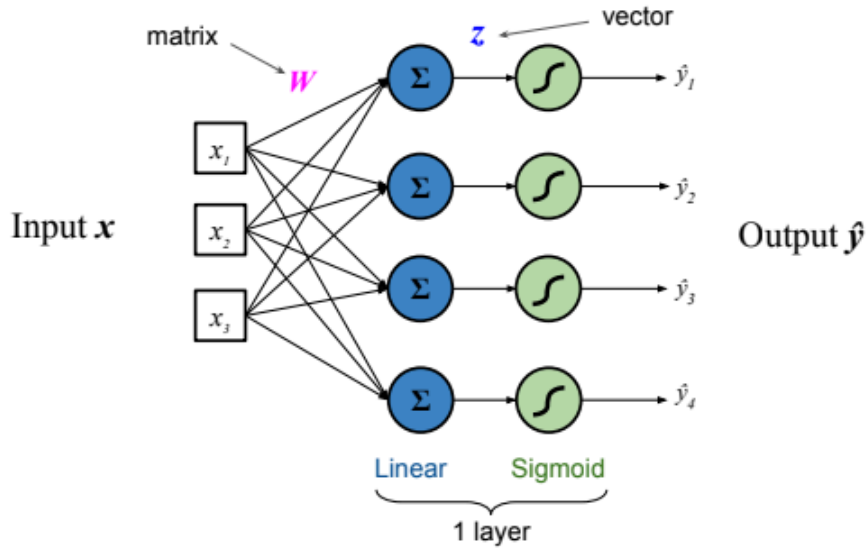
Squashes any real number $\rightarrow (0,1)$. Used for binary prob output $P(Y=1 | x)$.

Biological analogy: dendrites (inputs) $\rightarrow$ cell body (weighted sum + bias) $\rightarrow$ axon (output through activation fn). Real neuron fires (activates) only past threshold — sigmoid is smooth version of that step.

One neuron = one logistic regression unit. Stack many neurons in a layer, stack many layers $\rightarrow$ Neural Network / Multi-Layer Perceptron (MLP).

1-Layer NN (multiple neurons, one layer)

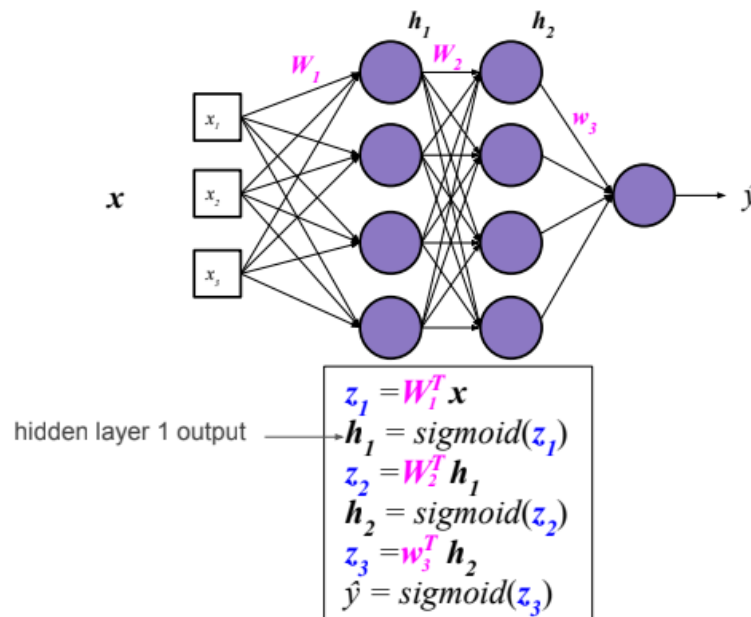
1-Layer Neural Network (with 4 neurons)



$z = W^T x$, $\hat{y} = \sigma(z)$ applied elementwise. Each of d neurons has own row of weights in W .

Multi-Layer NN (MLP)

Multi-Layer Neural Network (MLP)



22

- "Deep" NN = more than 1 hidden layer.
- Each layer = linear transform (matrix multiply + bias) + nonlinearity.

- Layer count, neurons per layer = architecture choices (hyperparameters).

Loss Functions (how wrong is $\hat{y}$)

Task	Loss	Formula
Regression	Squared error	$EE = \frac{1}{2}(t - \hat{y})^2 = \frac{1}{2}(t - \hat{y})^2$
Binary classification	Binary cross-entropy	$E = -[t \cdot \log(\hat{y}) + (1-t) \cdot \log(1-\hat{y})]$
Multi-class	Cross-entropy + softmax	$E = -\sum y_i \cdot \ln(\hat{y}_i), \hat{y}_i = e^{z_i} / \sum e^{z_i}$

Softmax normalizes output layer into valid probability distribution over K classes.

Training loop (Gradient Descent + Backprop)

1. Initialize weights (usually random small values)
2. For each batch:
 - a. Forward pass: compute $\hat{y}$ and loss E
 - b. Backward pass: compute $\partial E / \partial W$ for all weights (backpropagation)
 - c. Update: $W(t+1) = W(t) - \eta \cdot \partial E / \partial W(t)$
3. Repeat until loss converges

η = learning rate. This is Stochastic Gradient Descent (SGD); many better variants exist (Momentum, Adam, RMSProp — see "what next" section).

2. Forward Pass — worked example

Use the handwritten-note architecture: 2 inputs $\rightarrow$ 1 hidden layer (2 neurons) $\rightarrow$ 1 output neuron, sigmoid everywhere.

Notation (matches note):

- Input: x , Target: t
- Hidden weights: w_{xh} , hidden bias: b_h
- Output weights: w_{ho} , output bias: b_o
- Learning rate: η

Forward equations:

$$\begin{aligned}
 \text{Hidden: } z_h &= w_{xh} \cdot x + b_h & h &= \sigma(z_h) \\
 \text{Output: } z_o &= w_{ho} \cdot h + b_o & \hat{y} &= \sigma(z_o) \\
 \text{Loss: } E &= \frac{1}{2}(t - \hat{y})^2
 \end{aligned}$$

Numeric example (2 inputs, 2 hidden neurons, 1 output)

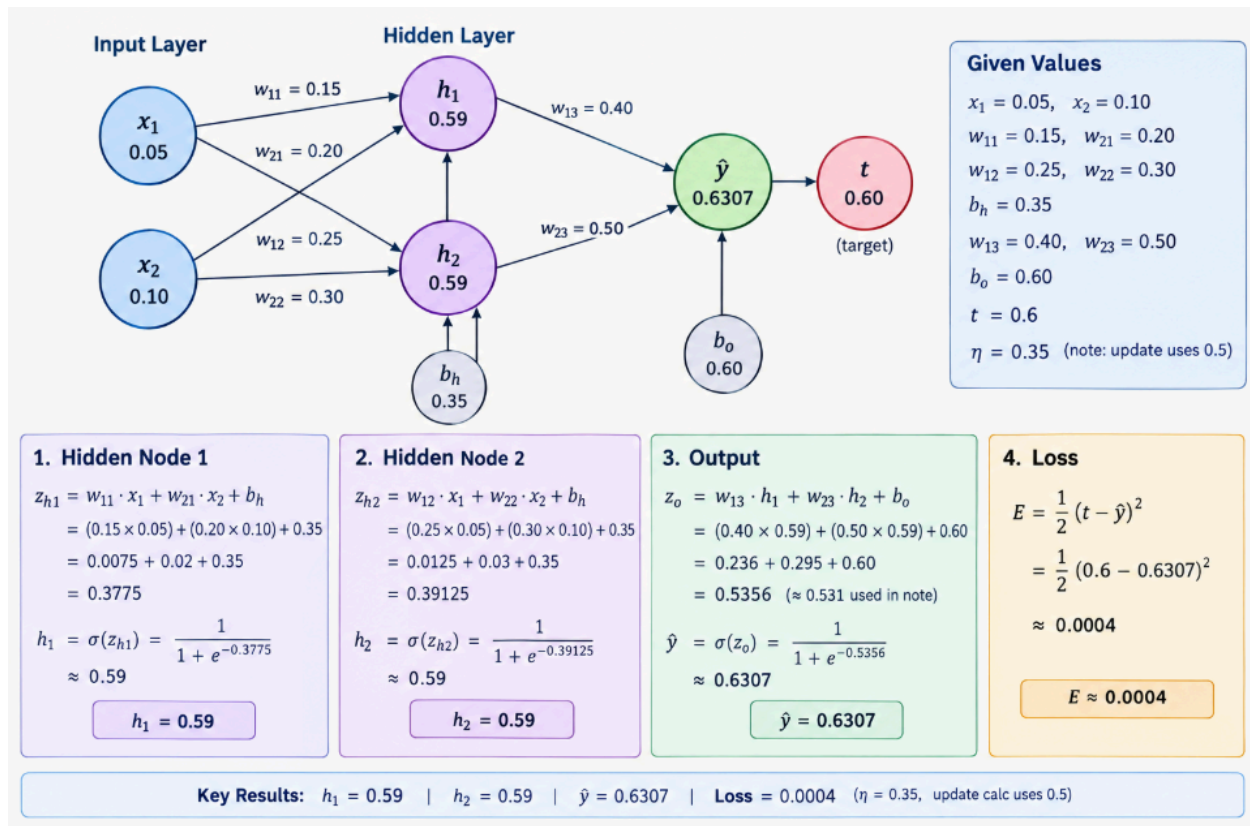
Given:

$$x_1 = 0.05, x_2 = 0.10$$

$$w_{11} = 0.15, w_{21} = 0.20 \quad (\text{weights into hidden node 1})$$

$$w_{12} = 0.25, w_{22} = 0.30 \quad (\text{weights into hidden node 2})$$

$b_h = 0.35$
 $w_{13} = 0.40, w_{23} = 0.50$ (weights hidden→output)
 $b_o = 0.60$
 $t = 0.6, \eta = 0.35$ (note: worked calc below uses $\eta = 0.5$ for the update)



Small error already (weights are near-decent starting point in this toy example — normally starts larger).

3. Backward Pass (Backpropagation) — full derivation

Core idea: Backprop = repeated chain rule. We want $\partial E / \partial w$ for every weight so we can update it. Compute gradient of E w.r.t. output first, then push backward layer by layer — each layer only needs the gradient signal from the layer *after* it (its "local gradient" recipe), never has to redo work.

3.1 Output layer error signal (δ_o)

Chain: $w_{ho} \rightarrow z_o \rightarrow \hat{y} \rightarrow E$
 $\partial E / \partial z_o = \partial E / \partial \hat{y} \times \partial \hat{y} / \partial z_o$

Derivative of squared-error loss:

$$\partial E / \partial \hat{y} = -(t - \hat{y}) = (\hat{y} - t)$$

Derivative of sigmoid (key identity — memorize this):

$$\partial \hat{y} / \partial z_o = \sigma(z_o) (1 - \sigma(z_o)) = \hat{y} (1 - \hat{y})$$

So the output error signal:

$$\delta_o = \partial E / \partial z_o = (\hat{y} - t) \cdot \hat{y} (1 - \hat{y})$$

This δ_o is the "blame" assigned to the output neuron's pre-activation — everything downstream (toward inputs) reuses it.

3.2 Gradient for output weight w_{ho}

Chain: $h \rightarrow w_{ho} \rightarrow z_o \rightarrow \hat{y} \rightarrow t$

$$\partial E / \partial w_{ho} = \partial E / \partial z_o \times \partial z_o / \partial w_{ho}$$

Since $z_o = w_{ho} \cdot h + b_o$:

$$\partial z_o / \partial w_{ho} = h$$

So:

$$\partial E / \partial w_{ho} = \delta_o \times h$$

Update rule:

$$w_{ho}(\text{new}) = w_{ho}(\text{old}) - \eta \cdot \delta_o \cdot h$$

3.3 Hidden layer error signal (δ_h)

Now push gradient one layer further back. Chain: $w_{xh} \rightarrow z_h \rightarrow h \rightarrow w_{ho} \rightarrow z_o \rightarrow \hat{y} \rightarrow t$

We want $\partial E / \partial z_h$:

$$\partial E / \partial z_h = \partial E / \partial z_o \times \partial z_o / \partial h \times \partial h / \partial z_h$$

Since $z_o = w_{ho} \cdot h + b_o$:

$$\partial z_o / \partial h = w_{ho}$$

Since $h = \sigma(z_h)$, same sigmoid-derivative identity:

$$\partial h / \partial z_h = h (1 - h)$$

So the hidden error signal:

$$\delta_h = \delta_o \times w_{ho} \times h (1 - h)$$

Interpretation: hidden neuron's error = output error, scaled by how much that hidden neuron influenced output (w_{ho}), scaled by hidden neuron's own sigmoid slope. This

is the "backpropagation" step proper — error flows backward through the same weight it went forward through.

3.4 Gradient for hidden weight w_{xh}

$$\partial E / \partial w_{xh} = \partial E / \partial z_h \times \partial z_h / \partial w_{xh}$$

Since $z_h = w_{xh} \cdot x + b_h$:
 $\partial z_h / \partial w_{xh} = x$

So:

$$\partial E / \partial w_{xh} = \delta_h \times x$$

Update rule:

$$w_{xh}(\text{new}) = w_{xh}(\text{old}) - \eta \cdot \delta_h \cdot x$$

3.5 Bias gradients

$z = w \cdot x + b \rightarrow \partial z / \partial b = 1$, so bias gradient = the error signal itself:

$$\begin{aligned} \partial E / \partial b_o &= \delta_o & b_o(\text{new}) &= b_o(\text{old}) - \eta \cdot \delta_o \\ \partial E / \partial b_h &= \delta_h & b_h(\text{new}) &= b_h(\text{old}) - \eta \cdot \delta_h \end{aligned}$$

3.6 General pattern (memorize this — it's the whole algorithm)

For ANY layer, weight gradient = (error signal of that layer's output) $\times$ (input that fed that layer).

$$\partial E / \partial W_{\text{layer}} = \delta_{\text{layer}} \times (\text{activation feeding into layer})$$

And error signal propagates backward via:

$$\delta_{\text{previous_layer}} = \delta_{\text{next_layer}} \times W_{\text{connecting_them}} \times (\text{local activation derivative})$$

This recursive structure is why it's called *back*-propagation — you compute δ at the output first (needs no recursion), then each earlier layer's δ is a cheap one-step extension of the layer after it. This reuse is what makes NN training tractable — without it you'd recompute the full chain rule product from scratch for every single weight.

3.7 Numeric example continued (w13 update, matches handwritten note)

Using values from Section 2: $h1 \approx 0.59$, $\hat{y} \approx 0.63$, $t = 0.6$, $\eta = 0.5$.

$$\partial E / \partial \hat{y} = (\hat{y} - t) = 0.63 - 0.6 = 0.03$$

$$\partial \hat{y} / \partial z_o = \hat{y}(1 - \hat{y}) = 0.63 \times 0.37 \approx 0.233$$

$$\delta_o = \partial E / \partial w_{13} \text{ chain uses:}$$

$$\begin{aligned}\partial E / \partial w_{13} &= (\hat{y} - t) \times \hat{y}(1 - \hat{y}) \times h_1 \\ &= (0.63 - 0.6) \times 0.63 \times (1 - 0.63) \times 0.59 \\ &\approx 0.004\end{aligned}$$

$$\begin{aligned}w_{13}(\text{new}) &= w_{13}(\text{old}) - \eta \times 0.004 \\ &= 0.4 - (0.5 \times 0.004) \\ &= 0.398\end{aligned}$$

Same pattern repeats for w_{23} , then propagates back to w_{11} , w_{21} , w_{12} , w_{22} using the δ_h formula above (multiply δ_o by w_{13} or w_{23} respectively, times $h(1-h)$, times the corresponding input x_1 or x_2).

4. Extra concepts worth knowing cold for exam

Why nonlinearity matters: stacking pure linear layers (no activation) collapses to one big linear layer — no representational gain. Nonlinearity is what lets NN approximate complex/non-linear functions (universal approximation).

Common activation functions:

Name	Formula	Derivative	Note
Sigmoid	$1/(1+e^{-x})$	$f(x)(1-f(x))$	saturates, vanishing gradient for large
Tanh	$2/(1+e^{-2x}) - 1$	$1 - f(x)^2$	zero-centered, still saturates
ReLU	$\max(0, x)$	1 if $x > 0$ else 0	usually best in practice, no saturation for $x > 0$
Binary step	0 or 1	0 (undefined at 0)	not differentiable, not used in practice

Vanishing gradient: sigmoid/tanh derivative max is 0.25/1.0 respectively and shrinks near saturation — multiplying many such small numbers across deep networks shrinks gradient toward 0 in early layers. Main motivation for ReLU family and for architectures like ResNet.

Non-convexity: NN loss surface has many local minima (roughly similar quality in high dimensions) — no guarantee of global optimum, but works well in practice anyway.

"Block view" mental model: every layer = differentiable block with a forward (compute output from input) and backward (compute input-gradient from output-gradient) method. This is literally how frameworks like PyTorch implement autograd.

Feature	Traditional Machine Learning	Deep Learning
Feature Extraction	Manual: Requires domain experts to identify and engineer features from data.	Automatic: Neural network layers learn features automatically from raw data.
Data Dependency	Performs well on smaller, tabular datasets; performance plateaus with massive data.	Scales significantly as dataset size increases; thrives on massive, unstructured data.
Algorithms	Linear Regression, Decision Trees, Random Forests, SVMs, XGBoost.	Multilayer Perceptrons (MLPs), CNNs, RNNs, Transformers.
Hardware	Runs efficiently on standard CPUs.	Typically requires specialized hardware (GPUs/TPUs) for high-parallel computation.
Interpretability	Higher ("White-box"): Decision paths are usually easier to audit and explain.	Lower ("Black-box"): Complex non-linear internal states make decisions hard to trace.

5. What to study next (after basic NN) to round out ML fundamentals

A. Classical ML foundations (if not solid yet)

- Linear regression, logistic regression (you already touched this — NN's building block)
- Bias-variance tradeoff, overfitting vs underfitting
- Regularization: L1/L2, weight decay, dropout
- Cross-validation, train/val/test splits
- Evaluation metrics: accuracy, precision/recall, F1, ROC-AUC, confusion matrix

B. Optimization deeper

- SGD variants: Momentum, Nesterov, RMSProp, Adam

- Learning rate schedules, warmup
- Batch size effects, mini-batch gradient descent
- Weight initialization (Xavier/Glorot, He initialization)
- Batch normalization, layer normalization

C. Architectures (natural next step from MLP)

- CNNs (Convolutional Neural Networks) — images, weight sharing, pooling
- RNNs / LSTMs / GRUs — sequences, hidden state
- Attention mechanism → Transformers (modern default for NLP/vision)
- Autoencoders, embeddings

D. Practical training concepts

- Gradient checking, exploding/vanishing gradients, gradient clipping
- Data augmentation
- Transfer learning, fine-tuning pretrained models
- Hyperparameter tuning

E. Math prerequisites to shore up

- Linear algebra: matrix/vector calculus, Jacobians (backprop is literally chained Jacobian-vector products)
- Probability & statistics: MLE, cross-entropy's connection to likelihood, Bayes basics
- Multivariable calculus: partial derivatives, chain rule (you just used this heavily)

F. Frameworks (hands-on)

- PyTorch or TensorFlow — implement the exact backprop above by hand once, then let autograd do it